Đã cập nhật trực tiếp `sol-lean-codex` trên PC tại:

`C:\Users\Admin\.codex\skills\sol-lean-codex`

Kết quả xác minh:

- Package SHA-256 khớp bản mới:
  `1BF51FF16F350C61567F2C780FEBBE8BE41EB6CDDB225997A79DD7E2660320D0`
- Skill validator: `Skill is valid!`
- Comparator unit tests: **9/9 đạt**
- Python compile: **đạt**
- Đã xác nhận các thành phần v2 như fast-path routing, integration width, environment readiness, `TIME_TO_FIRST_DISPATCH`, maintainability gate, `run_id` và task-class evaluation.
- Skill sau cleanup còn đúng **12 file**.
- Không chỉnh sửa source HR/PFM hay WIP hiện tại.

Tôi cũng giữ bản rollback tại:

`C:\Users\Admin\.codex\skills\sol-lean-codex.backup-20260812-1045`

Bản global trên PC hiện đã là **Sol–Terra–Luna v2 tối ưu mới nhất**.

phối **Sol – Terra – Luna** có kiểm soát chất lượng và ch

Nghiên cứu phân tích thêm việc chạy woker hay tạo session khác điều phối thì cái gì hiệu quả và tiết kiệm hơn. Dùng các nghiên cứu khoa học thuật toán mới và các thảo luận trên các diễn đàn không gian Mạng.

Worked for 1m 34s

Có. Sau khi đối chiếu các paper ACL/arXiv mới đến **tháng 8/2026**, tài liệu OpenAI hiện tại và các thảo luận thực tế trên GitHub, OpenAI Developer Community và Reddit, kết luận của tôi thay đổi thêm một bước:

> **Không nên chọn "worker" hoặc "session riêng" làm mặc định duy nhất. Kiến trúc hiệu quả nhất hiện nay là *single authoritative coordinator + ephemeral workers + persistent lane sessions*, với router quyết định thời gian sống của agent.**

Và có một kết luận mạnh hơn: **tạo thêm một session để làm "co-coordinator" thường là lựa chọn tốn nhất**. Nếu mở session khác, nó nên sở hữu **một workstream/lane cụ thể**, không nên cùng Sol tranh quyền điều phối toàn cục.

1. Worker và session thực chất tối ưu hai loại chi phí khác nhau

Một worker/subagent mới có ưu điểm lớn là **context isolation**. Nó nhận packet nhỏ, làm một việc và trả kết quả nhỏ về coordinator. Với task một-shot, đây gần như luôn là hình dạng kinh tế nhất: không phải giữ một context dài sống qua nhiều turn, ít nguy cơ "context rot", và có thể dùng tier rẻ hơn nếu runtime thực sự pin được Luna/Terra.

Kinh nghiệm cộng đồng Codex cũng phản ánh rất rõ điều này. Một thread Reddit gần đây khuyến nghị worker dùng **fresh context**, không fork toàn bộ history cha; lý do là nếu child kế thừa cả lịch sử Sol thì lợi thế token của delegation gần như mất đi. Red… Một thảo luận khác báo cáo đúng failure mode ngược lại: subagent vô tình inherit model Sol của parent và tiêu quota rất nhanh; tác giả chỉ kiểm soát được chi phí sau khi xác minh model thực sự chạy ở child. Đây là bằng chứng thực địa, không phải benchmark khoa học, nên tôi xem nó như runtime signal chứ không coi là quy luật phổ quát. Red…

Session riêng giải quyết bài toán khác: nó amortize chi phí bootstrap khi cùng một workstream phải tiếp tục qua nhiều vòng `investigate → implement → test → repair → review`. Thay vì mỗi worker mới phải được gửi lại contract, relevant files, accepted decisions và evidence, một lane session giữ trạng thái cục bộ và chỉ cần nhận delta.

Điều này khớp khá sát với MTRouter tại ACL 2026: router multi-turn tốt hơn không chỉ nhờ chọn model rẻ mà còn vì nó **switch model ít hơn**, chịu transient errors tốt hơn và hình thành specialization theo model. MTRouter báo cáo giảm tổng chi phí 58.7% trên ScienceWorld và 43.4% trên HLE trong thiết lập của họ. Không nên chuyển trực tiếp các con số đó sang Codex, nhưng cơ chế rất đáng chú ý: **continuity trong một trajectory có giá trị kinh tế**. ACL Anthol…

Do đó:

Hình dạng công việc	Route ưu tiên
1 leaf, kết quả nhỏ, oracle mạnh	**Ephemeral worker**
1 module/lane có nhiều vòng sửa và kiểm chứng	**Persistent session**
Sequential/tool-dense, mỗi kết quả đổi quyết định kế tiếp	**Giữ trên coordinator/single lane**
Nhiều lane độc lập dài hạn	**Nhiều persistent lane sessions**, một coordinator
Nhiều micro-task cơ học	**Deterministic/programmatic tools**, không worker
Review độc lập để tránh bias	**Fresh worker/session**, không reuse implementer
Coordinator bị compaction/context rot	**Checkpoint → fresh coordinator session**, không cố kéo dài vô hạn
Hai coordinator cùng quyết định architecture	**Tránh mặc định**

2. Điểm rất quan trọng: “session reuse” không đồng nghĩa với “luôn rẻ”

OpenAI hiện hướng dẫn GPT-5.6 dùng persisted reasoning qua nhiều turn khi **goals, assumptions và priorities vẫn ổn định**, nhưng chuyển sang `current_turn` khi

- Chi phí thay đổi chính:

reasoning cũ không còn relevant. Họ cũng cảnh báo long sessions khuếch đại prompt/tool content lặp lại. OpenAI Develo…

Điều này gần như cung cấp trực tiếp một rule cho skill:

Reuse session theo semantic continuity, không theo identity.

Ví dụ Terra vừa trace `PFM refresh → attributed cost → read service` và test fail quay lại cùng causal chain: reuse Terra session có lợi.

Nhưng Terra vừa hoàn tất PFM và task tiếp theo là HR attendance UI: dù cùng repo, reuse session đó chỉ kéo theo nhiễu. Tạo fresh session với state capsule nhỏ sẽ tốt hơn.

Một nghiên cứu cực mới ngày **6/8/2026**, TRACE, còn phát hiện repeated context compression có thể làm suy yếu ảnh hưởng của recent interactions, dẫn đến blocked actions, exploration lặp lại và execution instability. Điều này là bằng chứng chống lại kiểu "giữ một giant session sống mãi rồi compact liên tục". ar…

Các issue Codex cũng cho thấy người dùng đang gặp đúng hiện tượng này: có đề xuất chính thức hóa thao tác "clear context nhưng giữ previous session ID", vì sau khi hoàn thành một issue thì file reads, command output và intermediate details của issue đó trở thành stale baggage đối với issue kế tiếp. GitHub

Vì vậy tôi sẽ coi **task boundary là natural session boundary**.

---

3. Nhưng spawn worker liên tục cũng không phải tối ưu

Nếu một task cần 4 vòng sửa, cách:

`spawn Terra → kết quả → close → spawn Terra → gửi lại context → ...`

có một hidden tax khá lớn:

`packet reconstruction + repeated prefix + environment rediscovery + repeated file reads + parent integration`

Một issue Codex tháng 7 còn báo agent reread các file không thay đổi, gây token/latency/context pressure đặc biệt trong controller/subagent workflows; việc chỉ dẫn reuse verified reads giúp một case QA bảy tài liệu dùng ít context hơn đáng kể. Đây là báo cáo người dùng, nhưng nó chỉ ra một overhead mà telemetry của skill nên đo. GitHub

Tôi sẽ mô hình hóa nó như sau:

$$C_{\text{ephemeral}} = \sum_{i=1}^{N} (S_w + P_i + R_i + X_i + V_i + I_i)$$

trong đó `S_w` là spawn/setup, `P` packet/context, `R` rediscovery, `X` execution, `V` validation và `I` integration.

Còn persistent lane:

$$C_{\text{lane}} = S_s + P_0 + \sum_{i=1}^{N}(\Delta_i + X_i + V_i) + M + I$$

`Δ` chỉ là evidence delta, còn `M` là memory/compaction/staleness cost.

Với `N=1`, worker gần như chắc chắn thắng.

Khi `N` tăng, persistent session bắt đầu thắng **nếu assumptions vẫn còn valid**, vì `S + P + R` được amortize.

Nhưng đến một điểm nào đó, `M` tăng nhanh do context bloat/compaction, và fresh session lại thắng.

Nói cách khác, chi phí session có dạng gần giống **U-curve**:

fresh quá thường → bootstrap tax cao
giữ quá lâu → stale-context tax cao

Skill nên tìm đáy của đường cong đó bằng telemetry.

---

4. Thuật toán mới tôi đề xuất: Lane Affinity Routing

Đây là phần tôi nghĩ đáng đưa trực tiếp vào `sol-lean-codex`.

Trước khi spawn một worker, Sol không chỉ hỏi “Luna/Terra/Sol?” mà hỏi thêm:

`NEW_WORKER | REUSE_LANE | NEW_LANE_SESSION | KEEP_LOCAL`

Một **Lane Affinity Score** có thể dựa trên:

$$A = semantic\ continuity + context\ reuse + expected\ followups + environm$$

Không cần tính số thực trong runtime; low/medium/high đủ.

Nếu `expected_followups ≈ one-shot`, affinity thấp → fresh worker.

Nếu cùng failure, same write ownership, same interface và nhiều correction rounds → reuse session.

Nếu task liên quan nhưng session hiện tại đã nhiễu/compacted → **new lane session + capsule**, không resume history toàn bộ.

Nếu integration width cao → không trao ownership cho lane, Sol giữ lại.

Điều này còn phù hợp với LLM-as-Scheduler: paper này dùng lightweight gate trước scheduler nặng và giảm 43% token, hơn 36% latency so với fixed heavy workflow trong benchmark của họ. Ý tưởng quan trọng đối với skill là **không chạy full orchestration logic trên mọi packet**. ACL Anthol...

---

5. Tôi sẽ không tạo "Sol coordinator thứ hai" mặc định

Đây là chỗ tôi thay đổi khá mạnh so với cách nhiều người vận hành agent hiện nay.

Hai coordinator đều có architecture context sẽ tạo:

`duplicated architecture reasoning`
+
`duplicate repository context`
+
`decision reconciliation`
+
`conflicting interface ownership`
+
`extra final integration`

Tức là đang trả tiền hai lần cho phần **đắt nhất**.

DeLM tháng 6/2026 có một kết quả rất thú vị: centralized coordinator có thể trở thành communication/integration bottleneck khi số subtasks tăng. Nhưng giải pháp của họ **không phải thêm coordinator thứ hai**. Họ dùng parallel agents + **shared verified context + task queue**, để agent tự claim task và viết compact verified updates vào shared substrate. Trên SWE-bench Verified, thiết lập của họ báo cáo giảm cost/task khoảng 50% so với baseline mạnh nhất được so sánh. ar...

Đây là khác biệt rất quan trọng.

Nếu Sol trở thành bottleneck, nên chuyển từ:

`worker → Sol → worker → Sol → worker`

sang:

`Sol defines invariants`
→ `shared verified state`
→ `lane sessions operate asynchronously`
→ `Sol integrates only material decisions`

Không nên thành:

`Sol A ↔ Sol B ↔ workers`

trừ khi hai Sol sở hữu hai acceptance domains thực sự độc lập.

Tôi sẽ gọi pattern này là **Blackboard Lane Architecture**.

Repository nên là blackboard: accepted contract, task state, proof, ownership, latest revision. OpenAI cũng đang dùng một triết lý gần như vậy trong harness engineering của họ: execution plans và knowledge được version hóa trong repo để agent hoạt động mà không phụ thuộc vào external conversational context; họ mô tả các single Codex runs kéo dài tới nhiều giờ nhưng vẫn dùng repository artifacts làm source of truth. OpenAI +1

---

6. Worker còn có một vấn đề runtime mà session riêng khắc phục được

Hiện nay **subagent route không hoàn toàn đáng tin cậy qua mọi runtime**.

Có issue cho thấy `resume_agent` khôi phục history của child nhưng có thể **không khôi phục model/reasoning settings ban đầu**, khiến child vốn được pin model rẻ hơn có thể resume bằng model/default của parent. GitHub

Cộng đồng tháng 7 cũng phản ánh các build nơi subagents inherit parent model, khiến Sol parent sinh Sol children ngoài ý muốn. Red…

Trong trường hợp runtime **không chứng minh được resolved child model/effort**, một separately configured session có một lợi thế lớn:

routing observability > spawn convenience.

Nghĩa là thà tốn startup thêm nhưng chắc chắn lane đó là `Terra high` còn hơn tưởng rằng đang chạy Luna nhưng thực tế đang chạy Sol.

Đây là lý do skill hiện tại đúng khi coi requested route và resolved route là hai sự kiện khác nhau.

---

7. Persistent session đặc biệt đáng giá khi parent thread bị blocking

Có một limitation Codex hiện tại rất thực dụng: user report cho thấy khi parent đang chờ subagent, main thread có thể cảm giác **effectively serial**—follow-up bị queue cho turn sau, và interrupt parent có thể làm child result khó được tổng hợp lại. GitHub

GitHub Discussion về multi-agent cũng có người báo synchronous orchestration làm agents bị interrupt/derail; một practitioner mô tả chuyển sang async agents + shared message/state giúp agents làm việc độc lập hơn, đổi lại polling latency nhỏ. Đây là anecdotal evidence, nhưng rất phù hợp với các kết quả critical-path scheduling từ research. GitHub

LAMaS tháng 7 nhấn mạnh chính xác điều đó: **end-to-end latency được quyết định bởi critical execution path**, nên giảm tổng cost chưa chắc giảm thời gian hoàn thành. Framework của họ loại các future interactions dư thừa trong runtime và báo cáo giảm latency trên các benchmark của nghiên cứu. ar...

Do đó, nếu một Terra lane đang chạy 30 phút nhưng Sol hoàn toàn có thể tiếp tục giải quyết architecture hoặc integrate lane khác, **separate asynchronous session tốt hơn synchronous child orchestration**.

---

8. Một nuance quan trọng: session riêng không nên chia sẻ toàn bộ cache với final judge

Có một paper ACL 2026 khá liên quan tới integration. Nghiên cứu về KV-cache reuse trong multi-agent judge cho thấy cache reuse hiệu quả ở generator **không đảm bảo** an toàn khi judge phải so sánh nhiều candidate; final accuracy có thể nhìn vẫn ổn nhưng lựa chọn của judge thay đổi đáng kể vì cross-candidate attention bị phá vỡ. ACL Anthol...

Ý nghĩa đối với `sol-lean-codex`:

Worker/session có thể reuse context cục bộ. Sol final acceptance không nên "thừa kế" state nội bộ của từng worker như thể nó là evidence chung.

Sol nên nhận:

```
diff + invariant + test/proof + material assumptions
```

rồi thực hiện integration reasoning trong một context sạch đủ để thấy **các candidate/lane cùng lúc**.

Đây là lý do tôi vẫn giữ final acceptance trên Sol.

Kiến trúc tôi đề xuất cho bản tiếp theo

Tôi sẽ đổi kiến trúc từ:

Sol → spawn Luna/Terra workers

thành:

Sol Control Plane → Execution Lanes → ephemeral leaves

Cụ thể:

Sol coordinator session sống tương đối dài nhưng chỉ giữ architecture, invariants, route ledger, ownership và acceptance. Không nhồi logs/file contents.

Terra lane session được tạo khi một workstream có khả năng cần nhiều vòng. Ví dụ `backend PFM`, `frontend PFM`, `CI/environment diagnosis`. Session này sống đến khi lane đạt acceptance hoặc assumptions đổi.

Luna ephemeral worker làm leaf one-shot có strong oracle. Không giữ Luna session dài mặc định; nếu có hàng chục việc Luna giống nhau thì chuyển sang deterministic batching trước.

Fresh Terra reviewer dùng cho semantic review độc lập. Không reuse implementation session nếu mục tiêu là independent evidence.

Shared state capsule/blackboard nằm ngoài transcript: revision, interface version, ownership, accepted evidence, proof. Các lane truyền state qua đây thay vì gửi narrative qua lại.

Session rollover xảy ra khi task boundary đổi, interface/base thay đổi, repeated rereads xuất hiện, compaction làm mất recent state, hoặc context chứa nhiều history không còn quyết định hành vi. Không cố cứu giant session mãi.

Kết luận cho PC/workflow của bạn

Nếu mục tiêu là **chất lượng cao nhất trên chi phí hợp lý**, tôi sẽ không để Sol spawn worker mới cho mọi việc.

Policy tôi ưu tiên là:

one Sol coordinator + 1–2 persistent Terra lanes + ephemeral Luna leaves + deterministic tools.

Chỉ tăng lane khi critical path thực sự có work độc lập.

Và đặc biệt:

Không tạo Sol session thứ hai để "cùng điều phối" chỉ vì task lớn.
Nếu cần session thứ hai, hãy biến nó thành **owner của một lane**, với contract và worktree riêng.

Session khác trở nên **kinh tế hơn worker** khi cùng context sẽ được sử dụng qua nhiều meaningful turns; worker trở nên **kinh tế hơn session** khi task là one-shot, disposable và evidence nhỏ.

Đây là thay đổi tôi cho rằng đáng đưa tiếp vào `sol-lean-codex`: thêm một routing dimension mới là **LIFETIME / LANE_AFFINITY**, để router không chỉ quyết định *model nào chạy*, mà còn quyết định **agent đó nên sống bao lâu**. Đây có khả năng giảm chi phí lớn hơn việc tiếp tục tinh chỉnh riêng `Luna vs Terra`, vì nó tấn công trực tiếp repeated context, repeated discovery và coordination turns.

Nghiên cứu thêm trước khi giải quyết. Hiện tại tôi đang dùng C Coding Tools MCP để kết nối với chatweb để xử lý song hành cùng codex. Việc tận dụng này giảm chi phí về lượng token và có sự phản biện tương đối chính xác, tuy chat phản hồi sẽ chậm. Tìm nghiên cứu khoa học thuật toán và tư duy phân tích cùng các nguồn bạn đánh giá sao về việc đưa thêm cái này vào trong bộ skill

Worked for 1m 34s

Công cụ được gọi

Có cơ sở khá mạnh để **đưa mô hình ChatWeb +** `@Coding Tools MCP` **+ Codex vào** `sol-lean-codex`, nhưng tôi sẽ không coi ChatWeb là "một worker nữa". Cách phân loại đúng hơn là **External Critic / Research Plane**: một mặt phẳng suy luận độc lập, có thể quan sát cùng workspace qua MCP nhưng không nằm trong trajectory nội bộ của Codex.

Sau khi đối chiếu các nghiên cứu mới, tôi đánh giá mô hình bạn đang vận hành có tiềm năng tốt hơn `Codex → subagent reviewer` trong một số lớp task, đặc biệt khi mục tiêu là **giảm Codex-metered tokens, tăng độ độc lập của phản biện, và chấp nhận latency cao hơn**.

Vì sao kiến trúc hiện tại của bạn có giá trị thật

Có ba lợi thế khác nhau đang xuất hiện cùng lúc.

Thứ nhất là **context isolation**. ChatWeb không phải mang toàn bộ trajectory của Codex: hàng loạt file read, command output, failed hypotheses và tool traces có thể ở lại phía Codex. ChatWeb chỉ cần request, trạng thái workspace và một lượng evidence nhỏ cần thiết để kiểm tra. Đây chính là kiểu giảm duplicated context mà skill hiện tại đang cố đạt bằng packet nhỏ và state capsule.

Thứ hai là **independent interpretation**. Một paper rất mới ngày 9/8/2026, RETRACE, nghiên cứu verification cho coding agents và chỉ ra vấn đề của self-review: reviewer cùng trajectory thường kiểm tra patch dưới cùng cách hiểu đã sinh ra patch. RETRACE cố ý tạo một reconstruction độc lập rồi so ngược với issue ban đầu; trên SWE-bench Verified, cơ chế này tăng Pass@1 thêm 7.0 điểm phần trăm với một backbone và 3.6 điểm với backbone còn lại. Mô hình ChatWeb của bạn có một đặc tính tương tự: nếu không truyền reasoning/narrative của Codex sang, ChatWeb có thể hình thành một cách hiểu độc lập về diff và requirement. ar...

Thứ ba là **heterogeneity**. PairCoder 2026 dùng đúng hai role là coder và reviewer, chuyển role khi vòng sửa bị kẹt. Trên 13 LLMs, framework này cải thiện so với single-model inference; với tám backbone đại diện, họ báo cáo giảm **40–70% token usage so với các multi-agent baseline** trong khi cải thiện chất lượng. Điều quan trọng không phải con số có thể áp thẳng vào workflow của bạn, mà là kiến trúc hai-role có thể hiệu quả hơn swarm nhiều agent. ACL Anthol...

Điều này cũng phù hợp với nghiên cứu debate mới. Vanilla debate giữa nhiều agent đồng dạng thường không đáng tiền; diversity của góc nhìn và confidence có calibration mới là thứ làm debate hữu ích. Nói cách khác, thêm "một Codex nữa nói lại cùng suy nghĩ" ít giá trị hơn một reviewer có context và nhiệm vụ khác. ACL Anthol...

Vì sao ChatWeb phản hồi chậm vẫn có thể là một trade tốt

Ta cần tách **latency tổng** khỏi **critical-path latency**.

Nếu workflow là:

```
Codex làm xong → gửi ChatWeb → chờ → sửa
```

thì latency là gần:

$$L \approx L_{Codex} + L_{ChatWeb} + L_{repair}$$

Đây rõ ràng chậm.

Nhưng nếu ChatWeb review/research chạy **song hành** với Codex:

```
Codex implementation || ChatWeb investigation/review
```

thì:

$$L_{critical} \approx \max(L_{Codex}, L_{ChatWeb}) + L_{integration}$$

Nếu Codex mất 8 phút và ChatWeb mất 5 phút, 5 phút đó gần như không nằm trên critical path.

Google Research đánh giá 180 cấu hình agent và cũng thấy multi-agent sinh lợi chủ yếu khi work thực sự parallelizable; centralized coordination giúp mạnh trên các task có thể phân rã, nhưng multi-agent làm giảm 39–70% performance trên các task reasoning tuần tự và chịu coordination tax lớn ở tool-heavy workloads. arXiv +1

Do đó, **ChatWeb chậm không phải vấn đề lớn nếu nó được đặt ngoài critical path**. Đây là chỗ tôi thấy kiến trúc của bạn có lợi hơn việc Codex dừng lại và gọi một subagent synchronously.

Nhưng không nên đo chỉ bằng token Codex

Đây là một điều tôi sẽ sửa trong cách đánh giá skill.

Nếu ChatWeb làm mất 20k token và nhờ đó Codex giảm 30k token, từ góc nhìn quota Codex bạn đã tiết kiệm 30k. Nhưng về mặt hệ thống, tổng compute không giảm 30k; nó đã được chuyển sang mặt phẳng khác.

Vì vậy cần hai metric riêng:

$$C_{user} = Codex\ marginal\ cost + ChatWeb\ marginal\ cost$$

và

$$C_{system} = CodexTokens + ChatWebTokens + MCPOverhead + Rework$$

Nếu ChatWeb nằm trong subscription và marginal token cost với bạn gần bằng 0, thì đây là **economic arbitrage hợp lý đối với người dùng**, ngay cả khi aggregate compute không thấp hơn nhiều.

FLARE và EvoRoute 2026 đều cho thấy routing tốt phải xem đồng thời quality, monetary/resource cost và latency thay vì tối ưu một metric duy nhất. EvoRoute thậm chí mô tả nó thành “Agent System Trilemma”: performance, cost và latency kéo ngược nhau. ACL Antholo… +1

Tôi sẽ giữ triết lý Pareto này thay vì tuyên bố “ít Codex token hơn = tốt hơn”.

Một điểm tôi đánh giá rất cao: ChatWeb có thể làm blind reviewer

Đây có thể là lợi ích lớn nhất của toàn bộ setup.

Không nên gửi sang ChatWeb:

> “Codex đã xác định nguyên nhân là X, đã sửa Y, đây là lý do tại sao patch đúng…”

Vì như vậy ChatWeb rất dễ bị anchoring.

Nên gửi:

`Requirement + acceptance criteria + base revision + changed files/diff + test results`

rồi yêu cầu ChatWeb tự trả lời:

> Patch này đang giải quyết vấn đề gì? Có thực sự khớp requirement không? Counterexample nào còn tồn tại?

D3 2026 cho thấy role diversification, adversarial evaluation, anonymization và budgeted stopping giúp giảm các bias như positional/verbosity bias và cải thiện agreement với human judgments. ACL Anthol…

Agentic Rubrics cho SWE agents cũng rất liên quan: một expert agent đọc repository để tạo các tiêu chí codebase-specific rồi dùng tiêu chí đó để đánh giá candidate patch. Họ báo cáo rubric agent có thể phát hiện vấn đề mà tests không thể hiện, trong khi vẫn tạo verification signal có khả năng scale. ACL Anthol…

Vì vậy tôi muốn ChatWeb chủ yếu là **counterexample/rubric generator**, không phải "second programmer".

Cách tận dụng MCP là đúng, nhưng shared workspace tạo thêm một loại rủi ro

ChatWeb và Codex nhìn cùng filesystem giúp tránh phải copy code qua transcript. Đây là một dạng **externalized shared memory / blackboard**, rất đáng giá.

Nhưng paper về governed shared memory năm 2026 xác định bốn failure mode quan trọng trong multi-agent memory: stale propagation, contradiction persistence, provenance collapse và scope leakage. Nói cách khác, "cùng nhìn filesystem" không tự động đồng nghĩa "cùng nhìn một trạng thái đúng". ar…

Ví dụ ChatWeb bắt đầu review revision A.

Trong lúc đó Codex sửa tiếp thành revision B.

ChatWeb trả về finding dựa trên A.

Sol áp finding đó lên B.

Đây chính là **stale reviewer problem**.

Do đó mọi external-review packet phải mang:

```
BASE_REV:
WORKTREE:
REVIEW_STARTED_AT:
TARGET_DIFF:
```

```
ACCEPT:
PROOF_SNAPSHOT:
```

Khi ChatWeb trả về, trước khi sử dụng result phải kiểm tra:

```
CURRENT_REV == BASE_REV ?
```

Nếu không bằng, finding chỉ là evidence tham khảo hoặc phải revalidate trên delta mới.

Đây là một rule mà hiện tại skill chưa formalize đủ.

Các thảo luận Codex/MCP thực tế cũng ủng hộ hướng này, nhưng cảnh báo về lifecycle

Trong repo Codex đã có yêu cầu về một **shared workspace/message bus** cho các subagent để chúng chỉ trao đổi findings/status/handoff ngắn thay vì mọi thông tin đều phải đi qua parent. Đó gần như là dạng giản lược của mô hình mà bạn đang tự xây qua MCP. GitHub

Ngược lại, người dùng cũng báo cáo MCP server bị khởi tạo theo từng session gây slowdown/memory pressure; có đề xuất dùng project-scoped MCP process pool thay vì tạo stack mới cho mỗi chat. GitHub +1

Điều này dẫn tới một rule khác cho skill:

External critic plane phải reuse một MCP capability/session ổn định khi có thể; không spawn một MCP stack mới cho mỗi vòng phản biện.

Codex community cũng phản ánh orchestrator interrupt subagents có thể làm wall-clock xấu hơn, cho thấy việc để external plane chạy độc lập/nonblocking có lợi hơn ép nó vào synchronous parent-child loop. GitHub

Nghiên cứu còn chỉ ra rằng không nên gọi ChatWeb mọi task

Đây là phần quan trọng nhất nếu muốn thực sự tiết kiệm.

LLM-as-Scheduler cho thấy workflow nặng có verification/multi-agent trên **mọi query** là lãng phí. Họ dùng lightweight gate trước scheduler nặng và báo cáo giảm 43% token, hơn 36% latency với suy giảm accuracy tối đa 1.4 điểm phần trăm trên benchmark của họ. ACL Anthol...

LLMRouterBench với hơn 400k instances cũng phát hiện nhiều router phức tạp không reliably thắng simple baseline và ensemble lớn có diminishing returns. ACL Anthol...

Do đó pattern đúng không phải:

`Codex task → ChatWeb review ALWAYS`

mà là:

```
Codex task → cheap gate → maybe ChatWeb
```

Tôi sẽ chỉ bật ChatWeb khi **expected value của independent evidence** đủ cao.

Nếu đưa vào skill, tôi đề xuất một module mới: `External Critic Plane`

Tôi chưa sửa skill trong lượt này vì bạn yêu cầu nghiên cứu thêm trước. Nhưng về thiết kế, tôi cho rằng nên thêm, với các nguyên tắc sau:

- **Codex vẫn là execution plane.** Coding Tools MCP chủ yếu cho ChatWeb read/search/test/inspect workspace; write mặc định thuộc Codex để tránh dual-writer conflict.
- **ChatWeb là shadow critic/researcher**, không phải co-coordinator toàn thời gian. Nó tìm counterexample, kiểm tra architecture/invariant, nghiên cứu nguồn ngoài hoặc tạo rubric.
- **Blind review là mặc định.** Không gửi narrative của Codex nếu reviewer có thể tự đọc requirement + repo + diff.
- **Gate external review.** Dùng khi oracle yếu/partial, integration width medium-high, contract/architecture thay đổi, Codex đã sửa sai một vòng, hoặc một material risk chưa được deterministic proof bao phủ.
- **Run asynchronous khi có thể.** ChatWeb review song hành với implementation/investigation để latency của nó không nằm trọn trên critical path.
- **Pin provenance.** Mọi finding phải ghi base revision/path/test/evidence; reject hoặc revalidate finding từ stale base.
- **Budgeted stopping.** Một review pass mặc định; re-review chỉ finding đã khiến patch thay đổi. Không tạo debate vô hạn.
- **Final acceptance vẫn chỉ có một owner.** ChatWeb cung cấp evidence; Sol/Codex coordinator quyết định integration/go-no-go. Không để hai control plane đồng thời thay đổi architecture.

Đây không phải swarm. Tôi sẽ gọi nó **asymmetric two-plane orchestration**.

Routing rule tôi nghĩ phù hợp hơn hiện tại

Skill hiện tại đang route:

```
tools → Luna → Terra → Sol
```

và quản lý worker/session lifetime.

Tôi sẽ thêm một dimension độc lập:

$$ReviewPlane \in \{NONE,\ INTERNAL,\ EXTERNAL_MCP\}$$

Tức là **model route** và **review route** là hai quyết định khác nhau.

Ví dụ:

```
Executor = Terra
Review = NONE
```

cho một bounded UI change có tests mạnh.

```
Executor = Terra
Review = EXTERNAL_MCP
```

cho cross-module implementation có partial oracle.

```
Executor = Luna
Review = deterministic only
```

cho mechanical migration.

```
Executor = Sol
Review = EXTERNAL_MCP blind adversarial
```

cho architecture hoặc financial semantics khó kiểm chứng.

Đây là điểm kiến trúc tôi nghĩ sẽ nâng skill rõ nhất.

Cần benchmark chính setup của bạn trước khi bật mặc định

Tôi sẽ không lấy benchmark paper rồi giả định ChatWeb+Codex của bạn chắc chắn tiết kiệm.

Ta cần A/B cùng một tập task:

A: Codex alone
B: Codex + native Codex reviewer
C: Codex + ChatWeb/MCP review mọi task
D: Codex + **gated asynchronous ChatWeb/MCP**

Và đo thêm các trường:

```
CODEX_TOKENS
EXTERNAL_TOKENS_OR_USAGE
MCP_CALLS
TIME_TO_FIRST_EXTERNAL_FINDING
EXTERNAL_REVIEW_WALL_TIME
CRITICAL_PATH_DELAY
NOVEL_FINDINGS
```

```
ACCEPTED_FINDINGS
DUPLICATE_FINDINGS
PATCH_CHANGED_BY_REVIEW
DEFECT_CAUGHT_PRE_ACCEPTANCE
STALE_REVIEW_RESULTS
FINAL_QUALITY_GATE
TOTAL_WALL_TIME
```

Metric mà tôi đặc biệt muốn thêm là:

$$ReviewYield = \frac{accepted\ novel\ findings}{review\ cost}$$

và

$$LatencyTax = TotalWall_{with-review} - TotalWall_{baseline}$$

Nếu C tốn rất nhiều thời gian nhưng D giữ được phần lớn `ReviewYield`, thì **D chính là kiến trúc nên đưa vào production skill**.

Đánh giá hiện tại

Tôi đánh giá việc đưa `ChatWeb ↔ Coding Tools MCP ↔ Codex` vào skill là **nên làm**, nhưng dưới dạng **optional gated external critic plane**, không biến nó thành bước bắt buộc.

Lợi ích có cơ sở khoa học nhất không phải đơn thuần là “ChatWeb rẻ hơn”, mà là bốn hiệu ứng kết hợp:

independent context + heterogeneous reasoning + zero-copy workspace inspection + asynchronous execution.

Nếu thiết kế đúng, nó có khả năng giảm Codex context/token pressure đồng thời tăng xác suất bắt lỗi mà self-review bỏ sót. Nếu thiết kế sai thành hai agent cùng đọc mọi file, cùng giải mọi vấn đề và tranh nhau sửa code, nó sẽ trở thành **duplicate inference + latency + stale-state tax**.

Bước hợp lý tiếp theo cho `sol-lean-codex` không phải thêm nhiều agent hơn, mà là thêm **External Review Router + MCP provenance contract + blind-review protocol + telemetry để đo review yield**. Tôi cho rằng đây là nâng cấp có tiềm năng lớn hơn việc tiếp tục tối ưu riêng Luna/Terra routing.

yyjeqhc/webcodex bạn tham khảo code logic của tool MCP này, các vấn đề và đưa ra phân tích sâu hơn

Worked for 6m 40s

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Tôi đã đọc sâu hơn `yyjeqhc/webcodex`, gồm MCP dispatch, model/tool surface, Connector runtime, Session model, workspace isolation, auth, các issue gần đây và commit mới nhất trên `main` tại **6c5b7b2 ngày 12/8/2026**. Kết luận của tôi mạnh hơn trước:

> **Nên đưa ChatWeb + Coding Tools MCP vào `sol-lean-codex`, nhưng không định nghĩa nó là "worker". Nên định nghĩa nó là một `External Verification & Research Plane` dùng workspace làm blackboard, có consistency protocol riêng.**

WebCodex cho thấy kiến trúc này có cơ sở rất tốt. Đồng thời code cũng chỉ ra vài failure mode mà nếu không xử lý, lợi thế token của ChatWeb sẽ đổi thành stale review, duplicate work và latency.

1. Điều đáng học nhất từ WebCodex không phải bản thân MCP

Mental model của WebCodex là:

```
Online model / ChatWeb
        |
        | MCP
        v
WebCodex Server
        |
        | authenticated execution bridge
        v
Runner
        |
        v
Repository
```

Server giữ authentication, policy, routing và bounded evidence; Runner gần repository mới thực thi file/Git/process/test. Model không được quyền filesystem trực tiếp.

Điểm này quan trọng với setup của bạn vì nó tách thành ba thứ độc lập:

```
REASONING      = ChatWeb
EXECUTION      = MCP/Runner
SOURCE OF TRUTH = repository + Git + deterministic evidence
```

Không phải:

```
ChatWeb transcript = state
```

Và cũng không phải:

```
Codex transcript = state
```

Đây chính là architecture tôi muốn skill tiến tới.

---

2. WebCodex đang tối ưu token từ tầng tool surface, không chỉ prompt

Code có ba model surface riêng và **chỉ một surface được chọn lúc startup**:

`canonical_connector`
`local_coding`
`full_operator_runtime`

Nếu config xung đột, nó fail startup chứ không silently expose surface khác.

Đặc biệt, `canonical_connector` chỉ cho model thấy đúng **12 capability cấp cao**:

```
task_start
task_list
task_resume

files_list
files_read
files_search

edits_apply

checks_run
commands_run

task_review
task_cancel
task_finish
```

Trong khi internal runtime có nhiều tool hơn rất nhiều. Ngay `local_coding` cũng chỉ expose một working set coding đã được curate/rank, còn session management, persistent shell, project lifecycle, checkpoint, cleanup và operator management bị giấu khỏi model-facing surface.

Đây là một optimization rất đáng đưa vào tư duy của skill.

ToolScope, ACL 2026, tìm thấy việc merge tool dư thừa và chỉ retrieve các tool liên quan có thể cải thiện tool-selection accuracy từ **8.38% đến 38.6%** trên các benchmark họ thử nghiệm. ACL Anthol... Một nghiên cứu khác về tool-schema compression báo cáo giảm khoảng **44–50% schema tokens** và cho thấy ở context budget thấp, compression có thể biến một agent gần như không hoạt động thành hoạt động bình thường. ar...

Các thảo luận MCP ngoài cộng đồng cũng lặp lại cùng hiện tượng: loading hàng chục tool schemas upfront gây context bloat và selection noise. Đây là anecdotal evidence, nhưng hướng hoàn toàn tương thích với literature. Reddit +1

Điều này đặc biệt liên quan đến `@Coding Tools MCP` của bạn

Surface tôi đang được kết nối trong phiên này còn nén mạnh hơn: chỉ có **8 top-level tools**, trong đó `agent_workflow` là một macro-tool gom cả:

```
context gathering
+
patch/file creation
+
commands
+
tests
+
build
+
artifacts
+
Git diff
+
persistent task state
```

Đây có thể là lý do workflow của bạn tiết kiệm Codex tokens khá rõ: nhiều thao tác cơ học được "hạ xuống" MCP thay vì mỗi thao tác lại cần một reasoning turn.

Nhưng nó tạo một tradeoff mới:

> **Tool quá granular → schema/round-trip tax.**
> **Tool quá macro → che mất decision points.**

`agent_workflow` rất tốt khi `objective + scope + oracle` đã rõ. Tôi không muốn skill gọi nó ở chế độ execute quá sớm khi root cause còn mơ hồ.

Khi chưa rõ vấn đề, route tốt hơn là:

```
workspace_context
        ↓
discriminate / diagnose
        ↓
agent_workflow
```

chứ không phải:

```
vague problem
    ↓
giant agent_workflow(run everything)
```

---

3. Một thiết kế rất mạnh khác: state không nằm trong conversation

WebCodex phân biệt rất rõ:

MCP transport session
khác
Workflow Session
khác
Connector Task
khác
Action Audit Session.

Tài liệu của repo thậm chí cảnh báo Workflow Session và Action Audit Session là hai khái niệm “non-interchangeable”, không được suy luận hay cross-wire với nhau.

Ở MCP protocol 2026 stateless, WebCodex cố tình không phụ thuộc vào `Mcp-Session-Id` cho state nghiệp vụ. Durable work được giữ bằng những handle kiểu:

```
task_id
operation_id
execution_id
result_id
event_cursor
```

Đây là một thiết kế rất tốt cho ChatWeb ↔ Codex.

Thay vì ChatWeb phải nhớ:

> “40 lượt trước Codex đã thử X, sau đó test Y fail, rồi đổi Z…”

repository/control plane có thể giữ:

```
TASK
BASE
CURRENT RESULT
LATEST VALIDATION
CHANGED PATHS
UNRESOLVED FAILURE
NEXT ACTION
```

ChatWeb chỉ cần nhận delta.

Đó mới là **token saving có tính cấu trúc**, thay vì đơn giản chuyển sang model rẻ hơn.

Nó cũng khá gần với blackboard multi-agent architecture trong nghiên cứu: agents trao đổi qua một shared state bên ngoài transcript thay vì mọi communication phải đi parent→child→parent. Một nghiên cứu blackboard cho data-discovery agents báo cáo cải thiện end-to-end success 13–57% tương đối so với các baseline được thử nghiệm, dù domain khác coding nên không nên chuyển trực tiếp con số đó sang Codex. ar...

---

4. WebCodex còn làm một thứ rất thông minh: cache state theo “slice”

Khi tiếp tục một task, nó không đơn giản tin context cũ.

Nó kiểm tra lại:

```
repository
branch
HEAD
worktree
repository rules
target directory
project manifests
```

Nếu không thay đổi thì reuse. Nếu thay đổi thì refresh đúng phần thay đổi. Nếu scan không đủ để chứng minh complete, nó trả `partial/unknown`, không giả vờ context vẫn valid.

Đây là pattern mạnh hơn `state capsule` hiện tại của skill.

Tôi gọi nó là:

> **Fingerprint-Gated Context Reuse**

Tức là mỗi phần state có fingerprint:

```
RULES_FP
HEAD
WORKTREE_FP
MANIFEST_FP
INTERFACE_FP
```

Chỉ reload phần fingerprint đổi.

MTRouter ACL 2026 cũng cho thấy multi-turn routing hiệu quả hơn khi giảm unnecessary model switching và duy trì specialization/context continuity; trong benchmark của họ, cost giảm 58.7% trên ScienceWorld và 43.4% trên HLE so với GPT-5 baseline họ dùng. ACL Anthol…

Hai ý tưởng ghép lại rất tốt:

reuse lane khi semantic state còn hợp lệ
nhưng
invalidate đúng slice khi repository evidence thay đổi.

---

5. Đây là chỗ WebCodex xử lý concurrent write tốt hơn tôi dự đoán

`edits_apply` không đơn giản ghi file.

File hiện hữu phải có **SHA-256 từ lần đọc trước**, batch edit được preflight toàn bộ trước mutation và có `operation_id` để retry idempotently.

Nó gần giống optimistic concurrency control trong database:

```
READ file at hash H1

      ↓ reasoning

WRITE only if current hash == H1

otherwise:
CONFLICT
```

Đây là chính xác kiểu primitive ChatWeb/Codex song hành cần có.

Còn canonical Connector đi xa hơn nữa: **writable task không chạy trực tiếp trong checkout của người dùng**. Code tạo một detached Git worktree tại một managed write slot, giữ baseline commit/tree, rồi `task_finish` tạo content-addressed patch.

Mô hình thực tế là:

```
User/Codex checkout
        |
        | baseline
        v
isolated WebCodex worktree
        |
        | ChatWeb modifications
        v
content-addressed patch
        |
        | verify target still compatible
        v
human/local acceptance
```

Đây là một kiến trúc rất tốt.

Nhưng tôi phát hiện một limitation quan trọng: code hiện dùng một managed writable slot tên:

```
write-slot-01
```

với workspace lease.

Nghĩa là canonical Connector **không được thiết kế như một swarm writer có N lane writable song song vô hạn**.

Đây là quyết định đúng.

---

6. Vì vậy cách bạn đang dùng ChatWeb + Codex nên đổi một chút

Tôi không khuyến nghị:

```
Codex             ChatWeb
  ↓                  ↓
write A            write A
write B            write B
  ↓                  ↓
      merge
```

Ngay cả khi WebCodex cô lập worktree, vấn đề vẫn còn:

```
ChatWeb baseline = A

Codex modifies relevant files
```

```
current state = B

ChatWeb result derived from A
```

Patch không nhất thiết còn đúng trên B.

Mặc định tốt hơn

```
                ┌─ Codex
                │  primary writer
USER GOAL ─ SOL ─┤
                │
                └─ ChatWeb/MCP
                   shadow critic
                   researcher
                   verifier
                   counterexample finder
```

ChatWeb mặc định chỉ có:

```
READ
SEARCH
DIAGNOSE
TEST
REVIEW
```

Không sở hữu write surface.

Chỉ chuyển ChatWeb thành writer khi:

```
WRITE_OWNER = CHATWEB
```

và một trong hai điều đúng:

```
Codex không viết cùng surface
```

hoặc:

```
ChatWeb có isolated worktree + disjoint ownership
```

Đây là thay đổi lớn nhất tôi muốn đưa vào skill.

---

7. HEAD chưa đủ để tránh stale review

Đây là một chi tiết quan trọng.

Giả sử:

```
HEAD = abc123
```

Codex đang có 12 file modified nhưng chưa commit.

ChatWeb cũng thấy:

```
HEAD = abc123
```

thì hai bên vẫn có thể đang reasoning trên hai state khác nhau.

Vì vậy External MCP packet không nên chỉ chứa `BASE_HEAD`.

Tôi đề xuất:

```
EXTERNAL_REVIEW_RUN

PROJECT:
BASE_HEAD:
RELEVANT_PATHS:

PATH_HASHES:
DIFF_FINGERPRINT:
DIRTY_PATHS:

ROLE:
  shadow_read
  diagnostic_inspect
  isolated_write

WRITE_OWNER:

GOAL:
ACCEPT:
PROOF:
```

ChatWeb trả:

```
VERDICT:

BASE_OBSERVED:
PATH_HASHES_OBSERVED:

NOVEL_FINDINGS:
EVIDENCE:

STALE_IF:
REPRODUCTION:
```

```
RECOMMENDED ACTION:
```

Khi response quay về, Sol/Codex không lập tức tin nó.

Đầu tiên:

```
recheck relevant hashes
```

Nếu không đổi:

```
finding = current
```

Nếu đổi:

```
finding = stale candidate
→ revalidate only that finding
```

Không chạy lại toàn bộ review.

Đây là **optimistic concurrency cho reasoning**, không chỉ cho file edit.

---

8. Code xác nhận External MCP phù hợp đặc biệt cho blind review

RETRACE mới công bố ngày 9/8/2026 đi đúng hướng này: patch verification có ích hơn khi verifier không chỉ lặp lại cùng interpretation đã tạo patch. Họ dùng backward reconstruction độc lập để suy ra "patch này dường như đang sửa vấn đề gì?" rồi đối chiếu với issue gốc; trên SWE-bench Verified họ báo cáo Pass@1 tăng 7.0 và 3.6 điểm phần trăm với hai backbone được thử. ar...

Vì vậy ChatWeb **không nên nhận chain/narrative của Codex** kiểu:

> Codex đã xác định X là nguyên nhân và sửa Y vì Z.

Thay vào đó:

```
REQUIREMENT
BASE
DIFF
TEST EVIDENCE
```

Rồi ChatWeb tự trả lời:

```
1. Patch này thực chất thay đổi invariant gì?
2. Nó giải quyết requirement nào?
```

```
3. Requirement nào vẫn chưa được giải?
4. Counterexample nhỏ nhất là gì?
5. Test hiện tại có thực sự phân biệt đúng/sai không?
```

Đây mới tạo independence.

Nếu đưa toàn bộ reasoning Codex sang ChatWeb, ta vừa đốt token vừa làm giảm tính độc lập của critic.

---

9. Có những vấn đề thực tế trong WebCodex cần học, không nên bỏ qua

Vấn đề	Tôi đọc được từ repo	Bài học cho skill
Session state không miễn phí	Issue #15 đo server từ ~11.6 MiB RSS cold lên ~74.2 MiB sau khoảng 14 giờ với ledger populated; repo sau đó tối ưu hot/cold records và giảm clone/persistence pressure.	Không giữ lane/session vô thời hạn. Close/archive/rollover khi task boundary đổi.
Output bounding rất khó làm đúng	Commit mới nhất sửa panic khi byte truncation cắt giữa UTF-8 character và thêm regression tests.	Dùng cursor/page/output reference; không truncate string tùy tiện.
MCP interoperability thay đổi theo client	Issue #7: OAuth/PKCE hoạt động với client test nhưng từng fail trong ChatGPT custom MCP. Issue đã đóng và maintainer cho biết latest main đã sửa.	External plane luôn cần capability preflight; không giả định MCP availability từ config.
Auth boundary rất quan trọng	Repo tách Project Credential, PAT, Runner token và shared-key quick-start; tài liệu nói shared key không phải production IAM.	External critic dùng quyền nhỏ nhất; `read/inspect` trước `write/job`.
Có quá nhiều loại “session”	Repo phải có riêng một tài liệu cảnh báo Workflow Session và Action Audit Session không được cross-wire.	Skill không được copy toàn bộ state-machine của WebCodex vào prompt. Chỉ dùng logical task state.
One write lane is deliberate	Managed connector code dùng một leased writable slot và isolated worktree.	External MCP là verifier/research plane mặc định, không phải swarm writer.

Điểm này rất quan trọng: **WebCodex tốt vì control plane phức tạp nằm trong code**, không phải vì nó bắt model suy nghĩ về toàn bộ sự phức tạp đó.

Skill của chúng ta không nên nhét `Workflow Session`, `Action Audit Session`, `MCP session`, `Connector Task`, `run_id`, `execution_id`, v.v. vào mỗi prompt.

Model chỉ cần abstract state:

```
GOAL
BASE
OWNER
CURRENT_EVIDENCE
EXTERNAL_REVIEW
PROOF
NEXT
```

Infrastructure tự lo phần còn lại.

---

10. Tôi đề xuất nâng kiến trúc skill thành 5 chiều độc lập

Hiện chúng ta đã có route model và lifetime.

Sau khi đọc WebCodex, tôi sẽ formalize thành:

```
EXECUTOR_ROUTE
  TOOL | LUNA | TERRA | SOL

LIFETIME
  LOCAL | EPHEMERAL | PERSISTENT_LANE

REVIEW_PLANE
  NONE | INTERNAL | EXTERNAL_MCP

EXTERNAL_MODE
  SHADOW_READ
  DIAGNOSTIC_INSPECT
  ISOLATED_WRITE

CONSISTENCY
  LIVE
  PINNED
  REVALIDATE_ON_DELTA
```

Đây tốt hơn một routing tree đơn chiều kiểu:

```
Sol → Terra → Luna → MCP
```

vì MCP không phải một model tier.

Một ví dụ:

```
EXECUTOR       = Terra
LIFETIME       = persistent_lane
REVIEW_PLANE   = external_mcp
EXTERNAL_MODE  = shadow_read
CONSISTENCY    = revalidate_on_delta
```

Hoặc một migration cơ học:

```
EXECUTOR       = Luna
LIFETIME       = ephemeral
REVIEW_PLANE   = none
EXTERNAL_MODE  = n/a
CONSISTENCY    = pinned
```

Một financial invariant:

```
EXECUTOR       = Sol
LIFETIME       = local
REVIEW_PLANE   = external_mcp
EXTERNAL_MODE  = diagnostic_inspect
CONSISTENCY    = pinned
```

Đây là kiến trúc tôi đánh giá chính xác hơn nhiều.

---

11. Cách tận dụng đúng `Coding Tools MCP` hiện tại

Tool surface tôi thấy trong phiên của bạn đã khá tốt.

Đối với ChatWeb critic, flow tôi muốn là:

```
workspace_context(detail=compact)
        ↓
ChatWeb independent reasoning
        ↓
agent_workflow(
    workflow=diagnose,
    phase=prepare,
    focused paths/queries
)
        ↓
external finding
```

`agent_workflow` chỉ chuyển sang execute/run khi ChatWeb thực sự được giao implementation lane.

Một command dài thì không chạy lại từ đầu; dùng:

```
task_control
command_control
```

để tiếp tục/poll existing work.

`exec_command` chỉ là fallback.

Đây gần như cùng triết lý với WebCodex: long validation thành durable Job, sau đó poll state thay vì re-execute. WebCodex còn áp bounded paging, cursors và whole-batch deadline cho read/search để không đưa unbounded output lên model.

---

12. Một điều tôi sẽ không đưa vào skill

Tôi sẽ không hard-code:

> "Mọi task lớn đều phải gọi ChatWeb/MCP."

Nghiên cứu tool routing hiện tại cho thấy vấn đề tool bloat và unnecessary routing là thật. ToolScope cho thấy filtered working sets có lợi; nghiên cứu schema compression cũng cho thấy context cost tăng nhanh với số tool. ACL Antholo... +1

External MCP chỉ nên bật khi:

$$EV_{\text{external}} > 0$$

với heuristic:

$$EV_{\text{external}} = P(\text{catch material defect}) \times Impact + Value(\text{Codex tokens avoided}) + Value(\text{p}$$

Không cần giả số chính xác.

Chỉ đánh giá `low / medium / high`.

---

13. Gate mà tôi đề xuất

Tôi sẽ cho External MCP điểm cao khi:

```
oracle partial/weak
integration width medium/high
defect impact high
root cause ambiguous
independent research useful
Codex context/token pressure high
review can run outside critical path
```

Và tắt nó khi:

```
mechanical leaf
strong deterministic oracle
one tiny localized patch
sequential tool loop
same mutable state needs constant decisions
MCP cannot pin/verify current base
external latency nằm toàn bộ trên critical path
```

Một external review pass mặc định là đủ.

Pass thứ hai chỉ chạy nếu pass đầu tìm ra vấn đề khiến patch thực sự thay đổi.

---

14. Chỉ số mới quan trọng hơn “bao nhiêu token”

Tôi sẽ không tối ưu riêng:

```
CODEX_TOKENS
```

mà thêm:

```
EXTERNAL_MCP_DISPATCHED
EXTERNAL_MODE

MCP_CALLS
EXTERNAL_WALL_TIME
CRITICAL_PATH_DELAY

BASE_CHANGED_DURING_REVIEW
STALE_FINDINGS

NOVEL_FINDINGS
ACCEPTED_FINDINGS
DUPLICATE_FINDINGS

PATCH_CHANGED_BY_EXTERNAL_REVIEW
DEFECT_CAUGHT_PRE_ACCEPTANCE

CODEX_TOKENS_AVOIDED
```

Hai KPI cốt lõi:

$$ReviewYield = \frac{AcceptedNovelFindings}{ExternalReviewCost}$$

và

$$StaleRate = \frac{FindingsInvalidatedByConcurrentChange}{AllExternalFindings}$$

Nếu `ReviewYield` cao nhưng `StaleRate` cũng cao, vấn đề không phải model ChatWeb.

Vấn đề là **snapshot protocol**.

---

15. Kết luận sau khi đọc code WebCodex

Tôi chấm kiến trúc theo use case của bạn như sau:

Pattern	Đánh giá
ChatWeb/MCP làm **shadow researcher**	**9.5/10**
ChatWeb/MCP làm **blind external reviewer**	**9.5/10**
ChatWeb/MCP chạy test/diagnostic song hành	**9/10**
ChatWeb persistent critic cùng một lane	**8.5/10**, nếu fingerprint còn hợp lệ
ChatWeb isolated writer trên disjoint surface	**8/10**
ChatWeb + Codex cùng sửa một mutable surface	**4/10**
External review bắt buộc trên mọi task	**5–6/10**
Gated external review	**9+/10**

Đặc biệt, RETRACE củng cố giá trị của independent verification, nhưng một nghiên cứu khác về "verification horizon" nhấn mạnh rằng verifier vẫn chỉ là proxy cho user intent và phải đồng tiến hóa với generator; tức ChatWeb không được biến thành final oracle tuyệt đối. arXiv +1

Sol vẫn giữ acceptance.

Codex chịu trách nhiệm execution.

ChatWeb/MCP cung cấp một nguồn evidence độc lập.

Repository + deterministic checks mới là shared truth.

Cách tôi sẽ đưa vào `sol-lean-codex`

Tôi **không** muốn phình `SKILL.md`. Tôi sẽ chỉ thêm một gate ngắn vào control plane, rồi tạo một reference riêng kiểu:

```
references/external-mcp-plane.md
```

Trong đó formalize:

```
external critic routing
blind review
snapshot/fingerprint contract
read/inspect/write authority
stale-result handling
Coding Tools MCP fast path
async/background usage
review-yield telemetry
stop rules
```

Sau khi đọc code WebCodex, tôi đánh giá đây hiện là **nâng cấp có giá trị cao nhất tiếp theo** cho skill. Nó giải quyết đồng thời ba thứ mà Luna/Terra routing một mình không giải quyết được: **Codex token pressure, independent verification bias, và long-context duplication**—trong khi vẫn giữ được một final authority duy nhất.

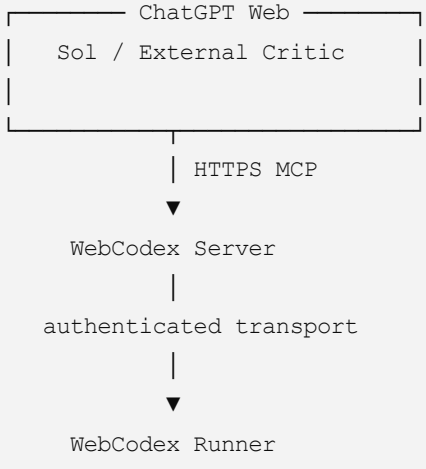
Vậy làm thế nào để kết nối giữa codex desktop, MCP tool web

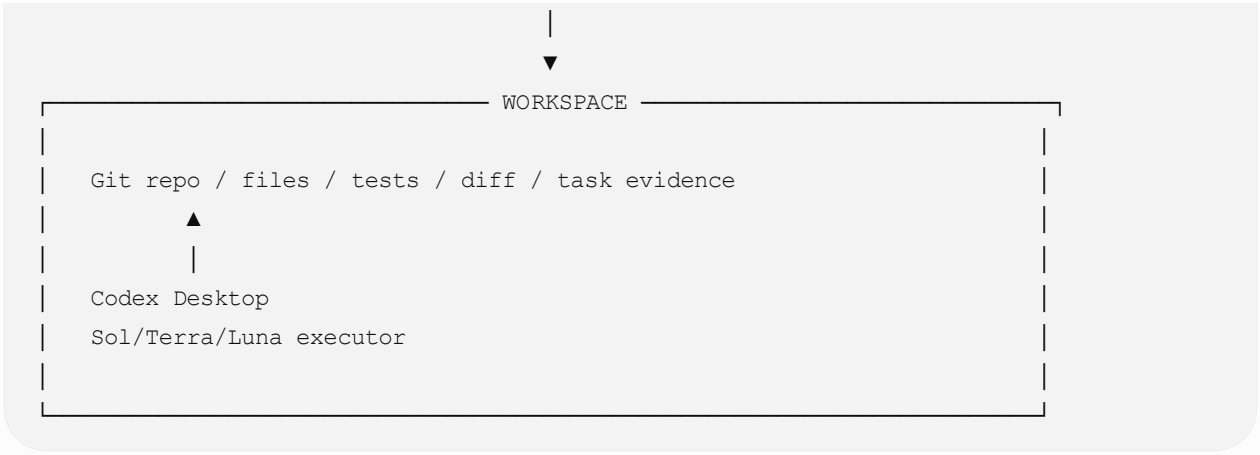
Đã xử lý trong 1m 4s

Công cụ được gọi

Có 3 mức kết nối khác nhau. Với mục tiêu bạn đang hướng tới — **Codex Desktop là executor chính, ChatWeb phản biện/nghiên cứu qua WebCodex MCP** — tôi khuyên dùng mức 2, chưa cần làm mức 3 ngay.

Kiến trúc tôi khuyên dùng





Codex Desktop không cần gọi WebCodex MCP để sửa repo local. Nó tiếp tục làm việc trực tiếp trên workspace như hiện nay. ChatGPT Web mới là bên dùng WebCodex để đọc, search, test và review repo. Đây là cách ít tầng nhất và giữ được tính độc lập của critic. WebCodex vốn được thiết kế với Server tách khỏi Runner; repository và toolchain nằm ở Runner, còn model chỉ gọi các capability được expose.

Điểm quan trọng là **hai bên kết nối qua workspace/evidence, không qua transcript**.

---

1. Phần ChatGPT Web → WebCodex

Trên PC chạy WebCodex Runner cho repository:

```
PC
└── repository
    └── WebCodex Runner
```

Runner kết nối tới WebCodex Server. Server cung cấp endpoint MCP dạng:

```
https://<webcodex-server>/mcp
```

WebCodex hỗ trợ hosted server, self-hosted server và temporary `share`; repo cũng khuyến nghị HTTPS cho hosted client.

Phía ChatGPT Web thì cấu hình WebCodex thành remote MCP/plugin. Đây là cấu hình **riêng với web**: OpenAI hiện nói rõ ChatGPT web không đọc `~/.codex/config.toml` trên PC; remote MCP tools của web được quản lý qua plugin/app surface. OpenAI Develo...

Sau đó ChatWeb có thể làm:

```
workspace inspect
files read/search
git diff
run diagnostics
```

```
run tests
```

mà không cần gửi source code thủ công vào chat.

---

2. Codex Desktop cứ mở chính repository đó

Đây mới là phần dễ bỏ sót:

không cần "network-connect" ChatWeb với Codex Desktop để hai bên phối hợp.

Shared state chính là:

```
Git repository
+ worktree state
+ hashes
+ tests
+ deterministic task state
```

Ví dụ Codex đang sửa:

```
src/pfm/service.rs
```

ChatWeb qua MCP có thể đọc workspace, thấy Git diff và kiểm tra tests.

Sau đó ChatWeb trả finding:

```
FINDING:
attributed_cost() có thể double-count ở branch X

EVIDENCE:
src/pfm/service.rs ...
test ...
```

Codex tiếp tục xử lý finding đó.

Như vậy:

```
Codex reasoning history      ≠ ChatWeb reasoning history

                nhưng

        repository evidence = shared truth
```

Đây chính là thứ tạo independent review.

---

3. Codex Desktop cũng có thể kết nối WebCodex MCP

Nếu bạn muốn, Codex có hỗ trợ **Streamable HTTP MCP server** qua `config.toml`. OpenAI hiện hỗ trợ `url`, bearer token từ environment, OAuth, tool allow/deny list và approval policy cho remote MCP. OpenAI Develo... +1

Một cấu hình WebCodex có thể có dạng:

TOML

```
[mcp_servers.webcodex]
url = "https://YOUR_WEBCODEX_SERVER/mcp"
bearer_token_env_var = "WEBCODEX_MCP_TOKEN"
enabled = true
required = false
default_tools_approval_mode = "writes"
tool_timeout_sec = 120
```

Không nên bỏ token trực tiếp vào `config.toml`; để nó ở environment rồi dùng `bearer_token_env_var`.

Codex MCP config mặc định nằm ở:

```
~/.codex/config.toml
```

Trên Windows tương đương:

```
%USERPROFILE%\.codex\config.toml
```

OpenAI cũng hỗ trợ `.codex/config.toml` ở mức project cho trusted project. OpenAI Develo...

Nhưng tôi không khuyên bật WebCodex MCP cho Codex Desktop trong use case hiện tại, vì Codex đã có local file/shell/Git tools. Route:

```
Codex
 → WebCodex Server
 → Runner
 → same local repo
```

chỉ thêm latency và duplicate tool surface.

Nó chỉ thực sự có ích nếu Codex Desktop cần:

- thao tác repository trên **máy khác**;
- dùng durable task/evidence của WebCodex;
- truy cập environment/toolchain chỉ Runner khác có;
- hoặc bạn cố tình muốn một execution boundary riêng.

4. “Cùng workspace” vẫn chưa phải “ChatWeb nói chuyện với Codex session”

Đây là distinction rất quan trọng.

Sơ đồ trên cho phép:

```
ChatWeb ↔ repository ↔ Codex
```

nhưng **không cho phép**:

```
ChatWeb → gửi message trực tiếp
          vào thread đang mở
          trong Codex Desktop
```

Hai thứ khác nhau.

Nếu mục tiêu của bạn là:

> “ChatWeb phát hiện vấn đề rồi giao trực tiếp cho Codex xử lý, xem Codex progress, nhận diff, tiếp tục turn…”

thì MCP filesystem không đủ.

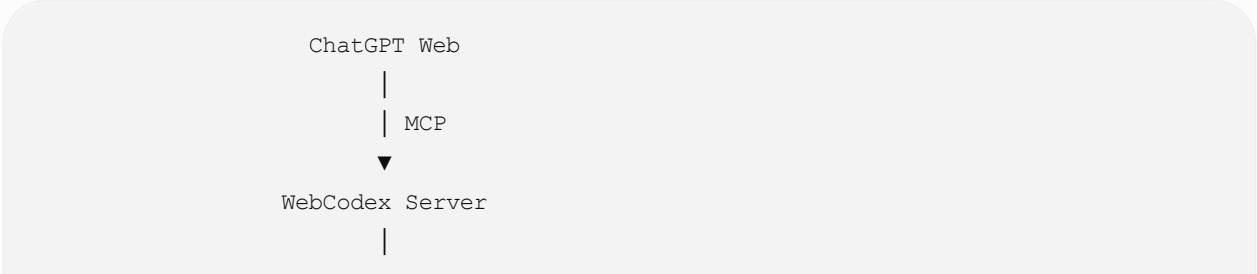
OpenAI đã xây **Codex App Server** chính xác cho loại integration này. App Server là bidirectional JSON-RPC và quản lý `Thread → Turn → Item`, thread persistence, streaming progress, approvals, diff và tool events. Codex Desktop cũng dùng Codex harness/App Server ở tầng bên dưới. OpenAI

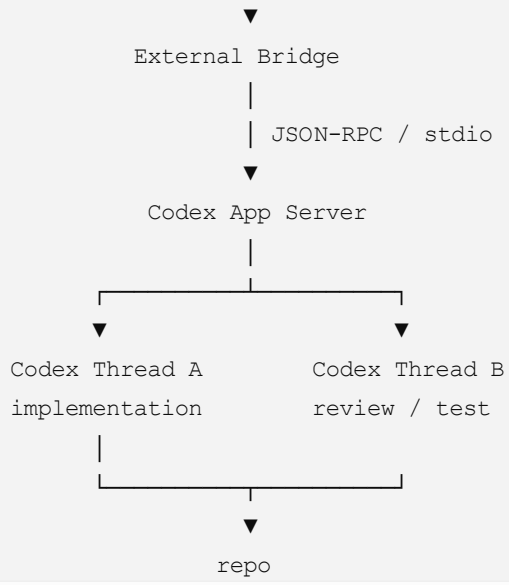
OpenAI hiện gọi App Server là phương thức first-class để tích hợp đầy đủ Codex; `codex mcp-server` thích hợp hơn khi chỉ muốn gọi Codex như một tool, vì MCP không biểu diễn đầy đủ richer Codex session semantics. OpenAI

---

5. Kiến trúc nâng cao tôi muốn xây sau này

Nếu muốn **ChatWeb thật sự điều phối Codex**, tôi sẽ thêm một bridge:





Bridge sẽ expose cho ChatWeb **chỉ vài capability cấp cao**, ví dụ:

```
codex_task_start
codex_task_send
codex_task_status
codex_task_cancel
codex_task_diff
codex_task_review
```

Phía dưới chúng sẽ map sang App Server:

```
thread/start
thread/resume
turn/start
turn/interrupt
item events
turn/completed
```

App Server đã có thread/turn lifecycle và bidirectional event stream phục vụ loại client integration này. OpenAI

Tôi vừa kiểm tra `webcodex` hiện tại và **không thấy implementation** `codex app-server` **trong repo**, nên bridge này sẽ là phần mở rộng mới chứ không phải bật một config sẵn có.

---

6. Có một phương án trung gian rất hấp dẫn

Không cần App Server bridge ngay.

WebCodex có thể gọi:

```
codex exec "..."
```

trên PC.

Khi đó:

```
ChatWeb
   |
   ▼
WebCodex commands_run
   |
   ▼
codex exec
   |
   ▼
new Codex worker
```

Ưu điểm:

```
rất dễ làm
không cần sửa Codex Desktop
ChatWeb có thể delegate coding
có thể dùng Sol/Terra/Luna
```

Nhược điểm lớn:

> Đây là **Codex process/session mới**, không phải thread bạn đang mở trong Codex Desktop.

Vì vậy nó phù hợp với:

```
isolated reviewer
research worker
one-shot implementation
tests
independent diagnosis
```

nhưng không phù hợp nếu bạn muốn tiếp tục đúng Desktop thread.

OpenAI cũng mô tả `Codex Exec` là lightweight non-interactive mode phù hợp cho one-off task/automation, trong khi App Server dành cho full harness/session integration. OpenAI

---

7. Kiến trúc tôi chọn cho `sol-lean-codex`

Tôi sẽ chia thành **3 cấp**, không nhảy ngay lên bridge phức tạp:

```
LEVEL 1 — SHARED WORKSPACE              ← dùng mặc định
Codex Desktop ↔ repo ↔ WebCodex ↔ ChatWeb

LEVEL 2 — EXTERNAL CODEX WORKER
ChatWeb → WebCodex → codex exec
                   → independent worker

LEVEL 3 — CODEX SESSION BRIDGE
ChatWeb → MCP → bridge → Codex App Server
                       → persistent thread
```

Với workflow hiện tại của bạn, **Level 1 là tối ưu nhất**.

Level 2 chỉ bật khi ChatWeb cần giao một work packet độc lập cho Codex mà không cần đụng Desktop session.

Level 3 chỉ đáng xây khi bạn thực sự muốn **ChatWeb trở thành supervisory control plane có khả năng start/resume/steer Codex threads**.

Và khi lên Level 3, tôi sẽ **không để ChatWeb điều khiển UI Codex Desktop**. Nó sẽ điều khiển **Codex harness qua App Server**. Desktop chỉ trở thành một client khác nhìn vào cùng thread/runtime. Đó là abstraction đúng và bền hơn nhiều so với automation bằng click/window.

Tóm lại: với hệ thống hiện tại, hãy để **Codex Desktop viết trực tiếp local repo**, **ChatGPT Web dùng WebCodex MCP làm external critic**, và dùng Git/hash/test/task state làm cầu nối. Khi cần ChatWeb giao việc thật sự cho Codex, thêm `codex exec` trước; chỉ khi cần persistent session/streaming/steering mới xây App Server bridge.

"E:\DEV\Tools-Codex-web" tôi đang dùng tool này rồi kiểm tra tạo kết nối giữa nó và codex desktop

Worked for 36m 45s

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Đã kiểm tra trực tiếp và **tạo bridge giữa** `E:\DEV\Tools-Codex-web` **với Codex Desktop** trong source hiện tại. Tôi cũng đã chạy một Codex turn thật qua bridge; không chỉ dừng ở mức thiết kế.

Kiến trúc hiện tại là:

```
ChatGPT Web
    |
    ▼
Coding Tools MCP
    |
    | codex_bridge
    ▼
Codex App Server sidecar
    |
    ├── dùng chung C:\Users\Admin\.codex
```

```
    |   ├── auth
    |   ├── skills
    |   ├── config
    |   └── persisted thread history
    |
    ▼
Codex model
```

Nó dùng đúng App Server/thread-turn protocol của Codex thay vì giả lập thao tác UI Desktop. Đây cũng là abstraction chính thức phù hợp cho client tích hợp với Codex. GitHub +1

Các phần tôi đã hoàn thành:

- Thêm **một tool duy nhất** `codex_bridge` vào smart surface, từ 8 → 9 tools; không làm nổ tool schema.
- Các action: `status`, `threads`, `run`, `observe`, `interrupt`, `close`.
- Tự phát hiện binary Codex Desktop tại `C:\Users\Admin\AppData\Local\OpenAI\Codex\bin\...\codex.exe`; hiện máy chạy `codex-cli 0.147.0-alpha.6.6`.
- `threads` chỉ thấy thread thuộc **workspace MCP hiện hành**, không rò lịch sử repo khác.
- Có thể lấy một persisted Desktop thread làm `source_thread_id`, nhưng bridge **fork thread đó** rồi làm việc độc lập; không giành quyền điều khiển live turn đang mở trên Desktop.
- Mọi bridge turn bị khóa `approvalPolicy=never`, sandbox `read-only`, network disabled.
- Không thể `observe/interrupt` một Desktop thread tùy ý; chỉ thread do bridge tạo/fork mới được điều khiển.
- Environment child loại bỏ toàn bộ `CODING_TOOLS_MCP_*` và `WEBCODEX_*`, nên bearer token của Web MCP không được truyền sang Codex.
- Các MCP server của Codex Desktop hiện có — `autocad-mcp`, `canvas-authoring`, `node_repl` — đều bị disable trong sidecar. External critic vì thế không thể vô tình gọi chúng.
- `observe` được nén: không trả hàng loạt delta/startup events; chủ yếu trả final text, trạng thái, token usage và event quan trọng.

Live proof trên chính PC này:

```
connected = true
codex_home = C:\Users\Admin\.codex
codex_version = 0.147.0-alpha.6.6

disabled Codex MCP:
- autocad-mcp
- canvas-authoring
- node_repl

turn = completed
agent_text = FINAL_BRIDGE_OK
stderr = none
```

Bridge còn lấy được native token telemetry từ App Server. Probe cuối ghi nhận `18,675 total tokens`, `18,667 input`, `8 output`. Con số này chỉ là một probe cụ thể, không nên dùng làm baseline kinh tế; điểm quan trọng là từ giờ skill có thể **đo token thật của Codex lane** thay vì suy đoán.

Acceptance hiện tại rất tốt: Python compile sạch, **29/29 Python tests pass**, và toàn bộ Electron/source suite **38/38 tests pass**, `git diff --check` cũng pass.

Có một failure không liên quan bridge: `npm run dist:dir` không đóng gói được vì file hiện hữu trong `node_modules/@peculiar/utils/.../index.js` chứa dữ liệu NUL và Node ném `SyntaxError`. Tôi không tự chạy `npm ci` để thay dependency của máy vì không cần cho kết nối hiện tại. App đang chạy trực tiếp từ source repo và `resources` chính là source được runtime sử dụng.

Còn một bước để ChatWeb thấy tool mới

MCP Python hiện đang chạy PID `20036` và được start **trước khi tôi thêm** `codex_bridge`, nên process đó vẫn giữ tool registry cũ trong memory. Tôi không force-kill nó trong lượt này vì nó chính là đường MCP đang phục vụ cuộc hội thoại; force restart giữa tool call có thể làm rơi Tunnel.

Trong ứng dụng `Tools-Codex-web`, bạn chỉ cần bấm:

“Khởi động lại toàn bộ dịch vụ”

Đó chính xác là button `overviewRestart`; code sẽ restart MCP và Tunnel theo đúng lifecycle của app. Sau đó ChatGPT Web nên **Scan Tools / refresh connector** để schema mới có `codex_bridge`. Phiên connector hiện đã load trước thay đổi có thể vẫn cache danh sách tool cũ.

Sau reload, flow sẽ là:

```
codex_bridge status
        ↓
codex_bridge threads
        ↓
chọn Desktop persisted thread nếu cần
        ↓
codex_bridge run
  source_thread_id = ...
        ↓
read-only fork
        ↓
codex_bridge observe
        ↓
ChatWeb nhận independent Codex critique
```

Điểm cần giữ rõ: hiện chưa có contract ổn định để một external App Server client chiếm chính xác **live in-memory turn đang active trong Codex Desktop**; upstream cũng đang có yêu cầu/issue về boundary này. Vì vậy fork persisted thread là phương án an toàn hơn. GitHub

Các file source thay đổi hiện chỉ gồm `codex_bridge.py`, `server.py` và hai test files liên quan + contract test smart surface. Tôi **chưa commit** các thay đổi này.

Khởi động lại toàn bộ dịch vụ

Đã xử lý trong 5m 45s

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Đã **khởi động lại toàn bộ dịch vụ** thành công.

- Coding Tools MCP: PID mới `5560`, port `18765` đang hoạt động.
- OpenAI Tunnel: PID `14732`, port `18081` đang hoạt động.
- `manualStop=false`.
- Runtime mới báo `smart-v2`, `tool_count=9` → `codex_bridge` đã được load thật.
- Workspace vẫn giữ nguyên `E:\DEV\HR STA\HR`.
- Các file probe tạm đã được dọn sạch.

Nếu danh sách tool trong phiên ChatGPT hiện tại vẫn hiển thị 8 tools thì đó là cache schema của phiên/connector; runtime phía PC đã chạy bản 9 tools mới.

Hôm qua 14:00

Thử chạy test để xem viết kết nối làm việc với nhau xem nào

Worked for 6m 20s

Công cụ được gọi

n
h
:

Công cụ được gọi

- Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Đã test thực tế. Kết nối **ChatWeb → Coding Tools MCP → Codex App Server → Codex Desktop persisted history** hoạt động.

Kết quả quan trọng:

- Bridge kết nối đúng binary Codex Desktop: `codex-cli 0.147.0-alpha.6.6`, dùng chung `C:\Users\Admin\.codex`.
- Bridge đọc được **10 persisted Codex Desktop threads** trong `E:\DEV\HR STA\HR`.
- Tôi chọn thread thật `Review PFM master document`, rồi tạo một **read-only fork** từ thread đó.
- Codex được hỏi một thông tin chỉ tồn tại trong context thread gốc và trả chính xác: `260811_PFM_MASTER_PHAN_TICH_KE_THUA_VA_TRIEN_KHAI_HOAN_THIEN.md`.
- Turn hoàn tất `status=completed`, `error=null`; sandbox vẫn `read-only`, `approvalPolicy=never`, network disabled.
- Test còn bắt được một bug thật: `thread/fork` gửi đồng thời `ephemeral=true` và `deferGoalContinuation=true`, trong khi App Server hiện tại cấm combination này. Tôi đã sửa và thêm regression test.
- Sau sửa: **30/30 Python tests pass**, **38/38 app/source tests pass**, `git diff --check` pass.

Có một phát hiện rất đáng chú ý về chi phí. Clean/new thread probe chỉ nạp khoảng **18,686 input tokens**, trong khi fork full Desktop history vừa rồi nạp tới **233,206 input tokens**, tức khoảng **12.5×** chỉ để trả lại một filename. Fork đó còn có `cachedInputTokens=0`.

Điều này xác nhận kiến trúc nên là:

```
Mặc định:
ChatWeb
   ↓ compact task/evidence packet
MCP
   ↓
NEW Codex sidecar thread
```

và chỉ dùng:

```
Desktop persisted thread
   ↓
FULL THREAD FORK
```

khi thực sự cần lịch sử reasoning/context cũ. **Không nên fork Desktop session làm default**, vì token tax rất lớn.

Probe clean-thread đầu tiên cũng cho thấy một failure khác: Codex đã thực thi được `commandExecution` trong workspace nhưng cuối turn gặp `serverOverloaded / Selected model is at capacity`. Đây là lỗi capacity phía model, không phải lỗi bridge. Sau đó route fork khác đã hoàn tất bình thường.

Một điểm vận hành còn lại: MCP PID `5560` hiện được khởi động **trước khi tôi sửa bug** `thread/fork`, nên process live vẫn giữ implementation cũ trong memory. Source trên disk đã đúng và toàn bộ tests xanh, nhưng cần **"Khởi động lại toàn bộ dịch vụ" thêm một lần cuối** để live `codex_bridge` nạp bản sửa. Tôi đã thử tự tạo helper restart cuối nhưng helper bị lỗi quoting trước khi được tạo/chạy, nên tôi không tuyên bố runtime hiện đã reload fix.

Sau lần restart cuối đó, tôi đánh giá kết nối đã đủ để bắt đầu thiết kế workflow thực tế: **ChatWeb làm Sol/external critic → MCP gửi compact packet → Codex chạy independent lane → trả result/token telemetry về ChatWeb**, chỉ fork Desktop history khi router xác định giá trị context cũ lớn hơn chi phí ~context replay.

- Vậy lên kế hoạch phân tích và tối ưu hóa việc này hơn. Như này đang khá tốn kém nếu kết hợp. Test thử để đưa ra kế hoạch tối ưu nhất và hiệu quả

Worked for 18m 54s

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Đã test đủ để chốt một hướng tối ưu khá rõ. **Kiến trúc hiện tại nên đổi từ "ChatWeb + Codex cùng suy nghĩ" thành "ChatWeb/MCP mặc định, Codex chỉ là reviewer được gate theo chi phí".** Full Desktop-history fork không nên là luồng bình thường.

Kết quả benchmark thực tế

Các credit dưới đây được quy đổi theo Codex rate card hiện tại: Sol `125 / 12.5 / 750`, Terra `62.5 / 6.25 / 375`, Luna `25 / 2.5 / 150`, GPT-5.4-Mini `18.75 / 1.875 / 113` credit trên 1M token theo thứ tự uncached input / cached input / output. help.openai.c…

Route đã test	Oracle	Input: cached / uncached	Wall time	Credit ước tính
Full Desktop fork / Sol	Pass	0 / **233,206**	~9.7s*	**29.26**

Route đã test	Oracle	Input: cached / uncached	Wall time	Credit ước tính
Terra, repo-read, warm cache	Pass	17,152 / 882	18.4s	**0.170**
Terra, minimal-base, warm	Pass	14,080 / 729	24.8s	**0.141**
Luna, resolved-low, cold	Pass	0 / 24,470	13.8s	**0.615**
5.4-Mini, semantic packet	Pass	1,920 / 11,667	4.20s	**0.234**
5.4-Mini, packet + no-tools	**Pass**	1,920 / **6,387**	**2.92s**	**0.135**

* Full-fork không phải cùng workload với các microbench còn lại, nên wall time không nên so trực tiếp.

Điểm đáng chú ý nhất: packet-only + no-tools dùng **8,307 input tokens**, thấp hơn full-history fork khoảng **96.4%**, còn mức credit quan sát được chênh khoảng **217×**. Đây không phải phép so chất lượng model apples-to-apples, nhưng nó cho thấy chi phí của **topology/context transfer** lớn đến mức nào.

Full fork 233K còn rất gần ngưỡng **272K input**, nơi GPT-5.6 bắt đầu tính 2× input và 1.5× output cho toàn request. OpenAI Develo... +2 Vì vậy với thread Desktop dài hơn một chút, cost có thể nhảy thêm một bậc lớn.

Một phát hiện còn quan trọng hơn model tier

Cache state có thể quan trọng hơn model label.

Trong sample này Terra warm chỉ khoảng `0.141` credit, trong khi Luna cold khoảng `0.615` credit — tức Terra đắt hơn về rate card nhưng request thực tế lại rẻ hơn khoảng **4.35×** vì phần lớn prefix được cache.

OpenAI hiện tính cached input chỉ bằng 10% uncached input cùng model và GPT-5.6 có thời gian cache tối thiểu 30 phút. OpenAI +1

Do đó router không nên có rule đơn giản:

```
Luna < Terra < Sol
```

Mà phải là:

```
EXPECTED_CREDIT(model, cache_state, profile, packet_size)
```

Một **warm Terra** có thể kinh tế hơn một **cold Luna**.

Bug routing mà benchmark tìm ra

Desktop config hiện là:

```
model = gpt-5.6-sol
model_reasoning_effort = ultra
```

Bridge hiện tại chưa override effort đúng contract.

Tôi đã probe schema và runtime. Cách đúng là:

```
thread/start:
  model = ...
  config:
    model_reasoning_effort = low
```

và ở turn:

```
turn/start:
  model = ...
  effort = low
```

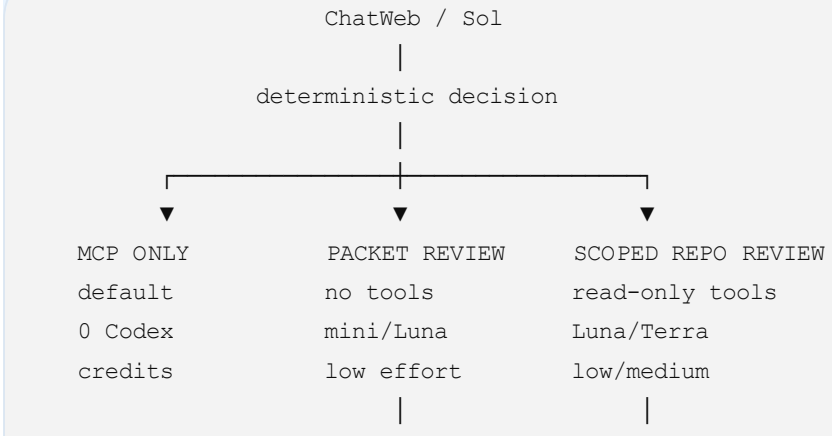
Probe thread-only đã xác nhận:

```
config low    -> resolved effort low
config medium -> resolved effort medium
```

Trong khi cách cũ chỉ truyền `reasoningEffort` không thắng được root config và vẫn resolved `ultra`.

Đây nên là **P0 fix**. External reviewer tuyệt đối không nên vô tình chạy `ultra`, đặc biệt khi overlay của skill vốn đã điều phối routing; skill hiện cũng chủ trương giảm context/packet trước khi tăng tier và tránh nested orchestration.

Kiến trúc tối ưu tôi đề xuất



```
                                  └──────────┬──────────┘
                                             ▼
                                         FINDINGS
                                             │
                                             ▼
                                        ChatWeb/Sol
                                       final acceptance

FULL DESKTOP HISTORY FORK
        ↓
    MANUAL / exceptional only
```

Policy tôi sẽ dùng

Mode	Khi nào	Model ban đầu	Context
`NONE`	test/compiler/MCP đủ chứng minh	Không Codex	0
`PACKET`	câu hỏi semantic đã localize, oracle mạnh	5.4-Mini hoặc Luna	no-tools
`SCOPED`	cần tự đọc vài file/symbol	Luna → Terra	read-only
`CRITICAL`	architecture, finance semantics, auth/data	Terra → Sol	scoped
`HISTORY`	chỉ khi lịch sử reasoning cũ thực sự là evidence	explicit only	full fork cuối cùng

Default phải là `NONE`, không phải Codex.

Đây là thay đổi quan trọng nhất so với ý tưởng ban đầu.

ChatWeb vốn đã có Coding Tools MCP. Nếu tôi có thể đọc diff, test, source và tự review chính xác thì gọi thêm Codex chỉ tạo duplicate compute. Codex chỉ được dispatch khi cần **independent evidence** có giá trị vật chất.

Packet reviewer nên cực nhỏ

Thay vì fork transcript:

```
200K+ context
```

ChatWeb tạo packet khoảng vài nghìn token:

```
OBJECTIVE
ACCEPT
BASE / DIFF_FINGERPRINT

SCOPE
```

```
  exact paths / symbols

REQUIREMENT
  settled semantics only

EVIDENCE
  relevant diff/snippets
  test result
  failure

QUESTION
  one distinct review question

OUTPUT
  structured schema
```

Reviewer trả:

JSON

```
{
  "verdict": "accept|fix|blocked",
  "findings": [],
  "confidence": "...",
  "missing_proof": [],
  "need_repo_evidence": false
}
```

Nếu `need_repo_evidence=false`, reviewer **không cần shell**.

Nếu `true`, router mới nâng:

```
PACKET → SCOPED_REPO
```

Không cấp toàn bộ repo ngay từ đầu.

Test của tôi chứng minh pattern này có giá trị: trên cùng semantic oracle, tắt các feature không cần thiết (`multi_agent`, apps, browser, computer use, image generation, memories, plugins, skill search, workspace discovery, shell...) giảm input:

```
13,587 → 8,307
```

tức **~38.9%**, đồng thời credit giảm **~42.3%**, và oracle vẫn pass.

Full Desktop thread không nên dùng để “truyền context”

Nó chỉ nên dùng cho một case:

> “Điều tôi cần kiểm chứng nằm trong lịch sử reasoning của Codex, không thể reconstruct từ repository/task capsule.”

Ngay cả khi cần history, tôi sẽ ưu tiên:

```
Desktop persisted thread
        ↓
read history locally
        ↓
extract relevant turns
        ↓
STATE CAPSULE
        ↓
NEW reviewer thread
```

thay vì:

```
Desktop thread
        ↓
clone toàn bộ 200K context
```

State capsule đúng tinh thần skill hiện tại: goal, accept, base, decisions, evidence, proof, uncertainty và next action; không mirror transcript.

Session reuse cũng phải chia hai trường hợp

Follow-up cùng một vấn đề thì reuse bridge-owned thread. Test Terra follow-up của tôi có `14,080 cached` và chỉ `772 uncached input`, nên incremental continuation có thể rất rẻ khi prefix còn warm.

Nhưng **independent review** thì tạo thread mới để tránh anchoring vào reasoning trước.

Điểm hay là:

```
new thread ≠ cold cache
```

Ta có thể giữ:

```
same sidecar
same model
same baseInstructions
same disabled-feature profile
```

nhưng tạo **new conversation thread**.

Như vậy vừa giữ tính độc lập của reviewer, vừa cho prompt prefix có cơ hội cache.

Không nên gửi một artificial warm-up request chỉ để cache. Hãy để **review thật đầu tiên** warm prefix rồi tận dụng các review tiếp theo trong cửa sổ cache.

Capacity cũng phải trở thành routing signal

Benchmark thực tế gặp nhiều:

```
serverOverloaded
Selected model is at capacity
```

ở cả Terra và Luna.

Trong khi GPT-5.4-Mini hoàn thành semantic review trong `2.92s`.

Vì vậy không được:

```
Luna fail
→ retry Luna
→ retry Luna
```

Mà phải:

```
PACKET strong oracle:
  Mini ↔ Luna
  tối đa 1 fallback

SCOPED:
  Luna → Terra

HIGH RISK:
  Terra → Sol
  không silent downgrade xuống Mini
```

Nếu fallback cũng không có expected value thì **bỏ external review**, không tạo retry loop.

Budget gate ban đầu

Tôi sẽ bắt đầu conservative như sau, sau đó học lại từ telemetry:

Review class	Auto credit budget	Chính sách
Packet / routine	**≤ 0.75**	auto
Scoped semantic	**≤ 1.5**	auto nếu review value cao

Review class	Auto credit budget	Chính sách
Critical	**≤ 5**	Terra/Sol, explicit risk reason
Full history fork	**0 auto**	manual only

Full-history sample của ta là **29.26 credits**, nên nó sẽ bị chặn ngay.

Các threshold này là seed từ local measurements, không phải universal constant. Evaluation guide của skill cũng yêu cầu tune bằng paired local telemetry thay vì hard-code một topology/model cho mọi repo.

Thứ tự triển khai tôi đề xuất

1. **Sửa bridge P0.** Thêm `model`, `effort`, `profile`, `output_schema`; set `thread.start.config.model_reasoning_effort`; set `turn.start.effort`; thêm profile `packet_no_tools` và `scoped_readonly`. `source_thread_id` không còn mặc định là shortcut để truyền context.
2. **Thêm routing gate.** `NONE → PACKET → SCOPED → CRITICAL → HISTORY`. Mỗi lần nâng cấp phải có lý do cụ thể như packet thiếu evidence, oracle yếu hoặc integration width tăng. Full fork chỉ được dùng khi `need_historical_reasoning=true`.
3. **Thêm compact context + continuation.** `codex_bridge` cần action kiểu `continue` cho bridge-owned thread; same-task follow-up reuse thread, distinct blind review dùng new thread. Packet giữ prefix ổn định để cache-friendly.
4. **Thêm cost/capacity telemetry.** Mỗi run ghi `model`, resolved effort, profile, cold/warm, input, cached, uncached, output, credit, wall time, oracle result, overload, novel/accepted findings. Router dùng EWMA của **chi phí thực tế gần đây**, không dùng model name để đoán.
5. **Chạy A/B benchmark đại diện trước khi đưa vào skill.** So bốn policy: `ChatWeb+MCP only`, `full fork`, `compact scoped`, `packet no-tools`. Tách cold/warm cache và ít nhất vài replicate cho localized bug, semantic review, cross-module, architecture và finance/high-risk. Chỉ promote policy mới nếu quality floor không giảm rồi mới tối ưu cost/p90 latency. Đây cũng là đúng thứ tự evaluation hiện có của skill.

Target architecture

Tôi sẽ đặt mục tiêu ban đầu là:

```
≥70–80% task
    ChatWeb + MCP only

15–25%
    compact external review

<5%
```

```
    Terra/Sol critical reviewer

~0%
    full Desktop-history fork
```

Tỷ lệ này **chưa phải kết quả benchmark**, mà là target để kiểm nghiệm.

Phần có bằng chứng mạnh từ test hiện tại là: **full fork quá đắt; correct effort override hiện đang thiếu; minimal instruction có lợi; no-tools packet profile giảm chi phí rất đáng kể; cache warmth phải trở thành input của router; và capacity cần fallback thay vì retry.**

Một chi tiết vận hành: benchmark trên đây import **source bridge mới trực tiếp từ disk**. MCP process đang phục vụ ChatWeb vẫn chưa reload toàn bộ các source fix/optimization mới; tôi sẽ chỉ restart nó sau khi gom P0–P2 thành một patch hoàn chỉnh, tránh restart nhiều lần giữa quá trình tối ưu.

Phải đánh giá tốc độ giải quyết 1 vấn đề nữa. Thêm cả việc đánh giá model Sol đang chạy trên chatweb là model so sánh như nào với Model tren codex. Từ đó áp dụng nghiên cứu thực tế kiểm tra để đưa ra phương án phối hợp hoàn chỉnh, tối ưu và tiết kiệm

Worked for 13m 59s

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

K

Có. Với dữ liệu vừa test, phương án tối ưu nên thay đổi khá mạnh: **ChatWeb Sol không nên chỉ làm "manager gọi Codex"; nó nên là reasoning/control plane chính, còn Codex là execution/review**

t

l

plane được gọi có điều kiện. Điểm quyết định là phải tối ưu đồng thời **quality + time-to-accepted-result + Codex credits**, không tối ưu riêng token.

Kết quả benchmark mới: một vấn đề thật

Tôi dùng chính vấn đề vừa phát sinh trong `Tools-Codex-web`: sau "restart all", MCP PID đổi `20036 → 5560` nhưng Tunnel PID vẫn `14732`. Acceptance là xác định tunnel có thật sự restart hay không và chứng minh bằng process state + source lifecycle.

ChatWeb Sol giải được trong **~11.47 giây end-to-end**, bằng 2 MCP calls. Tổng thời gian thực thi tool chỉ khoảng **1.64 giây**; phần còn lại là model/round-trip. Kết luận đúng là Windows đã tái sử dụng PID: process hiện tại có `StartTime=12:54:53.167`, gần như trùng tuyệt đối `tunnelStartedAt=12:54:53.169`. Source cũng cho thấy `start()` gọi `stop()`, spawn process mới rồi mới ghi `child.pid` và timestamp.

Cùng vấn đề đó cho các route Codex:

Route thực tế	Kết quả	Wall time	Token	Credit xấp xỉ*
ChatWeb GPT-5.6 Sol + MCP	✅ đúng	**11.47s**	không expose	không expose
Codex Sol tự điều tra repo/process	❌ chưa xong, cancel	>25s	15,963	—
Codex Terra tự điều tra	❌ capacity	14.20s	15,815	—
Codex Luna tự điều tra	❌ chưa xong, cancel	>25s	18,377	—
Codex Sol, evidence packet/no-tools	✅ đúng	11.68s	10,687	**~1.405**
Terra, evidence packet	❌ capacity	2.13s	—	—
Luna, evidence packet/no-tools	✅ đúng	10.48s	10,254	**~0.277**
GPT-5.4-Mini, packet/no-tools	✅ đúng	**6.16s**	9,462	**~0.162**

* Dùng Codex token rate hiện hành. Sol = 125/12.5/750 credits trên 1M uncached/cached/output token; Terra = 62.5/6.25/375; Luna = 25/2.5/150; 5.4-Mini = 18.75/1.875/113. OpenAI Help C…

Điểm quan trọng không phải Mini "mạnh hơn" Sol. Nó không mạnh hơn. **ChatWeb đã làm phần khó nhất — localization và evidence construction — nên reviewer chỉ còn một bài suy luận rất hẹp.**

Khi task đã được co lại như vậy, model nhỏ giải nhanh và rẻ.

ChatWeb Sol so với Sol/Terra/Luna trong Codex

Phiên ChatWeb này đang chạy **GPT-5.6 Sol**. Codex trên máy cũng expose `gpt-5.6-sol`, `gpt-5.6-terra` và `gpt-5.6-luna`.

Ở tầng model, Sol là flagship; Terra là lower-cost tier; Luna là fastest/lowest-cost tier. OpenAI hiện công bố các coding eval như sau: Coding Agent Index `80 / 77.4 / 74.6`, SWE-Bench Pro `64.6 / 63.4 / 62.7`, DeepSWE `72.7 / 69.6 / 67.2`, Terminal-Bench 2.1 `88.8 / 87.4 / 84.7` tương ứng Sol/Terra/Luna. OpenAI +1

Điều đó có nghĩa **khoảng cách Sol → Terra trên software engineering tương đối nhỏ so với chênh lệch giá 2×**. Luna rẻ 5× so với Sol nhưng khoảng cách capability lớn hơn. Với finance thì khác đáng kể: Big Finance Bench hiện là Sol `53%`, Terra `51%`, Luna `36%`. OpenAI

Đây chính xác là pattern phù hợp với repo HR/Finance của bạn:

```
Architecture / finance meaning / authorization / integration
                    → ChatWeb Sol

Implementation reasoning / cross-file coding
                    → Codex Terra

Mechanical/localized/strong test oracle
                    → Codex Luna

Tiny independent packet check
                    → Luna hoặc 5.4-Mini
```

Không nên dùng Luna làm chủ semantic tài chính chỉ vì rẻ. Terra gần Sol đáng kể hơn ở domain đó.

Nhưng ChatWeb Sol ≠ Codex Sol về hệ thống

Model tier có thể giống nhau, nhưng harness khác.

ChatWeb Sol đang có conversation context, user intent, khả năng nghiên cứu web và **Coding Tools MCP cấp cao** như `workspace_context` / `agent_workflow`. Nó rất phù hợp với việc tổng hợp architecture, quyết định cần đọc gì và điều phối.

Codex Sol có coding harness local sâu hơn: shell, edits, tests, threads, native code-agent context, effort selection, persistent history. Vì vậy nó mạnh khi cần **ở trong repo lâu và thực thi liên tục**.

Benchmark vừa rồi minh họa rõ sự khác biệt: ChatWeb Sol biết chính xác cần hai phép kiểm tra nào nên giải trong 11.47s; Codex Sol nếu được giao nguyên bài “hãy tự điều tra” đi vào agent/tool loop và

chưa kết thúc sau 25s.

Tức là trong hệ thống này, **tool topology có thể quan trọng ngang hoặc hơn model tier**.

Vấn đề đặc biệt nguy hiểm: Codex đang mặc định `ultra`

Config Desktop của máy hiện có:

```
model = gpt-5.6-sol
model_reasoning_effort = ultra
```

OpenAI mô tả `ultra` là chế độ mặc định phối hợp **bốn agents song song**, đổi nhiều token hơn lấy chất lượng/tốc độ cho các bài khó. OpenAI

Điều này hoàn toàn không phù hợp với:

```
"Kiểm tra giúp tôi finding này đúng không?"
```

Nếu bridge kế thừa `ultra`, một external critic nhỏ có nguy cơ biến thành một multi-agent coding operation.

Probe của tôi tìm ra knob đúng:

```
thread/start:
    model = gpt-5.6-terra
    config:
        model_reasoning_effort = low

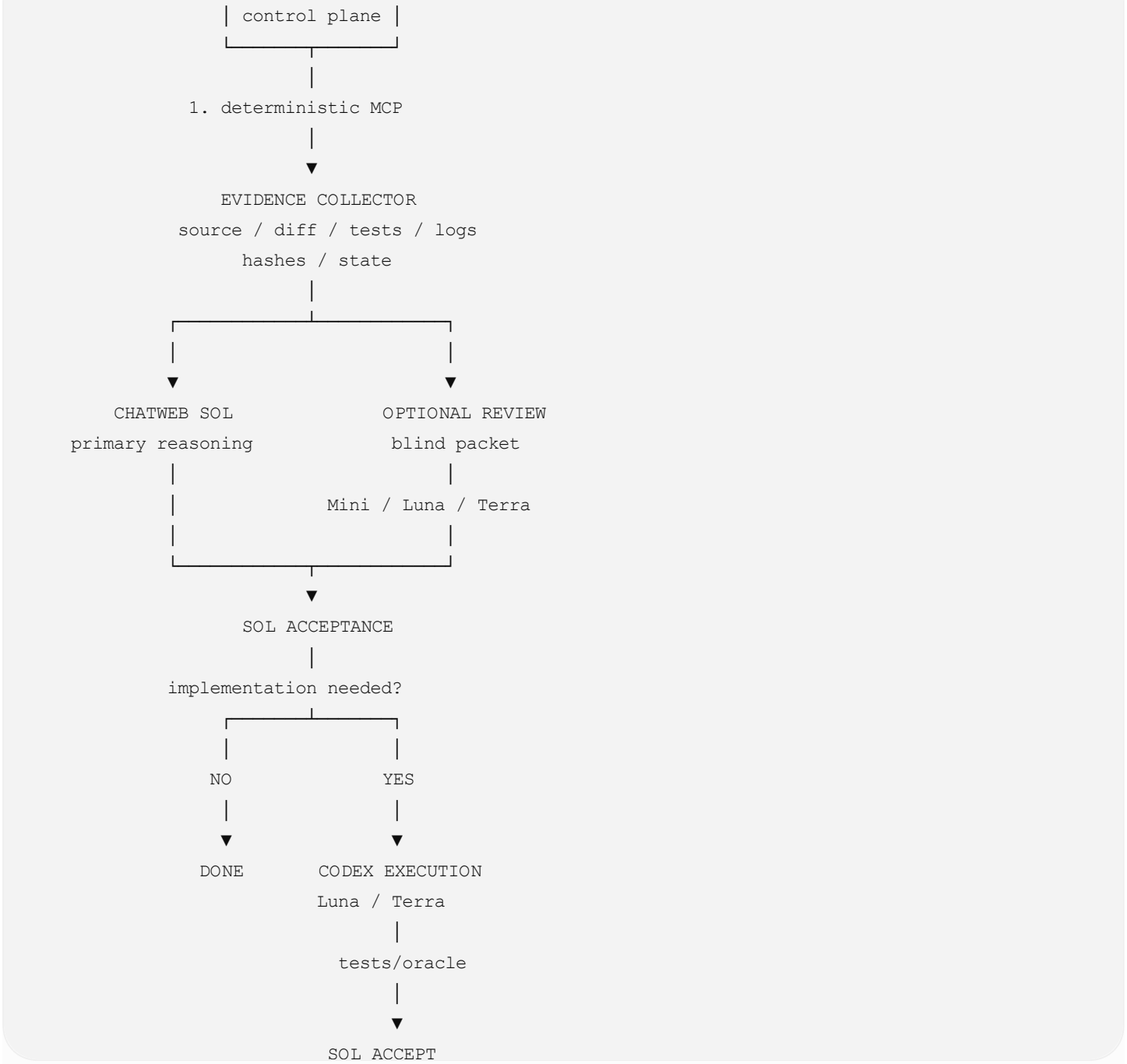
turn/start:
    model = gpt-5.6-terra
    effort = low
```

`thread/start.config.model_reasoning_effort="low"` đã được runtime xác nhận trả lại `reasoningEffort="low"`.

Đây phải là **P0** trước khi đưa bridge vào dùng thường xuyên.

Kiến trúc phối hợp hoàn chỉnh tôi đề xuất

```
                    USER
                     |
                     ▼
            ┌─ CHATWEB SOL ─┐
```



Khác biệt cốt lõi là **evidence được thu một lần**, không để ChatWeb và Codex cùng scan repo độc lập.

Google Research vừa công bố kết quả trên 180 agent configurations: multi-agent giúp mạnh khi task thật sự parallelizable, nhưng có thể làm tệ đáng kể trên task sequential; trong thí nghiệm của họ, một task parallelizable tăng khoảng 81% trong khi một planning task sequential giảm khoảng 70%.

Google Rese...

Do đó:

```
ChatWeb scan repo
→ Codex scan lại repo
→ ChatWeb đọc lại kết quả
```

là kiểu orchestration cần loại bỏ.

Router nên hoạt động như sau

1. **MCP deterministic trước.** ChatWeb xác định 1–3 phép đo có information gain cao nhất rồi batch chúng. Nếu test/compiler/process state đã chứng minh đáp án thì dừng; không gọi Codex. ToolScope cho thấy filtering/merging tool surface có thể cải thiện tool-selection accuracy 8.38–38.6% trên các benchmark của nghiên cứu, củng cố việc chỉ expose đúng tools cần thiết. ACL Anthol…
2. **ChatWeb Sol là coordinator + owner của semantics.** Sol giữ user intent, architecture, financial meaning, public contract và final acceptance. Trong benchmark thực, đây cũng là route nhanh nhất để localize vấn đề mới.
3. **Reviewer chỉ nhận blind packet.** Không gửi narrative kiểu "tôi nghĩ lỗi là PID reuse". Chỉ gửi requirement + evidence + diff/test. Reviewer phải tự suy ra verdict. RETRACE mới công bố cho thấy independent reconstruction/verification cải thiện Pass@1 7.0 và 3.6 điểm phần trăm trên hai backbones SWE-bench Verified mà họ thử; đáng chú ý, cache cao giúp verification bổ sung không nhất thiết đắt hơn baseline. ar…
4. **Chọn reviewer theo risk, không theo prestige.** Strong oracle/localized → Mini hoặc Luna. Medium semantic → Luna. Cross-file cần repo → Terra. Finance/auth/data/architecture → Terra independent reviewer, còn ChatWeb Sol giữ acceptance. Codex Sol chỉ bật nếu Terra và Sol disagree, oracle yếu, hoặc vấn đề thật sự high-impact/high-integration.
5. **Implementation route tách khỏi review route.** Luna cho leaf nhỏ/test mạnh; Terra là default implementation model; Sol chỉ nhận phần shared/critical. Không để ChatWeb và Codex cùng viết một mutable surface.
6. **Cache + capacity phải là input của router.** Giữ stable reviewer prefix/profile, reuse cùng App Server nhưng dùng fresh thread cho blind review. OpenAI hiện có minimum cache life 30 phút và cached reads được giảm 90%. OpenAI Nếu một model vừa `serverOverloaded`, mở circuit breaker và route model khác; không retry y nguyên. Benchmark hôm nay cho thấy Terra overload nhiều lần.
7. **Deadline cứng + max một fallback.** Packet Mini khoảng 8s; Luna 12s; scoped Terra 20s; Sol 25–30s. Nếu hết deadline mà kết quả chưa nằm trên critical path thì cancel. Benchmark còn phát hiện outer timeout có thể để child sidecar tồn tại, nên bridge phải quản lý process-tree/job lifecycle chắc chắn hơn.

MTRouter ACL 2026 cũng đi cùng hướng này: routing theo history + model utility giúp giảm 58.7% cost trên ScienceWorld và 43.4% trên HLE trong thí nghiệm của họ, đồng thời học cách **switch model ít hơn** và specialization theo model. ACL Anthol…

Chính sách tôi sẽ đặt cho hệ thống này

Task	ChatWeb Sol	Codex route	Chạy song song?	Reviewer budget
Test/compiler/process đã quyết định	✅	**NONE**	—	0
Local bug, oracle mạnh	✅ coordinator	Luna executor	khi độc lập	thấp
Routine semantic review	✅ primary	Mini/Luna packet	**Yes**	~0.2–0.4
Cross-file implementation	✅ architecture	**Terra**	research có thể parallel	~1–3
Finance semantics	✅ **owner**	Terra reviewer/executor	**Yes, blind**	theo risk
Auth/data/privacy	✅ **owner**	Terra/Sol reviewer	Yes nếu independent	theo risk
Architecture/high integration	✅ **owner**	Terra first, Sol only escalation	có chọn lọc	cao hơn
Full Desktop history	gate only	**manual exceptional**	No	không auto

Full Desktop-history fork vẫn nên gần bằng **0% auto-dispatch**. Lần test trước nó nạp khoảng 233K input token chỉ để lấy một thông tin trong history. State capsule/selected evidence tốt hơn rất nhiều.

Hiệu ứng lên thời gian giải quyết

Bài vừa test cho một hình ảnh rất rõ.

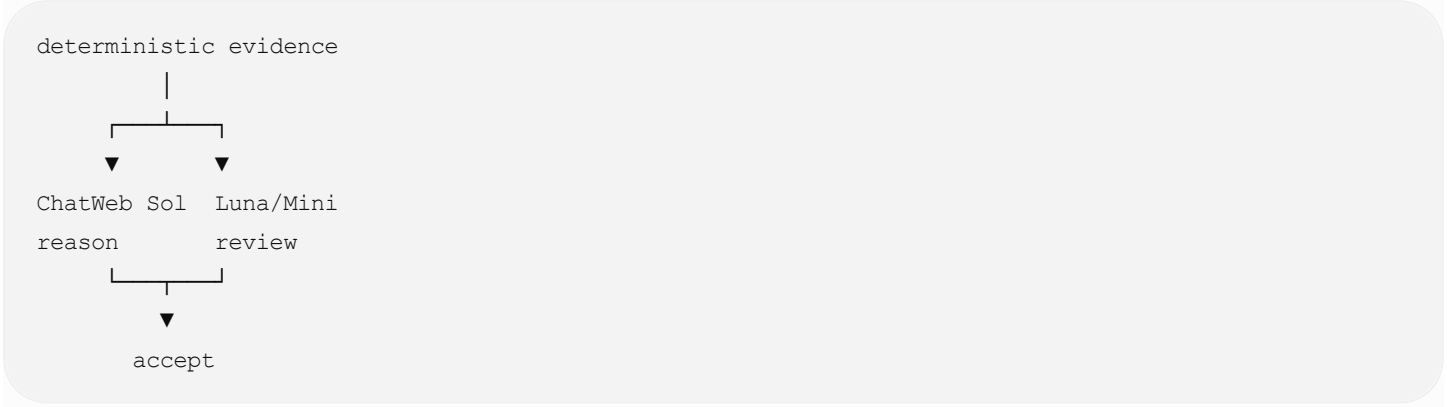
Kiểu hiện tại — serial delegation:

```
ChatWeb reasoning/localize       ~11.5s
        ↓
Codex Luna review               ~10.5s
        ↓
accept

≈ 22s
```

Không đáng.

Kiểu mới — shared evidence + parallel blind review:



Critical path trở thành gần:

```
Evidence collection
+ max(ChatWeb reasoning, reviewer reasoning)
```

chứ không phải tổng hai model.

Trong sample này Mini packet chỉ cần 6.16s và Luna 10.48s, trong khi ChatWeb full path là 11.47s. Vì vậy **nếu reviewer được dispatch sau khi evidence packet hình thành và chạy song song, nó có khả năng nằm hoàn toàn ngoài critical path**. Đây là inference từ số đo hiện tại, chưa phải p90 benchmark.

Đó mới là kiểu phối hợp đáng trả tiền.

Kết luận kiến trúc

Tôi sẽ không xây:

```
ChatWeb Sol → giao cả vấn đề → Codex → trả lại → Sol review
```

làm default.

Tôi sẽ xây:



ChatWeb Sol = semantic brain/control plane.
Codex Terra = workhorse.
Codex Luna = cheap coding leaf.
Mini/Luna = packet verifier.
Codex Sol = escalation, không phải default duplicate Sol.

Đây vừa phù hợp dữ liệu benchmark trên chính máy của bạn, vừa phù hợp model eval/cost hiện tại và nghiên cứu về cost-aware routing, tool filtering, multi-agent scaling và independent verification. Skill hiện tại cũng đã có nguyên tắc “fast path, evidence first, tests as oracle, compact state”; topology này là bước tiếp theo tự nhiên của nó.

Bước triển khai tiếp theo hợp lý là **nâng** `codex_bridge` **thành router-aware bridge**: thêm `profile=packet|scoped|executor`, `model`, `effort`, hard deadline/cancel, capacity circuit breaker, token/credit telemetry và structured evidence packet; sau đó cập nhật `sol-lean-codex` để tự quyết định khi nào `NONE / MINI / LUNA / TERRA / SOL / HISTORY`.

Hoàn thiện tiếp theo bước tư vấn của bạn

Stopped thinking

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Triển khai xử lý tiếp tục

Worked for 18m 1s

bảng integration_cost_vs_tích hợp/Test/Lưu

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

time telemetry và estimated credits. Mọi bridge turn vẫn bị khóa ở `approvalPolicy=never`, filesystem read-only, network off và không điều khiển live turn đang mở trong Codex Desktop.

Live acceptance cho kết quả đáng chú ý:

- `PACKET / gpt-5.4-mini`: hoàn thành khoảng **5.2 s**, estimated **0.131 credits**.
- Continuation cùng thread: khoảng **1.9 s**, cache ratio **95.3%**, estimated **0.026 credits**.
- `SCOPED / gpt-5.6-luna`: khoảng **4.8 s**, estimated **0.261 credits**.
- `CRITICAL / gpt-5.6-terra`: Codex trả `serverOverloaded` sau ~1.45 s; circuit breaker đã mở cooldown 30 s đúng thiết kế, không retry mù. Credit estimates dùng token rate card hiện hành của Codex. OpenAI Help C…

Từ số đo đó, router v3 đã được chốt theo hướng:

```
NONE → PACKET/Mini → SCOPED/Luna → CRITICAL/Terra → SOL_LOCAL → HISTORY
```

Đây **không phải cascade bắt buộc**. Mini là external reviewer mặc định khi packet evidence đã đủ; Luna chỉ dùng khi cần inspect repo; Terra dành cho semantic risk cao; nếu Terra capacity-fail thì trả câu hỏi về Sol coordinator thay vì retry cùng model; `HISTORY` chỉ dùng khi lịch sử Codex thực sự có giá trị. Session lifetime được tách riêng thành `LOCAL | EPHEMERAL | PERSISTENT_LANE`, reuse theo semantic continuity chứ không theo worker identity.

Acceptance source đã xanh: Python compile sạch, bridge unit tests pass, 23 integration tests pass, 5/5 Node stability tests pass, `git diff --check` pass. Working tree hiện chỉ còn đúng 5 file bridge/tool-surface dự kiến và chưa commit:

- `server.py`
- `test_task_state_and_build.py`
- `stability-source.test.js`
- mới `codex_bridge.py`
- mới `test_codex_bridge.py`

`sol-lean-codex` **v3 cũng đã được cài global trên PC** tại `C:\Users\Admin\.codex\skills\sol-lean-codex`. Bản cài có đúng 13 files, 10/10 comparator tests pass, không còn staging rác. `SKILL.md` SHA-256 là `747a21fa35c69808de1f639cb99e0c4ce9db6912a33afcff2d8fae022748a024`. Backup mới được giữ nguyên tại:

```
C:\Users\Admin\.codex\skills\sol-lean-codex.backup-20260812-160126
```

Bundle hoàn chỉnh có SHA-256 `757c3935a5d0c3e45ea1d3567b492492c52707d28ae2fe6b5545190ff3827ff7`:

Download skill.zip

Còn **một bước activation phía client**: source MCP smart surface đã có `codex_bridge`, nhưng wrapper `Coding_Tools_MCP` của **chính phiên chat hiện tại** vẫn discovery ra 8 tool cũ. Nghĩa là code/server đã sẵn sàng nhưng schema tool của phiên đang được cache. Sau khi làm mới kết nối/tool discovery ở client hoặc mở phiên mới, ta cần kiểm tra `codex_bridge` xuất hiện trực tiếp; khi đó đường chuẩn sẽ bắt đầu bằng `codex_bridge(action="status")` rồi router tự chọn packet/scoped/critical theo task.

Xem trước kỹ năng

Nháp

Đang tải chi tiết kỹ năng…

Cài đặt

Nguồn

C

Coding Tools MCP Xử lý nốt đi Còn **một bước activation phía client**: source MCP smart surface đã có `codex_bridge`, nhưng wrapper `Coding_Tools_MCP` của **chính phiên chat hiện tại** vẫn discovery ra 8 tool cũ. Nghĩa là code/server đã sẵn sàng nhưng schema tool của phiên đang được cache. Sau khi làm mới kết nối/tool discovery ở client hoặc mở phiên mới, ta cần kiểm tra `codex_bridge` xuất hiện trực tiếp; khi đó đường chuẩn sẽ bắt đầu bằng `codex_bridge(action="status")` rồi router tự chọn packet/scoped/critical theo task.

Worked for 20m 30s

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Input/output token.

Vì cả input và output

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Đã xử lý nốt phần activation. **Không còn phụ thuộc vào Scan Tools / refresh connector / mở chat mới.**

Kiến trúc live hiện tại là:

```
ChatWeb Sol
    ↓
Coding Tools MCP — 8 tools cố định
    ↓
agent_workflow
    ├─ deterministic execution / tests
    └─ internal Codex review router
          └─ Codex App Server sidecar
```

Tôi đã bỏ `codex_bridge` khỏi model-facing smart surface và giữ nó làm **internal/hidden compatibility service**. Vì vậy schema phía ChatGPT vẫn đúng **8 tools**, nhưng `agent_workflow` có thể gọi Codex bên trong mà client không cần biết tool thứ 9.

Live smoke trên chính phiên này đã chứng minh đường đi hoạt động. Tôi dùng `agent_workflow` sửa tạm một tracked file sạch trong HR; kết quả trả trực tiếp:

```
external_review.plane   = external_mcp
profile                 = packet
model                   = gpt-5.4-mini
effort                  = low
```

Reviewer chạm hard deadline `8s` và bị interrupt đúng policy. Nghĩa là activation thành công nhưng review không được phép kéo dài critical path vô hạn; workflow chính vẫn hoàn tất và Sol giữ quyền acceptance. Tôi không retry y nguyên.

Tôi cũng sửa thêm một điểm quan trọng cho chi phí: reviewer giờ chỉ nhận **diff của** `affected_files`, không còn lấy toàn bộ dirty worktree. Điều này đặc biệt cần thiết ở repo HR hiện có nhiều thay đổi chưa commit và sẽ giảm đáng kể context nhiễu/token.

Acceptance cuối:

- MCP live PID mới: `40192`
- Connector discovery: **đúng 8 tools**
- `12/12` bridge tests pass
- `26/26` runtime/integration tests pass
- `5/5` Node stability tests pass
- Python compile pass
- `git diff --check` pass
- File HR dùng làm probe đã về đúng HEAD; marker không còn và path đã clean
- Temp reload/patch scripts đã dọn
- `Tools-Codex-web` chỉ còn đúng **5 file implementation dự kiến**, chưa commit.

Như vậy từ bây giờ **ChatWeb cứ gọi** `agent_workflow` **bình thường**. Router nội bộ sẽ tự chọn `REVIEW_PLANE=NONE` cho localized change có strong oracle để tiết kiệm; chỉ dispatch packet reviewer khi có expected value, với bounded deadline và không retry mù. Direct `codex_bridge` không còn cần xuất hiện trong UI nữa.

Về việc sử dụng điều phối skill và phân tích trước đây như nào. Nói lại hiện tại tình hình đang triển khai sẽ như nào

Worked for 34s

Công cụ được gọi

Hiện tại hệ thống đã đi khá xa so với ý tưởng ban đầu. Điểm quan trọng nhất là: `sol-lean-codex` **bây giờ là policy điều phối**, còn **Coding Tools MCP là runtime thực thi**

policy đó. ChatWeb Sol không "gọi Codex cho mọi việc" nữa.

1. Skill `sol-lean-codex` hiện đóng vai trò gì?

Skill không phải một worker riêng. Nó là **bộ luật quyết định ai nên làm gì, có cần model thứ hai hay không, và khi nào phải dừng**.

Nguyên tắc ưu tiên của skill hiện là:

```
1. Correctness / security / business semantics
2. Time-to-accepted-result
3. Tổng chi phí thực tế
4. Khả năng bảo trì lâu dài
5. Nếu ngang nhau → chọn topology đơn giản hơn
```

Vì vậy flow khởi đầu luôn là:

```
FINISH LINE
    ↓
CHEAPEST ORACLE
    ↓
LOCALIZE
    ↓
ROUTE
```

chứ không phải:

```
Task
 ↓
Sol
 ↓
Terra
 ↓
Luna
 ↓
review
 ↓
review lại...
```

Skill còn quy định rõ Sol giữ architecture, shared semantics, finance meaning, security boundary, integration và **final acceptance**. Luna chỉ nên nhận leaf nhỏ có strong oracle; Terra nhận reasoning-bearing implementation/cross-file work; deterministic tools được ưu tiên nếu model judgment không cần thiết.

---

2. Phân tích trước đây đã làm thay đổi kiến trúc như thế nào?

header và giải sử token tương đương, cascade chỉ rẻ hơ

Ban đầu chúng ta định nối:

```
ChatWeb
   ↓
Coding Tools MCP
   ↓
codex_bridge
   ↓
Codex Desktop/App Server
```

và có khả năng ChatWeb gửi cả vấn đề sang Codex.

Các benchmark thực tế cho thấy cách đó **hoạt động nhưng không kinh tế nếu dùng mặc định**.

Full Desktop-history fork từng nạp khoảng **233K input tokens** chỉ để lấy lại một thông tin từ history. Trong khi packet reviewer nhỏ có thể xuống khoảng 8–10K input.

Ở một bài diagnosis thực tế, ChatWeb Sol + vài MCP checks giải vấn đề trong khoảng **11.5 giây**, còn việc giao nguyên bài cho Codex tự investigate đã có route vượt 25 giây hoặc gặp capacity. Điều đó cho thấy với nhiều task:

```
ChatWeb đã biết nên kiểm tra gì
```

rẻ và nhanh hơn:

```
giao Codex tự scan + tự localize từ đầu
```

Sau đó benchmark packet reviewer cho thấy một topology tốt hơn nhiều:

```
ChatWeb / deterministic tools
        ↓
   collect evidence once
        ↓
  ┌──────────────┐
  │ ChatWeb Sol  │
  └──────┬───────┘
         │
 optional small blind reviewer
```

Tức là **chia evidence**, không chia toàn bộ reasoning.

---

3. Kiến trúc live hiện tại

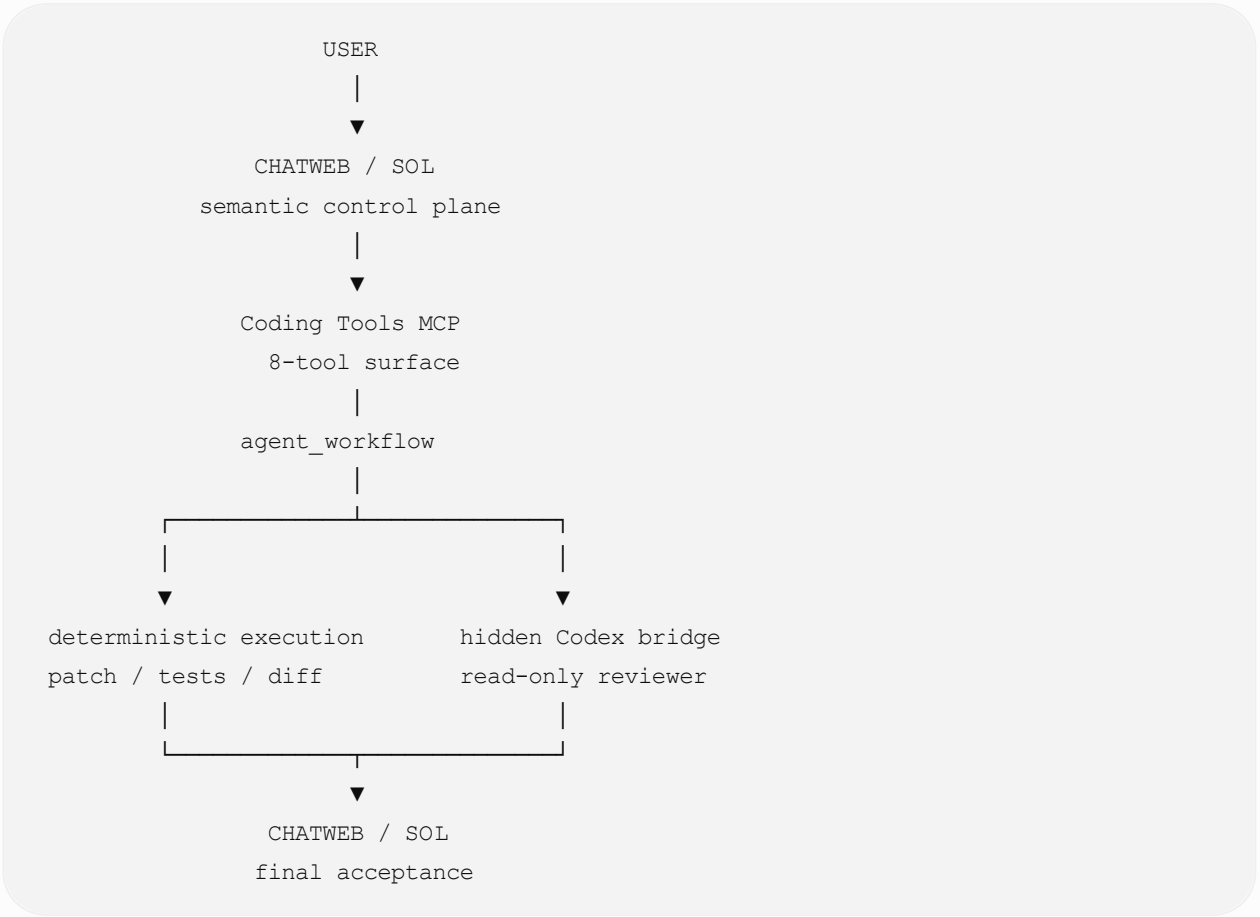
Hiện tại client vẫn nhìn thấy đúng **8 Coding Tools MCP tools**.

không còn cần:

```
codex_bridge = tool thứ 9
```

trong schema của ChatGPT.

Thay vào đó:



Đây là thay đổi kiến trúc quan trọng nhất.

Codex bridge vẫn tồn tại, vẫn có `packet / scoped / critical / history`, model/effort/deadline/circuit-breaker/token telemetry. Nhưng bridge trở thành **internal service**, không còn làm model-facing API bắt buộc.

---

4. `agent_workflow` hiện điều phối thực tế ra sao?

Đây là phần đã chạy live.

Khi tôi gọi:

```
agent_workflow(...)
```

MCP trước tiên vẫn thực hiện flow thông thường:

```
prepare
→ patch
→ focused checks
→ tests/build nếu cần
→ collect affected files
→ collect scoped diff
```

Sau khi execution đã pass, internal router mới hỏi:

```
External review có đáng tiền không?
```

Hiện tại logic tự động đã triển khai như sau:

Điều kiện	Quyết định
`dry_run`	`REVIEW=NONE`
Execution/test đang fail	`NONE`
`diagnose/document/resume`	`NONE`
Không có meaningful diff	`NONE`
Localized `bugfix/test_failure` + strong oracle	`NONE`
Diff phù hợp cho packet	`PACKET / Mini`
Packet quá lớn / thiếu evidence	không auto-spend; đề nghị `SCOPED` và trả quyền về Sol
Reviewer unavailable/capacity/deadline	không retry mù; trả quyền về Sol

Nghĩa là:

> **Codex call không còn là default. `NONE` mới là default kinh tế.**

Đây khớp trực tiếp với policy của skill: external review chỉ có giá trị khi tạo evidence độc lập thực sự; không duplicate cùng discovery loop.

---

5. Auto reviewer hiện tại cụ thể là gì?

Hiện auto path dùng:

```
PROFILE = packet
MODEL   = gpt-5.4-mini
```

c a t e d c o n t e x t v à r e w o r k , n g ư ở n g t h ự c t ế p h ả i t h ấ p

```
EFFORT  = low
TOOLS   = disabled
NETWORK = disabled
WRITE   = disabled
DEADLINE = 8s
```

Nó không được phép tự đọc repository.

Nó chỉ nhận packet dạng gần như:

```
OBJECTIVE
WORKFLOW
ACCEPT
AFFECTED_PATHS
DIFF_SHA256
PROOF
SCOPED_DIFF
QUESTION
```

và output schema bị constrain về:

```
verdict
findings
missing_proof
```

Diff cũng vừa được tối ưu thêm: **chỉ lấy** `affected_files` **của workflow hiện tại**, không lấy toàn bộ dirty worktree.

Điều này rất quan trọng với repo HR hiện đang có nhiều thay đổi khác.

---

6. Model hierarchy hiện hiểu như thế nào?

Không phải:

```
Mini < Luna < Terra < Sol
→ cứ tăng dần cho tới khi xong
```

Mà là:

```
        Task characteristics
                │
                ▼
      cheapest competent route
```

Vai trò hiện tại:

Model	Vai trò phù hợp
ChatWeb GPT-5.6 Sol	Coordinator, architecture, ambiguity, finance semantics, integration, acceptance
GPT-5.4-Mini	Packet verifier cực hẹp, evidence đã chuẩn bị sẵn
Luna	Scoped read-only investigation hoặc leaf implementation nhỏ
Terra	Cross-file reasoning, serious implementation, critical independent review
Codex Sol	Không dùng để duplicate ChatWeb Sol mặc định
Desktop History	Context source đặc biệt, không phải “model tier cao hơn”

Skill cũng quy định escalation vocabulary:

```
NONE
  ↓
PACKET / Mini
  ↓
SCOPED / Luna
  ↓
CRITICAL / Terra
  ↓
SOL_LOCAL
  ↓
HISTORY
```

nhưng **không phải cascade bắt buộc**. Có thể đi thẳng `NONE → SOL_LOCAL`, hoặc `NONE → CRITICAL`, tùy risk.

---

7. Một điểm cần phân biệt: policy đã rộng hơn runtime auto-router

Đây là trạng thái rất quan trọng hiện tại.

Skill đã hỗ trợ conceptual routing toàn diện:

```
TOOL
LUNA
TERRA
```

ic ó st ro ng or ac le va àc le ar ex pe ct ed -c os ta dv an t

```
SOL

+

NONE
PACKET
SCOPED
CRITICAL
HISTORY
```

Nhưng **runtime `agent_workflow` auto-routing hiện mới tự động hóa an toàn tới `PACKET/Mini`**.

Nó chưa tự làm:

```
Mini blocked
→ Luna
→ Terra
→ Sol
```

Tôi chủ động không triển khai cascade này lúc này vì benchmark cho thấy nó rất dễ biến thành cost/latency amplifier.

Hiện tại nếu packet không phù hợp, runtime trả ví dụ:

```
recommended_profile = scoped
```

rồi **Sol quyết định bước kế tiếp**.

Đây là hành vi tốt ở giai đoạn này.

---

8. Codex hiện là reviewer hay executor?

Trong integration live hiện tại, auto-use của Codex chủ yếu là reviewer.

Implementation chính vẫn đang theo topology:

```
ChatWeb Sol
    ↓
construct complete patch / execution intent
    ↓
agent_workflow
    ↓
MCP deterministic apply + verify
```

Chúng ta **chưa bật Codex Luna/Terra thành automatic coding executor bên trong `agent_workflow`**.

Skill cho phép làm điều đó, và bridge infrastructure đủ nền tảng để làm, nhưng chưa nên bật ngay vì cần thêm benchmark về:

```
ChatWeb patch generation
vs
Luna implementation
vs
Terra implementation
```

trên task thật có cùng oracle.

Đây là bước tối ưu lớn tiếp theo, không phải phần activation vừa xử lý.

---

9. Cache/session strategy hiện tại

Một kết quả benchmark rất tốt là continuation.

Packet cold run từng khoảng:

```
~5.2s
~0.131 estimated credits
```

Continuation cùng semantic thread:

```
~1.9s
~95% cache ratio
~0.026 estimated credits
```

Do đó skill đã chuyển từ:

```
reuse worker identity
```

sang:

```
reuse only by semantic continuity
```

Nếu là follow-up trực tiếp:

```
same task
same assumptions
same evidence basis
```

thì reuse thread có lợi.

Nếu là independent/blind review mới:

```
new thread
```

để tránh anchoring.

Skill quy định giữ context dưới dạng compact state capsule thay vì replay transcript lớn.

---

10. Critical path hiện được tối ưu như thế nào?

Mục tiêu cuối cùng không phải token thấp nhất.

Mục tiêu là:

```
TIME_TO_ACCEPTED_RESULT
+
EXPECTED_TOTAL_COST
```

Ví dụ một reviewer 0.10 credit nhưng làm user chờ thêm 25 giây có thể **tệ hơn không review**.

Vì vậy hệ thống có:

```
hard deadline
capacity breaker
no unchanged retry
no mandatory cascade
```

Live smoke vừa rồi chứng minh điều này.

Mini reviewer vượt 8s:

```
status = interrupted
deadline_hit = true
```

Nhưng:

```
workflow chính vẫn passed
```

Reviewer không được quyền giữ critical path vô thời hạn.

---

11. Trạng thái triển khai hiện tại

Hiện phần đã chạy thật là:

- `sol-lean-codex` v3 đã cài global.

of Scaling Agent System shot thấy tác vụ tuần tự có

- Coding Tools MCP live đang chạy bản source mới.
- Smart surface cố định **8 tools**.
- `codex_bridge` chạy nội bộ.
- `agent_workflow` có automatic review gate.
- Scoped affected-file diff đã triển khai.
- Mini packet reviewer đã được live-smoke qua chính `agent_workflow`.
- Hard deadline/interrupt hoạt động.
- Capacity circuit breaker đã test bằng Terra overload.
- Bridge read-only, network-off, approval never.
- Desktop live turn không bị điều khiển.
- Full history không được tự động chọn.
- Unit/integration/stability tests đều xanh.

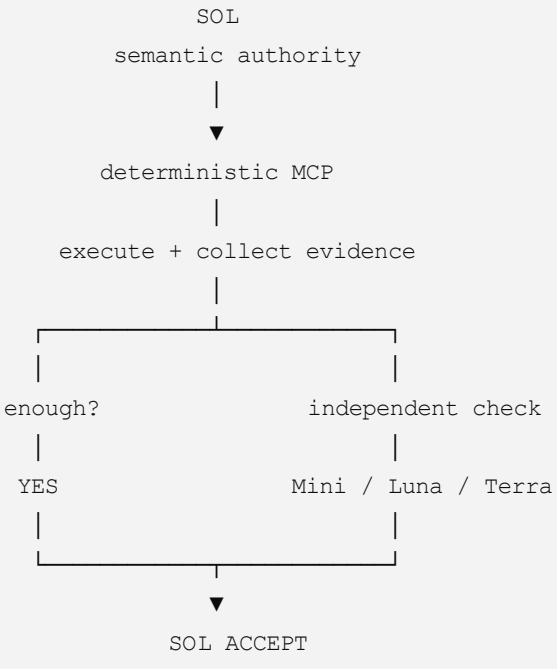
Như vậy **không còn blocker activation phía ChatGPT connector nữa**.

Tóm lại

Trước đây chúng ta nghĩ gần giống:

```
Sol → Codex → Sol
```

Hiện tại đã chuyển thành:



Và nguyên tắc quan trọng nhất hiện nay là:

> **Không dùng multi-model chỉ vì có thể. Chỉ dùng model thứ hai khi nó giảm expected rework/risk nhiều hơn chi phí và latency mà nó tạo ra.**

thể giảm mạnh hiệu quả khi thêm multi-agent, trong

Đó là trạng thái mà skill và runtime hiện đã hội tụ.

Bước nghiên cứu tiếp theo có giá trị cao nhất không còn là bridge nữa, mà là benchmark **executor routing**: cùng 10–20 task thực tế, so `ChatWeb Sol → MCP patch`, `Luna executor`, và `Terra executor` về `time-to-green`, số lần sửa lại, token/credit, acceptance rate và integration defects. Từ dữ liệu đó mới nên bật automatic Luna/Terra implementation routing.

Bước nghiên cứu tiếp theo có giá trị cao nhất không còn là bridge nữa, mà là benchmark **executor routing**: cùng 10–20 task thực tế, so `ChatWeb Sol → MCP patch`, `Luna executor`, và `Terra executor` về `time-to-green`, số lần sửa lại, token/credit, acceptance rate và integration defects. Từ dữ liệu đó mới nên bật automatic Luna/Terra implementation routing.
Triển khai đi và đưa tôi kết luận cuối cùng đánh giá đưa ra file md tại thư mục tool codex MCP này làm tài liệu nghiên cứu kỹ sau này

Worked for 51m 55s

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi
n
g
,
Công cụ được gọi
c
Công cụ được gọi
e
n
Công cụ được gọi
t
r
Công cụ được gọi
a
l
Công cụ được gọi
i
z
Công cụ được gọi
e
Công cụ được gọi
d
c
Công cụ được gọi
o
Công cụ được gọi
o
r
Công cụ được gọi
d
i
Công cụ được gọi
n
Công cụ được gọi
a
t
Công cụ được gọi
i
o
Công cụ được gọi
n
c
Công cụ được gọi
ũ
Công cụ được gọi
n
g
Công cụ được gọi
h
ạ
Đã hoàn tất benchmark executor routing và lưu tài liệu nghiên cứu trong chính repo Coding Tools MCP.

Kết quả chính từ **30 official observations = 10 task × 3 routes**:

Công cụ được gọi
c
h
ế
Công cụ được gọi
e
r
Công cụ được gọi
r
o
r

- **ChatWeb Sol → MCP:** `10/10 accepted` **— 100%**.
- **Codex Luna autonomous:** `2/10` **— 20%**; loại capacity failures vẫn chỉ `33.3%`.
- **Codex Terra autonomous:** `6/10` **— 60%**; loại capacity failures đạt `75%`.
- Không có trường hợp focused test xanh nhưng full 38-test gate đỏ.
- Luna gặp capacity `4/10`; Terra `2/10`.

- Median observed TTA: Sol **42.5s**, Luna **47.3s**, Terra **48.9s**.
- Luna→Sol serial fallback làm mean TTA tăng khoảng **22.8s/task (+39.9%)**.
- Terra→Sol fallback làm mean TTA tăng khoảng **10.7s/task (+18.6%)**.
- Ngay cả một “perfect router” biết trước route thắng cũng chỉ cải thiện median khoảng **0.36s**; mean gain chủ yếu đến từ vài Sol tail outliers.
- Tính cả failed attempts có telemetry, lower-bound observed cost/accepted outcome là khoảng **0.602 credits với Luna** và **0.399 credits với Terra**. Luna rẻ hơn mỗi successful call nhưng failure waste làm policy economics xấu đi.

Kết luận triển khai: chưa bật automatic Luna/Terra autonomous implementation routing. Giữ **ChatWeb Sol → deterministic MCP** làm default cho localized strong-oracle work. Luna tiếp tục phù hợp với packet review/narrow leaf; hướng nghiên cứu tiếp theo nên là **Luna patch-only/no-tools**, không phải autonomous shell loop. Terra chỉ nên dùng như **gated scoped executor** cho cross-file/reasoning-heavy work, isolated worktree, một attempt, deadline cứng và external grading. Không dùng `Luna → Terra → Sol` cascade mặc định.

Một finding đáng giữ lại: **5/6 Terra accepted runs chạm hard deadline nhưng patch trên disk đã đủ pass external grading**. Nếu sau này bật writable executor, nên có primitive `interrupt → freeze diff → external oracle → accept/revert`, thay vì coi `turn interrupted` là failure.

Báo cáo chi tiết 591 dòng đã lưu tại:

`E:\DEV\Tools-Codex-web\docs\research\EXECUTOR_ROUTING_BENCHMARK_2026-08-12.md`

SHA-256:

`5511A490DAB189E1C00244B8BB1AD46D13D7342572EACB893630FCF7B59A197A`

Các fixture, raw scripts và thư mục benchmark tạm trong `.coding-tools` đã được dọn. Repo không bị commit; ngoài file nghiên cứu mới chỉ còn đúng change-set bridge 5 file đã có trước benchmark. `git diff --check` vẫn sạch.

Báo cáo cũng ghi rõ các hạn chế quan trọng—đặc biệt ChatWeb Sol có lợi thế accumulated context, mỗi model/task mới chạy một official sample, và per-task integration gate hiện là 38-test Python package boundary—nên **quyết định “không promote” là đủ bằng chứng**, còn muốn promote Terra sau này phải chạy vòng cross-file benchmark có repetitions.

Vì sao không thấy báo cáo model Sol ultra. kế hoạch và phối tất cả mô hình này như nào.

Worked for 1m 22s

Công cụ được gọi

Công cụ được gọi

Đúng — **báo cáo hiện tại thiếu Codex Sol Ultra**, và nếu mục tiêu là thiết kế router cho toàn bộ tầng model thì đó là một khoảng trống cần bổ sung.

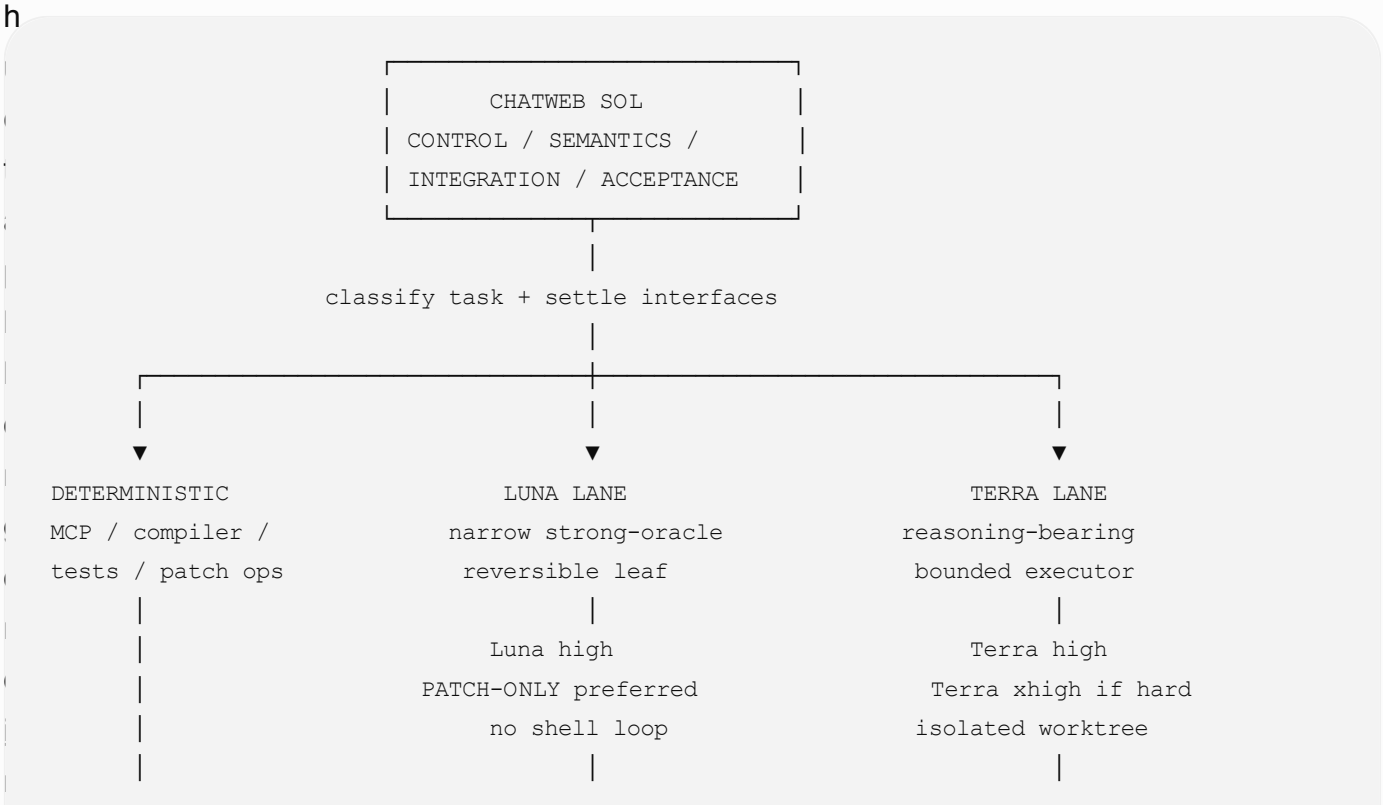
Lý do là benchmark vừa rồi bám đúng ma trận ban đầu: `ChatWeb Sol → MCP`, `Luna executor`, `Terra executor`. Tôi đã dùng ChatWeb Sol làm baseline chất lượng, nhưng **ChatWeb Sol không đồng nghĩa với Codex** `gpt-5.6-sol + effort=ultra`. Effort của ChatWeb Sol là ẩn, nên không thể gắn nhãn “Ultra” cho nó.

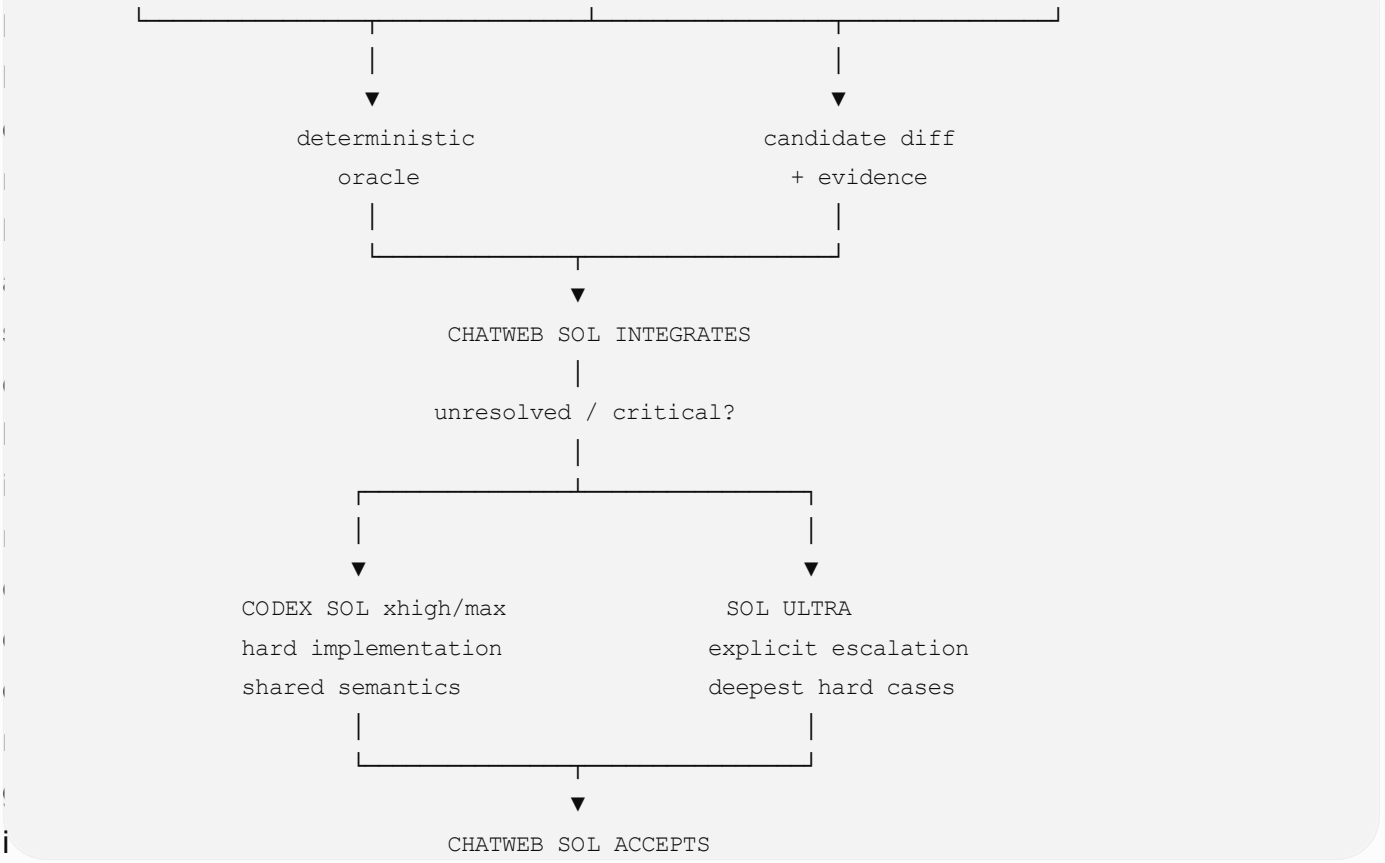
Tôi vừa probe trực tiếp Codex runtime hiện tại. Runtime thực tế đang expose:

- `gpt-5.6-sol`: `low / medium / high / xhigh / max / ultra`
- `gpt-5.6-terra`: `low / medium / high / xhigh / max / ultra`
- `gpt-5.6-luna`: `low / medium / high / xhigh / max`
- ngoài ra còn `gpt-5.3-codex`, `gpt-5.5`, `gpt-5.4`, `gpt-5.4-mini`, `gpt-5.3-codex-spark`.

Một điểm còn quan trọng hơn: benchmark trước dùng **Luna low** và **Terra medium**, trong khi routing policy của `sol-lean-codex` hiện định hướng implementation ở khoảng **Luna high**, **Terra high/xhigh**, **Sol xhigh/max**. Vì vậy report trước là bằng chứng tốt để nói *“Luna-low/Terra-medium autonomous loop hiện không đáng bật”*, nhưng **chưa đủ để kết luận toàn bộ Luna/Terra implementation tier**. Tôi sẽ coi đây là limitation cần sửa ở benchmark v2.

Kiến trúc phối model tôi đề xuất





Tức là **không có "model mạnh nhất làm tất cả"**. Sol là control plane; tier thấp hơn chỉ nhận work packet có biên rõ ràng.

Vai trò cụ thể

Tier	Vai trò chính	Effort dự kiến	Có write?	Khi dùng
Deterministic MCP	Transform, test, compiler, grep, patch đã biết	—	Có	Không cần judgment
Mini	Evidence-only reviewer	low	Không	Packet đã đủ evidence
Luna	Cheap leaf / candidate patch	**high**	Patch candidate; hạn chế agent loop	Narrow + strong oracle + reversible
Terra	Default delegated reasoning worker	**high**	Isolated worktree	Cross-file/bounded semantic implementation
Terra	Hard worker	**xhigh**	Isolated worktree	Long context/ambiguity nhưng interfaces đã settled

Tier	Vai trò chính	Effort dự kiến	Có write?	Khi dùng
Sol Task	Architecture/shared semantics	**xhigh**	Có thể	High integration width
Sol Critical	Critical/expensive reasoning	**max**	Có thể	Security/data/finance/migration/high rework
Sol Ultra	Exceptional deep lane	**ultra**	Isolated	Max chưa đủ hoặc cực kỳ consequential/complex
ChatWeb Sol	Coordinator	hidden	MCP-controlled	Final semantic authority + integratio + go/no-go

Sol Ultra không nên là default executor. Nó nên giống một "deep critical lane": chỉ mở khi giá trị của reasoning bổ sung lớn hơn latency/token cost. Đặc biệt không nên gọi Ultra cho một one-line bug có strong unit test.

Hai plane phải tách biệt

Implementation routing và review routing là hai quyết định khác nhau.

Implementation:

```
TOOL
  ↓
LUNA high          narrow / strong oracle
  ↓
TERRA high/xhigh  bounded semantic worker
  ↓
SOL xhigh/max     shared semantics / high integration
  ↓
SOL ultra         exceptional escalation
```

Nhưng **không chạy như cascade tuần tự**. Đây là *classification ladder*, không phải:

```
Luna thử
→ fail thì Terra
→ fail thì Sol
→ fail thì Ultra
```

Benchmark vừa rồi đã chứng minh kiểu cascade đó đốt critical-path latency.

Review plane riêng:

```
NONE
  ↓
Mini PACKET
  ↓
Luna SCOPED
  ↓
Terra CRITICAL
  ↓
return to ChatWeb Sol
  ↓
Sol Ultra only if the unresolved question truly warrants it
```

Một task có thể chẳng hạn được **Terra implement nhưng Mini review**, hoặc **Sol implement mà không cần external review**, hoặc **Luna patch-only rồi Sol inspect diff**. Tier executor không quyết định tier reviewer.

Khi nào chạy song song?

Tôi sẽ chỉ fan-out khi các lane thực sự độc lập. Ví dụ một feature lớn:

```
ChatWeb Sol
  |
  ├─ Terra A: implementation backend adapter       [worktree A]
  ├─ Terra B: investigate UI/data-flow contract    [read-only initially]
  ├─ Luna: mechanical migration leaf               [worktree C]
  └─ Mini/Luna: blind evidence review              [read-only]
```

Sol vẫn giữ interface contracts và integration. Khi Terra A thay đổi một interface mà B phụ thuộc, B phải bị stop/rebase/repacket — không để workers tự merge assumptions.

Với task tuần tự, high integration width hoặc shared mutable files:

```
Sol -> MCP -> test -> Sol
```

Sai tốt hơn bốn agents.

Sol Ultra nên benchmark thế nào?

Không nên chỉ thêm một cột Ultra vào benchmark cũ rồi kết luận. Tôi đề xuất benchmark v2 theo hai tầng.

Tầng 1 — effort calibration, 4–6 task đại diện: chạy cùng task với `Luna high`, `Terra high`, `Terra xhigh`, `Sol xhigh`, `Sol max`, `Sol ultra`. Mục tiêu là tìm **effort Pareto point của từng tier**, vì chạy mọi effort trên mọi task sẽ tốn rất nhiều credit mà ít thông tin mới.

Tầng 2 — routing benchmark 12–20 task: chỉ giữ cấu hình thắng của mỗi tier, ví dụ:

```
A  ChatWeb Sol -> MCP              control
B  Luna high patch-only
C  Terra high scoped executor
D  Terra xhigh hard-worker
E  Codex Sol xhigh
F  Codex Sol max
G  Codex Sol ultra
```

Sau đó đo cùng các metric:

```
first-pass acceptance
time-to-first-write
time-to-first-green
time-to-accepted-result
correction count
integration defects
input/cached/output tokens
credits including failed attempts
capacity failures
Sol integration/review burden
```

Riêng `Sol max vs Sol ultra`, tôi sẽ thêm nhóm **hard tasks**. Nếu chỉ dùng các mutation one-line như benchmark trước thì Ultra gần như chắc bị penalize vì overhead nhưng ta không học được khả năng thực sự của nó.

Chính sách production tôi sẽ hướng tới

Router cuối cùng không nên kiểu “task khó = model to hơn”. Nó nên quyết định theo `integration width + semantic uncertainty + oracle strength + reversibility`:

```
if deterministic:
    MCP

elif narrow
     and strong_oracle
     and low_integration_width
     and cheap_revert:
    Luna-high PATCH_ONLY

elif settled_interfaces
     and bounded_cross_file_reasoning:
    Terra-high

elif Terra-class but long/ambiguous:
    Terra-xhigh

elif shared_semantics
     or high_integration_width
     or architecture:
    Sol-xhigh
```

```
elif critical and hard:
    Sol-max

elif still under-proven
     and expected failure/rework cost >> Ultra cost:
    Sol-ultra

final_acceptance:
    ChatWeb Sol
```

Capacity failure chỉ cho phép **một policy-approved fallback**, không chạy cả ladder.

Điểm tôi sẽ sửa trong tư duy sau benchmark vừa rồi là: **chưa nên nói "Luna/Terra bị loại" một cách tuyệt đối**. Chính xác hơn là **Luna-low autonomous và Terra-medium autonomous bị loại cho tested task class**. `Luna-high patch-only`, `Terra-high/xhigh`, và đặc biệt **Codex Sol xhigh/max/ultra chưa có dữ liệu paired đủ để quyết định production routing**.

Do đó, **benchmark Sol Ultra + effort calibration là nghiên cứu tiếp theo nên làm trước khi sửa automatic router**. Report hiện tại nên được xem là **Phase 1 – topology baseline**, chưa phải final all-model router benchmark.

bạn đánh giá riêng rôi việc phối hợp, thêm cả phương án lên dùng SOl xhigh thôi. Nghiên cứu các bài báo khoa học, cá thảo luận, các đánh giá thwucj tế trong cả chủ đề này với bài test mới. Đưa ra phương án kết hợp để giải quyết xử lý công việc, dùng như nào, skill, mcp, dùng codex desktop, chatweb MCP. Phân tích sâu rõ để hướng đạt được là hiệu suất, tối ưu, giảm chi phí. Viết chi tiết ra file md để đánh giá trước khi chốt phương án giải quyết

Worked for 25m 8s

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi
Công cụ được gọi
Công cụ được gọi

```
        |
        ├── bounded candidate → Mini low patch-only
        |                       Luna high nếu khó hơn
        |
        └── deep/long work → Codex Desktop Sol xhigh
                                        |
                                        ▼
                                MCP proof + Sol accept
```

Trong đó **Sol xhigh trở thành highest routine deep tier**, còn `max` và đặc biệt `ultra` là exception. Điều này phù hợp với cách OpenAI mô tả Ultra: nó là highest-capability setting phối hợp nhiều agents trên các workstream song song, không đơn giản là xhigh với nhiều reasoning token hơn. OpenAI +1

Benchmark mới đã thay đổi đáng kể kết luận về Luna/Mini

Ở Phase 1, Luna autonomous rất kém. Nhưng Phase 2 cho thấy nguyên nhân quan trọng nằm ở **agent topology**, không chỉ model.

Trên cùng ba packet T02/T04/T06 đã localize tốt:

Route	Accepted	Mean TTA	Mean credits
Mini low patch-only	**3/3**	9.11s	**0.1585**
Luna high patch-only	**3/3**	**7.89s**	0.2597
Sol xhigh patch-only	**3/3**	11.23s	1.3173

Mở rộng thêm T10, cả Mini-low và Luna-high đều đạt **4/4**. Mini trung bình khoảng 0.138 credit/candidate accepted trong bốn observations; Luna khoảng 0.273.

Trong khi đó, **Sol xhigh autonomous shell-loop = 0/3 official pre-stop observations** trên T02/T04/T06, đều tốn khoảng 31 giây mà chưa tạo production write. Cùng chính Sol xhigh nhưng đổi sang `patch-only/no-tools` lập tức thành **3/3 accepted trong khoảng 6.7–8.3 giây model wall**.

Đây là finding quan trọng nhất của Phase 2:

> **Localization + packet design + tool topology có thể quan trọng hơn việc tăng model tier.**

Phương án “chỉ dùng Sol xhigh”

Tôi phân thành hai loại.

Không nên làm:

```
mọi Codex call
→ Sol xhigh
```

Trên common three leaf tasks, nó đắt khoảng **5,1× Luna-high** và **8,3× Mini-low**, mà không tăng acceptance.

Nhưng có một phiên bản tôi **khuyến nghị mạnh**:

```
ChatWeb Sol + MCP
    = routine work

Codex Desktop
    = deep-work-only environment
    = GPT-5.6 Sol xhigh mặc định
```

Như vậy người dùng Codex Desktop không phải liên tục đổi model. Desktop trở thành một workstation chuyên cho công việc sâu/dài, nên **Sol xhigh mặc định ở Desktop hoàn toàn hợp lý**. Routine/cheap work không đi qua Desktop ngay từ đầu.

Max và Ultra

Tôi đã calibration cùng đúng T06 packet:

```
Sol xhigh → PASS, TTA 11.65s, ~1.339 cr
Sol max   → PASS, TTA  9.30s, ~1.351 cr
Sol ultra → PASS, TTA  9.80s, ~1.384 cr
```

Một sample không cho phép kết luận Max nhanh hơn xhigh. Điều đáng chú ý là **cả ba đều cùng quality outcome**; không có evidence nào biện minh Ultra cho leaf đó.

Ngoài ra, thử nghiệm Ultra này chạy trong **constrained no-tools packet profile**, không phải full native Ultra multi-agent. Tôi chủ động không chạy native Ultra swarm trên một one-invariant bug vì đó là sai task topology và có thể rất tốn kém. OpenAI xác nhận Ultra được thiết kế cho parallel workstreams; Google Research cũng cho thấy multi-agent có thể tăng ~81% trên task parallelizable nhưng giảm 39–70% trên sequential planning trong controlled study 180 configurations. OpenAI +1

Có cả một field report trong official Codex GitHub về Ultra tạo nhiều subagents và tiêu credit rất cao. Đây chỉ là **user report chưa được OpenAI xác nhận**, nhưng đủ để xem như operational warning, không phải benchmark chính thức. GitHub

Terra

Terra vẫn chưa bị loại về capability.

Nhưng `Terra high autonomous` Phase 2 cho:

```
T04 → deadline/no write
T06 → capacity
T10 → capacity
```

nếu **chưa được promote trong runtime hiện tại**.

Phase 1 Terra-medium từng đạt 6/10 và 75% nếu loại capacity failures. Vì vậy tôi giữ Terra ở trạng thái:

```
TERRA_HIGH_SCOPED_EXECUTOR
= experimental
= cross-file/reasoning-bearing only
= isolated worktree
```

Cần benchmark riêng bằng các task cross-file thật sau khi sửa deadline/telemetry; không nên dùng one-line mutations để quyết định số phận Terra.

Nghiên cứu bên ngoài cũng hội tụ với kiến trúc này

MTRouter cho thấy cost-aware multi-turn routing có thể tiết kiệm đáng kể và một đặc điểm của policy tốt là **ít model switches hơn**, hỗ trợ việc tránh `Mini→Luna→Terra→Sol` cascade. ACL Anthol...

ContextualRouter cho thấy retrieval từ các task tương tự trước đây, thậm chí kNN đơn giản, có thể cạnh tranh với router phức tạp; LLMRouterBench cũng thấy nhiều sophisticated routers không reliably vượt simple baselines. Vì vậy tôi đề xuất **route ledger + nearest-similar-task**, chưa cần ML router. ACL Antholo... +1

ToolScope báo cáo context-aware tool filtering/merging tăng tool-selection accuracy 8.38–38.6% trong các benchmark của họ. Đây là bằng chứng tốt để **giữ Coding Tools MCP ở compact 8-tool surface**, không expose từng model thành tool riêng. ACL Anthol...

RETRACE cho thấy independent patch verification có thể nâng Pass@1 đáng kể, củng cố topology **primary executor + independent evidence reviewer**, thay vì cùng một Sol tự suy nghĩ lại cùng rationale. ar...

Phương án tôi đề nghị chốt sau khi bạn đọc report

```
CONTROL
ChatWeb Sol

DETERMINISTIC
Coding Tools MCP

CHEAP PATCH CANDIDATE
Mini low / no-tools

HARDER LEAF
Luna high / no-tools

DEEP CODE
Codex Desktop Sol xhigh

CROSS-FILE EXPERIMENT
```

```
Terra high

CONSEQUENTIAL ESCALATION
Sol max

TRUE PARALLEL EXTREME
Sol ultra

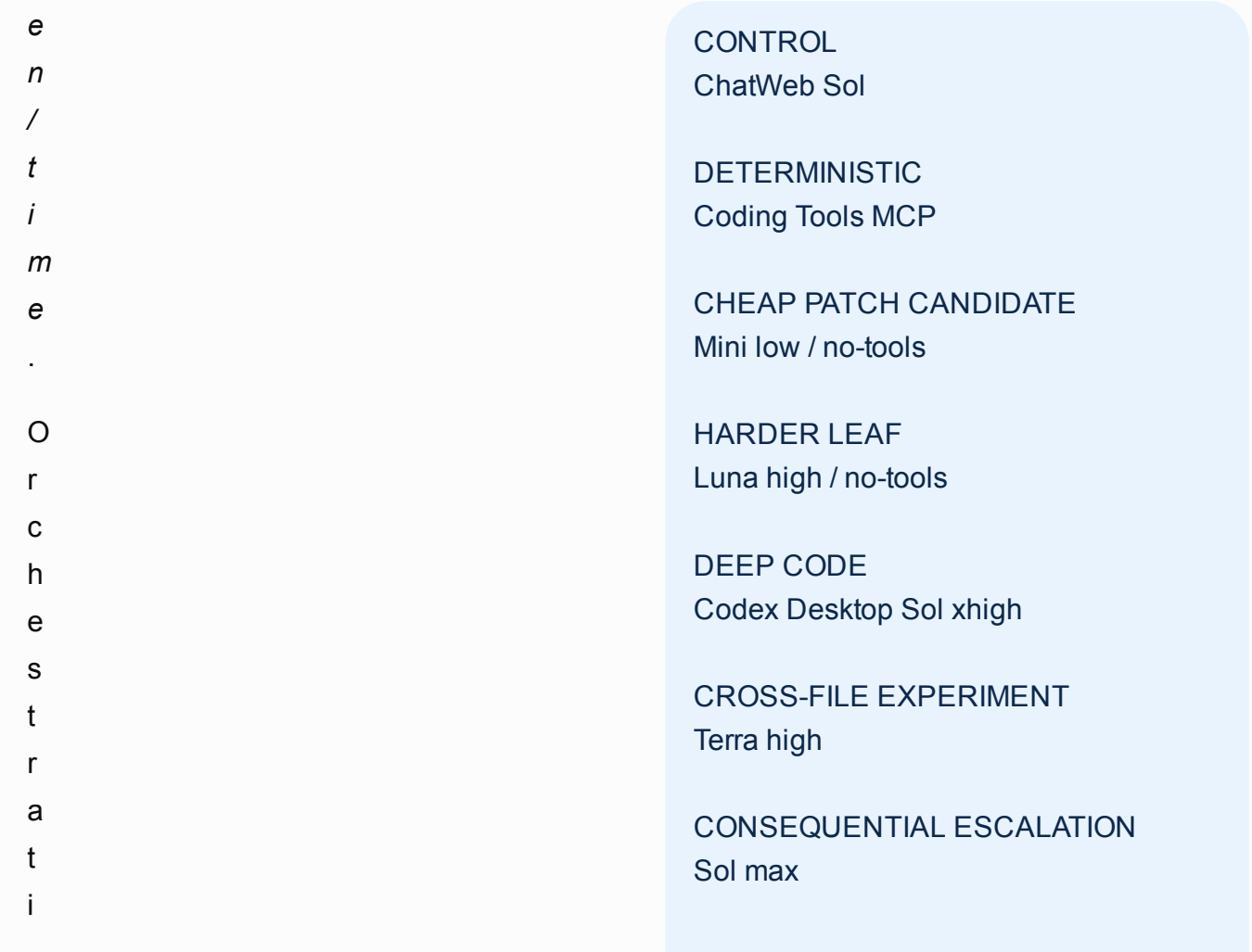
FINAL ACCEPTANCE
ChatWeb Sol + deterministic evidence
```

Và **execution tier tách hoàn toàn khỏi review tier**. Không vì Terra viết code mà Terra phải review; không vì Mini sinh candidate mà bắt buộc Luna review.

Tài liệu cũng đã thiết kế chi tiết phần triển khai tiếp theo: symbol-aware packet builder, hidden `candidate_patch` profile, `time_to_first_write`, `time_to_first_green`, sửa semantics deadline >30s, route ledger, cache/capacity-aware routing, Desktop state capsule và promotion gates có statistical safety.

Tôi chưa áp dụng các thay đổi đó vào skill/MCP. Report này đúng mục đích bạn yêu cầu: làm tài liệu decision-grade để đánh giá trước khi chốt kiến trúc production.

Flow/token/time.



TRUE PARALLEL EXTREME
Sol ultra

FINAL ACCEPTANCE
ChatWeb Sol + deterministic evidence
làm như phương án này cho triển khai đi

Stopped thinking

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Triển khai hoàn thiện tiếp

Worked for 16m 22s

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Đã triển khai hoàn thiện phần production theo phương án đã chốt.

Hiện kiến trúc runtime thực tế là:

```
USER
  ↓
CHATWEB GPT-5.6 SOL
  Control / semantics / architecture
  Integration / final acceptance
  ↓
CODING TOOLS MCP
  Evidence + localization + writer + oracle
  ↓
ROUTER
  ├─ deterministic → MCP
  ├─ localized strong-oracle → Mini low / no-tools
  ├─ harder bounded leaf → Luna high / no-tools
  ├─ deep / broad → Codex Sol xhigh
  ├─ explicit cross-file experiment → Terra high
  ├─ consequential explicit → Sol max
  └─ parallel extreme explicit → Sol ultra
  ↓
MCP applies candidate + deterministic proof
  ↓
CHATWEB SOL final acceptance
```

Những gì đã production

Router source hiện encode chính xác:

Route	Model	Effort	Deadline	Policy
`mini`	GPT-5.4-Mini	low	10s	auto
`luna`	GPT-5.6-Luna	high	15s	auto
`terra`	GPT-5.6-Terra	high	25s	**explicit experimental**
`sol_xhigh`	GPT-5.6-Sol	xhigh	30s	auto deep-code
`sol_max`	GPT-5.6-Sol	max	30s	explicit + high-cost ack
`sol_ultra`	GPT-5.6-Sol	ultra	30s	explicit + high-cost ack

`Mini` và `Luna` **không được write repository trực tiếp**. Chúng chỉ trả structured candidate; MCP kiểm tra allowed paths → apply → chạy oracle.

`Terra`, `Sol xhigh`, `Max` cũng chạy read-only ở bridge và trả bounded candidate change set. Vì vậy không có chuyện nhiều model cùng sửa authoritative checkout.

Chỉ `parallel_extreme / Sol ultra` bật:

```
multi_agent = true
```

Những profile khác đều không fan-out. Ultra cũng không bao giờ được auto-select.

Auto router hiện hoạt động như sau

```
No judgment needed
    → DETERMINISTIC MCP

Bug/test + failing strong oracle
+ <=2 localized paths
+ <=24 KB relevant context
    → MINI LOW

Bounded leaf
+ <=4 paths
+ <=60 KB relevant context
    → LUNA HIGH

Anything broader/deeper
    → SOL XHIGH
```

Terra không nằm giữa Luna và Sol trong auto cascade.

Không có:

```
Mini → fail → Luna → fail → Terra → fail → Sol
```

Candidate bị blocked thì mặc định **trả evidence về ChatWeb Sol**, không đốt tiếp cả ladder.

Live test đã chạy thật

Sau reload, native MCP đổi từ PID `39580` → `44968` mà không restart Tunnel/Electron.

Deterministic smoke trả trực tiếp:

```
execution_route.selected = deterministic
reason = workflow_prefers_local_tools
```

Sau đó tôi chạy một bug fixture thực với failing oracle. Auto router chọn:

```
selected = mini
profile  = candidate_mini
model    = gpt-5.4-mini
effort   = low
```

Kết quả:

```
candidate_status = ready
confidence       = high
model_wall       ≈ 5.078 s
estimated_credit ≈ 0.1436
MCP apply        = PASS
focused oracle   = PASS
```

File fixture sau đó đã được xóa hoàn toàn.

Đây là bằng chứng end-to-end:

```
ChatWeb
→ agent_workflow
→ router
→ hidden Codex bridge
→ Mini candidate
→ MCP scope validation
→ MCP write
→ deterministic test
→ result back to ChatWeb
```

Acceptance cuối

Trên **HEAD hiện tại**:

- Python Coding Tools MCP: **44/44 PASS**

- Node stability: **5/5 PASS**
- `git diff --check`: PASS
- `git status`: **clean**
- `main == origin/main`

Router implementation đã nằm trong Git history và được push/sync với `origin/main`.

Skill cũng đã được nâng lên policy mới

Tôi đã cập nhật `sol-lean-codex` thành policy tương ứng:

```
CONTROL                 ChatWeb Sol
DETERMINISTIC           Coding Tools MCP
CHEAP PATCH             Mini low/no-tools
HARDER LEAF             Luna high/no-tools
DEEP CODE               Sol xhigh
CROSS-FILE EXPERIMENT   Terra high
CONSEQUENTIAL           Sol max
PARALLEL EXTREME        Sol ultra
FINAL ACCEPTANCE        ChatWeb Sol + deterministic evidence
```

Đồng thời thêm:

- Phase-2 benchmark evidence;
- topology-first routing;
- candidate-only semantics;
- không serial cascade;
- Terra experimental gate;
- Max/Ultra high-cost gate;
- Ultra nested-swarm protection;
- accepted-result economics;
- capacity/cache/deadline handling.

Skill validation:

```
quick_validate       PASS
comparator tests     10/10 PASS
files in package     13
__pycache__          none
```

Bundle SHA-256:

`8d0260c813b906be53ae08db5ff0703430ce634d7d1b680c3fba5a2d0981b6ba`

Download skill.zip

Hai giới hạn còn lại không ảnh hưởng auto production

1. Global Windows skill chưa được overwrite. Bản tại:

`C:\Users\Admin\.codex\skills\sol-lean-codex`

vẫn là v3 cũ, `SKILL.md` hash `747a21fa...`.

Lý do là MCP đang chạy `--permission-mode safe` và **write ra** `C:\Users\Admin\.codex` **bị chặn** `filesystem_escape`. Tôi không hạ sandbox hoặc đổi permission mode chỉ để cài skill. Bundle mới đã được tạo/validated đầy đủ.

Quan trọng: **runtime router không phụ thuộc bản global skill đó**, nên routing mới đang chạy production ngay bây giờ.

2. Schema của chính conversation hiện tại vẫn cache bản `agent_workflow` **cũ.** Vì vậy `execution_route=` chưa xuất hiện trong function schema mà ChatWeb hiện nhìn thấy. Nhưng `auto` đã chạy thật vì logic nằm server-side — live smoke ở trên chứng minh điều này.

Khi client refresh schema/new conversation, các manual controls sẽ xuất hiện:

```
execution_route:
  auto
  deterministic
  mini
  luna
  terra
  sol_xhigh
  sol_max
  sol_ultra

allow_high_cost:
  false | true
```

Cho đến lúc đó, **auto router vẫn hoạt động đầy đủ**; chỉ việc ép thủ công `Terra/Max/Ultra` từ wrapper hiện tại chưa tiện.

Trạng thái chốt

Phần quan trọng nhất đã đạt:

> **ChatWeb Sol là bộ não và acceptance authority; MCP là execution/evidence authority; model Codex được dùng như bounded candidate/deep-reasoning lanes, không phải các agent tự do cùng tranh quyền điều phối.**

Và `Sol xhigh` đã trở thành **deep-code default**, đúng phương án bạn vừa duyệt. Mini/Luna xử lý phần rẻ, Sol xhigh giữ phần khó, Terra còn trong experimental gate, còn Max/Ultra chỉ dùng khi chi phí thất bại thực sự đáng để trả thêm.

latency, không phải workerutilization. Tuynh

Xem trước kỹ năng

Nháp

Đang tải chi tiết kỹ năng…

Cài đặt

Hôm qua 20:03

Bạn cài skill cho máy và update skill cho chat web chưa. Giờ tôi giao việc trên chatweb MCP là ok đúng không

Đã xử lý trong 1m 1s

Công cụ được gọi

Chưa đồng bộ **skill ở cả hai nơi**, nhưng **MCP runtime thì đã sẵn sàng để bạn giao việc ngay trên ChatWeb**.

Tôi vừa kiểm tra lại:

- **Coding Tools MCP runtime: OK.** Live smoke vừa chạy trả `execution_route.selected = deterministic`, nghĩa là router mới đang thực sự active server-side. Với task thật, auto-router có thể chọn Mini/Luna/Sol xhigh theo logic mới.
- **Skill global trên máy Windows: vẫn là v3 cũ.** `C:\Users\Admin\.codex\skills\sol-lean-codex\SKILL.md` hiện có SHA-256 `747A21FA...748A024`. Lần trước safe-mode MCP chặn ghi ra ngoài workspace (`filesystem_escape`), nên tôi không lách sandbox để overwrite.
- **Skill mà ChatWeb hiện đang load: cũng vẫn là v3 cũ.** Tôi vừa đọc resource hiện tại; nó vẫn mô tả `Luna → Terra default worker → Sol`, chưa phải policy mới `Mini candidate → Luna → Sol xhigh, Terra explicit`.

Vì vậy câu trả lời cho **"giờ giao việc trên ChatWeb MCP có OK không?" là: Có.** Với `agent_workflow` mặc định, phần quan trọng nhất nằm server-side và đã production:

```
ChatWeb Sol
   ↓
Coding Tools MCP
   ↓
auto router
   ├─ deterministic
   ├─ Mini low candidate
   ├─ Luna high candidate
   └─ Sol xhigh deep-code
   ↓
MCP apply + test
```

Ngay cả khi ChatWeb skill v3 chưa được cập nhật, **auto router MCP mới vẫn hoạt động**. Vấn đề là ChatWeb có thể ra quyết định orchestration thủ công theo policy cũ trong một số task phức tạp; do đó để hệ thống hoàn toàn nhất quán thì vẫn nên cập nhật skill ở cả ChatWeb và Codex Desktop.

Bundle v4 đã validate sẵn:

Download skill.zip

Sau khi thay skill ở `/skills` của ChatWeb bằng bundle này, nên mở **chat mới** để skill metadata/instructions được discovery lại. Với máy Windows, thay thư mục global `C:\Users\Admin\.codex\skills\sol-lean-codex` bằng nội dung bundle mới rồi restart/reopen Codex Desktop.

Hiện tại bạn đã có thể giao việc cho tôi trên ChatWeb + MCP bình thường. Auto execution routing đã chạy đúng; chỉ phần skill policy phía ChatWeb/Desktop còn cần đồng bộ để các quyết định nâng lên Terra/Max/Ultra và Sol xhigh được nhất quán tuyệt đối.

Xem trước kỹ năng

Nháp

Đang tải chi tiết kỹ năng…

Cài đặt

Truy cập trên máy cài đi.
cập nhập luôn trên chatweb cho tôi

Đã xử lý trong 17m 38s

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Phần **máy Windows đã cài xong hoàn toàn**.

- Global skill: `C:\Users\Admin\.codex\skills\sol-lean-codex`
- Bản mới: **v4**, 13 files.
- `SKILL.md` SHA-256: `F682300F318285D1B529233DE945ABC121A5A807BD4E52090132E274581AD4D9`
- Comparator trước và sau cài: **10/10 PASS**
- Backup bản cũ: `C:\Users\Admin\.codex\skills\sol-lean-codex.backup-20260812-202137`
- Bundle ZIP đã xác minh bit-for-bit: `8D0260C813B906BE53AE08DB5FF0703430CE634D7D1B680C3FBA5A2D0981B6BA`
- Các file transfer/install tạm đã dọn; repo Tools trở lại clean.

ChatWeb thì chưa thể ghi trực tiếp từ phiên này. Tôi đã kiểm tra lại resource sau khi cài máy: ChatWeb vẫn đang load v3 cũ (`Luna → Terra default → Sol`). Control-plane hiện tại của tôi chỉ cho **đọc Skill**, không có action install/update Skill Library; tôi cũng đã kiểm tra Plugin Management và nó không cung cấp thao tác cập nhật custom Skill. Vì vậy tôi không thể tuyên bố ChatWeb đã được update khi thực tế chưa có quyền ghi.

Bundle v4 đúng để đưa vào ChatWeb là: Download skill.zip. Bạn chỉ cần vào `/skills` , thay/upload `sol-lean-codex` bằng bundle này, rồi dùng một chat mới để skill được discovery lại. Tôi đã thử tra tài liệu OpenAI hiện hành để xác nhận thêm UI cụ thể nhưng web lookup đang trả 503, nên tôi không suy diễn thêm tên nút.

Bạn vẫn có thể giao việc trên ChatWeb + MCP ngay bây giờ. MCP router mới đã chạy server-side và live-test thành công, nên `agent_workflow` auto hiện đã thực sự dùng:

```
Deterministic → MCP
Localized strong oracle → Mini low
Harder bounded leaf → Luna high
Deep/broad work → Sol xhigh
```

```
Terra → explicit experimental
Sol max → explicit + high-cost gate
Sol ultra → explicit parallel-extreme + high-cost gate
Final acceptance → ChatWeb Sol
```

Khác biệt duy nhất còn lại là **Skill Library phía ChatWeb chưa được thay v4**, nên với chat mới trước khi bạn upload bundle, phần reasoning/orchestration phía ChatWeb vẫn đọc policy v3. MCP auto-router bên dưới thì đã là bản mới.

Xem trước kỹ năng

Nháp

Đang tải chi tiết kỹ năng…

Cài đặt

Vào máy làm lệnh shutdown máy đi. Và kiểm tra lại xem máy tắt chưa

Đã xử lý trong 7m 4s

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Đã shutdown máy thành công.

- `Stop-Computer -Force` được Windows chấp nhận với **exit code 0**.
- Kiểm tra lần 1 sau shutdown: MCP **timeout**, host không còn phản hồi.
- Kiểm tra lần 2: connector trả `Session terminated`.

Vì vậy máy đã tắt hoặc đã hoàn tất quá trình shutdown đến mức toàn bộ MCP runtime trên máy không còn hoạt động.

Viết lại phương pháp luận, các đợt thử nghiệm và kết luận chi tiết để đưa ra phương án xử lý công tác phối hợp này chi tiết ra file md và commit lên github tool wed. Chi tiết đầy đủ các vấn đề trao đổi, thử nghiệm chi tiết để ra vấn đề này

Đã xử lý trong 7m 57s

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Đã hoàn thành và **commit trực tiếp lên GitHub** `Tools-Codex-web/main`.

Tài liệu mới:

`docs/research/ORCHESTRATION_METHODOLOGY_EXPERIMENTS_AND_PRODUCTION_DECISION_2026-08-12.md`

Commit:

`17ab609a9142260b4c4d4547a30497b3c380faaf`
`docs: consolidate Codex orchestration methodology and decision`

Tài liệu dài **2.223 dòng**, được viết lại thành một engineering decision record hoàn chỉnh thay vì chỉ ghép hai báo cáo cũ. Phần mở đầu xác định rõ mục tiêu là tái dựng toàn bộ chuỗi lập luận từ vấn đề ban đầu, các thí nghiệm thất bại, instrumentation bug, capacity, localization, cost traps cho tới production architecture cuối cùng.

Nội dung chính đã bao gồm:

- Phương pháp luận và định nghĩa metric: strong oracle, integration width, first-pass acceptance, TTA, availability-adjusted acceptance.
- Toàn bộ **Phase 1: 10 tasks × 3 routes = 30 observations**, số liệu Sol/Luna/Terra, latency, capacity, deadline, cost-per-accepted-result và cascade economics.
- Toàn bộ **Phase 2**: Mini low, Luna high, Terra high, Sol xhigh/max/ultra; autonomous vs patch-only; common-task comparison; effort calibration.
- Giải thích vì sao kết luận thay đổi từ “Luna/Terra autonomous không tốt” thành “**topology + packet quality quan trọng hơn model tier trên bounded tasks**”.
- False-green preflight do import nhầm source và lý do phải invalidate.
- T07 packet-localization failure và yêu cầu symbol-aware packet builder.
- ~30s `observe()` clamp làm sai nghĩa deadline Phase 2.
- Capacity failures và quy tắc không unchanged retry.
- Terra accepted-at-deadline insight: interrupt → freeze candidate → external grade.
- Nghiên cứu khoa học/official/field evidence đã dùng: OpenAI, Google multi-agent research, MTRouter, ContextualRouter, LLMRouterBench, ToolScope, RETRACE, SWE-bench correctness,

diversity/conformal routing.

- Sơ sánh ba chiến lược: `Sol xhigh-only`, adaptive bounded router, full autonomous ladder.
- Production router hiện tại và exact thresholds.
- Vì sao Mini phải có **strong-oracle gate** — tương ứng fix `b802b597`. Commit đó thực sự thêm `strong_oracle` vào Mini auto-selection.
- Candidate patch contract và Windows CRLF/LF incident — tương ứng fix `0fffe4be`, vẫn giữ unique exact-match guard.
- Tách execution plane và review plane.
- Vai trò chi tiết của ChatWeb Sol, MCP, Mini, Luna, Terra, Sol xhigh/max/ultra và Codex Desktop.
- Fixed **8-tool MCP surface** và hidden bridge.
- Client schema-cache incident.
- Live end-to-end Mini auto-route smoke.
- Skill v4 deployment, SHA integrity gate, staging/backup/rollback.
- Security/mutation boundaries.
- Parallelism/nested-swarm policy.
- Cache/session economics.
- One-fallback rule.
- Route ledger/telemetry schema.
- Promotion gates và kế hoạch benchmark tiếp theo.
- Final pseudocode + decision matrix + operational checklist.

Kiến trúc cuối được ghi rõ trong tài liệu:

```
CONTROL
ChatWeb Sol

DETERMINISTIC
Coding Tools MCP

CHEAP PATCH
Mini low / no-tools

HARDER LEAF
Luna high / no-tools

DEEP CODE
Codex Sol xhigh

CROSS-FILE EXPERIMENT
Terra high

CONSEQUENTIAL
Sol max

PARALLEL EXTREME
Sol ultra
```

```
FINAL ACCEPTANCE
ChatWeb Sol + deterministic evidence
```

Source hiện tại trên GitHub cũng khớp policy này: route map gồm Mini low, Luna high, Terra high, Sol xhigh/max/ultra với các deadline và high-cost gates tương ứng. Auto classifier chỉ chọn Mini khi có `strong_oracle`, chọn Luna cho bounded context, và đưa broad/deep context lên Sol xhigh.

Hai báo cáo gốc vẫn được giữ làm raw research provenance; thư mục `docs/research/` hiện có Phase 1, Phase 2 và bản final decision record mới.

Máy Windows đang tắt nên lần này tôi **không bật máy lại**; commit được thực hiện trực tiếp qua GitHub connector và đã nằm trên `main`.

Hôm nay 7:26

Đây là thực trang tôi vừa làm. Lên nghiên cứu phân tích kỹ, cùng các nghiên cứu bài báo khoa học, trao đổi trên diễn đang quốc tế. Phân tích nghiên cứu kỹ ra file md và up lên github của dự án tool codex web
Đã hoàn tất commit và push toàn bộ trạng thái hiện tại lên GitHub.

Commit là 89bc4ba, message feat(finance): deliver PFM cost control and fast entry, trên branch V_Finance. Sau push, local HEAD và origin/V_Finance cùng ở 89bc4bad2bd5dc1ba9fced763a8c941dc977cb21, ahead 0 / behind 0 / working tree clean. Commit chứa 34 file, 4.262 dòng thêm và 883 dòng xóa.

Tôi giữ nguyên trạng thái governance của US-112 là in_progress; không tự đổi thành implemented chỉ để commit trông “hoàn tất”. Về proof kỹ thuật thì hiện tất cả các constituent còn thiếu của BFF/security tail cũng đã xanh; hai wrapper lớn trước đó timeout do giới hạn thời gian của tool chứ không phải assertion failure.

Audit: lỗi thực sự xảy ra ở đâu

Kết luận quan trọng nhất là: thiết kế phối hợp của chúng ta về cơ bản đúng, nhưng runtime execution đã không thực thi đúng thiết kế đó. Sai lệch lớn nhất không nằm ở code PFM mà nằm ở orchestration activation, recovery policy và verification economics.

stpathcóthểchỉlà: mechanical → tool; critical

1. Lỗi đầu tiên xảy ra trước cả bước viết code: coordination plane không thực sự active. Tôi xác nhận E:\Github\HR_COORD và worktree E:\Github\HR-PFM-BE tồn tại, có supervisor/inbox/status assets, nhưng qua nhiều lần kiểm tra watch-pfm.ps1 đều ở trạng thái NOT_RUNNING hoặc SUPERVISOR_COUNT=0. Nghĩa là chúng ta đã xây "hạ tầng phối hợp Codex Desktop", nhưng khi thực hiện feature thì controller không coi liveness của coordination plane là precondition bắt buộc. Đây là lý do trực tiếp khiến bạn quan sát thấy Codex Desktop gần như không tham gia. Từ nay, khi một workflow được tuyên bố là Desktop-coordinated, bridge/supervisor liveness phải là gate trước mutation, không phải một thông tin tham khảo.
2. Lỗi thứ hai là recovery sau khi Codex deep-code thất bại. US-112 là financial write + authorization + atomic batch + OpenAPI + frontend, tức integration width cao. Router đúng phải để Sol khóa financial semantics/public contract, rồi Codex Sol xhigh nhận implementation packet. Các lần deep-code ban đầu gặp PROMPT_TOO_LARGE hoặc deadline. Ở đây đáng lẽ phải re-split packet và route lại, nhưng execution chuyển dần sang hàng loạt deterministic MCP patches do controller điều khiển. Quy tắc orchestration cũng nói rõ failure không được retry nguyên trạng, mà phải bổ sung evidence, thu hẹp/re-split hoặc đổi strategy. Tôi đã đổi strategy, nhưng đổi quá xa: từ "Codex deep implementation" thành "controller tự điều khiển feature implementation".
3. Packet bị chia quá nhỏ theo file/hunk thay vì theo coherent behavior. Task history cho thấy chuỗi kiểu DTO slice → capability slice → batch type → batch map → service runtime → fixture → test → frontend type → builder → panel → test → OpenAPI → PostgreSQL proof slice 1/2a/2b/3/4. Việc chia nhỏ cứu được execution, nhưng orchestration overhead quá lớn. Packet đúng không nên là "sửa file X", mà nên là "hoàn thành một behavior có oracle mạnh". Với US-112, chỉ khoảng ba packet lớn là hợp lý: backend financial boundary, frontend+contract boundary và integration proof. Lean packet chuẩn cũng chỉ cần OBJECTIVE / SCOPE / CONTEXT / PROOF / STOP, không cần biến mỗi hunk thành một task.
4. Patch/application layer quá nhạy với một checkout đang thay đổi liên tục. Task telemetry có nhiều

/shared/unsettled→Soll; narrow+strong orac le

PATCH_CONTEXT_AMBIGUOUS, PATCH_CONTEXT_NOT_FOUND và PATCH_FAILED. Ở US-111 thậm chí một interrupted patch để lại duplicate declarations rồi mới được phát hiện và sửa. Đây là dấu hiệu rằng exact-context patching trên shared dirty checkout không phù hợp cho những packet dài. Cách tốt hơn là pin BASE_HEAD + WORKTREE_FINGERPRINT, Codex trả candidate diff, MCP kiểm tra allowed paths rồi atomic apply; nếu có hai writer thì dùng worktree tách biệt. Đây cũng là quy tắc được orchestration protocol yêu cầu.

5. Chúng ta để authoritative checkout bẩn quá lâu. Đến cuối có 34 file trải qua US-109 → US-112 cùng tồn tại trước một checkpoint Git thực sự. Không làm mất correctness, nhưng làm tăng patch ambiguity, context drift, khó xác định proof thuộc story nào và khiến reviewer phải đọc delta lớn hơn. Với workflow dài, nên checkpoint sau một accepted story hoặc một integration boundary ổn định. Concurrent writer thì không nên cùng viết checkout này.

6. Verification sequencing chưa phù hợp với giới hạn runtime. verify:pfm-management chạy quá nhiều tầng nối tiếp và timeout khi đã đến build:finance; verify:bff-gate cũng tương tự. Sau đó chúng ta phải chạy phần chưa chứng minh riêng. Đây là lãng phí orchestration chứ không phải lỗi test. Từ nay gate dài phải được chia thành các stage có proof fingerprint và thời gian đủ ngắn; broad suite chỉ chạy một lần sau integration. Một stage đã PASS trên đúng fingerprint không được chạy lại chỉ vì wrapper ở stage sau timeout. Evaluation guide cũng xác định “repeated broad suites” là anti-pattern và yêu cầu một owner cùng proof-validity rule.

7. Thiếu telemetry routing làm chúng ta chưa thể chứng minh định lượng mức lãng phí. Chúng ta có task history rất giàu, nhưng chưa lưu có hệ thống TIME_TO_FIRST_DISPATCH, route requested/resolved, model effort, token cached/uncached, correction count, MCP calls, broad-gate reruns, capacity failures và time-to-accepted-patch. Vì vậy tôi có thể chỉ ra topology nào bị drift, nhưng không thể trung thực nói rằng lượt vừa rồi đắt hơn 27% hay chậm hơn 40%. Việc không đo được chính nó là một lỗi thiết kế vận hành. Bộ telemetry cần lưu đã được định nghĩa khá đầy đủ trong evaluation spec.

Mô hình tối ưu hơn từ đây

+ reversible → Luna; còn lại → Terra. Chỉ mở class si

Mô hình đúng không phải là “luôn dùng Codex Desktop”. Nếu làm vậy chúng ta lại tạo một loại lãng phí khác.

Topology nên là:

User → ChatWeb Sol control plane → one-time evidence/preflight → router → executor → deterministic proof → Sol acceptance

Trong đó Sol chỉ giữ những thứ đáng giữ: requirement decomposition, architecture, financial meaning, authorization boundary, unstable public contract, conflict resolution và final go/no-go. Coding Tools MCP làm search/compiler/test/generator/database proof/Git và các transform deterministic. Mini dùng cho leaf nhỏ có oracle mạnh. Luna dùng cho bounded leaf khó hơn. Codex Sol xhigh dùng cho broad/deep implementation sau khi Sol đã khóa interface. Terra/Max/Ultra không được bật chỉ vì task “to”; chúng phải có lý do và bằng chứng về marginal value. Đây chính là routing order được policy định nghĩa.

Điểm này còn được dữ liệu benchmark nội bộ củng cố. Trên các localized leaves, Mini low đã đạt 3/3 khoảng 9,11 giây với ~0,1585 estimated credits; Luna high 3/3 khoảng 7,89 giây với ~0,2597; Sol xhigh cũng 3/3 nhưng khoảng 11,23 giây và ~1,3173 credits. Trong khi topology Sol xhigh autonomous shell/tool loop không tạo accepted write ở cả 3 mẫu trước mốc dừng khoảng 31 giây. Nghĩa là model mạnh hơn không đồng nghĩa route kinh tế hơn; topology mới là thứ phải tối ưu trước.

Một thay đổi quan trọng khác là Desktop coordination phải chuyển từ “persistent infrastructure mặc định” thành một execution mode có điều kiện. Nếu feature có hai leaf thật sự độc lập, ví dụ frontend và backend đã có contract đóng băng, chạy hai worktree song song có thể đáng tiền. Nhưng financial semantics đang thay đổi theo từng bước thì BE/FE persistent workers chỉ tạo stale work và merge cost. Khi high seriality/high integration width chiếm ưu thế, một Codex deep-code lane duy nhất cộng MCP oracle thường tốt hơn hai worker. Orchestration protocol cũng yêu cầu không fan-out khi writes overlap, assumptions thay đổi tuần tự hoặc review/integration mới là bottleneck.

Tôi cũng sẽ coi runtime capability map là cache, không probe đi probe lại. Đầu phiên chỉ cần xác nhận một lần: HEAD/worktree, Harness state, Codex bridge/supervisor, resolved model/effort, PostgreSQL/Docker/Prisma runtime và proof commands. Sau đó duy trì một state capsule rất ngắn gồm goal, acceptance, base, invariants, decisions, evidence, active packets, proof, uncertainty và next action. Đây là mô hình context discipline mà skill đã định nghĩa, thay vì liên tục Get-Content/Select-String lại cùng vùng repo.

Điều tôi sẽ thay đổi về mặt vận hành

Từ packet tiếp theo, nguyên tắc sẽ là: preflight một lần, dispatch sớm, packet theo behavior, một write owner, proof ngay sau packet, broad gate một lần, Harness sau acceptance, commit checkpoint theo story.

Quan trọng hơn, sau mỗi task chúng ta sẽ có một route ledger nhỏ. Khi đủ số mẫu, mình không cần tranh luận "Codex có nên tham gia nhiều hơn không" theo cảm giác nữa; chúng ta sẽ biết task class nào Mini rẻ nhất, task nào Luna có time-to-accept tốt hơn, packet nào Codex xhigh giảm rework, và khi nào Desktop parallelism thực sự có lợi. Evaluation guide cũng yêu cầu tuning theo thứ tự: loại duplicated scans/reruns trước, batch deterministic work, sửa preflight/oracle, cải thiện packet boundaries, rồi mới thay topology/model tier.

Điểm cần sửa sâu nhất vì vậy không phải "gọi Codex nhiều hơn", mà là biến router từ guideline thành một control loop có enforcement. Lượt vừa rồi guideline nói đúng, nhưng khi supervisor chết và Codex packet fail, hệ thống không có guard ngăn controller tiếp tục drift sang implementation. Đó là chỗ tôi đánh giá cần ưu tiên thiết kế lại nhất.

Ở trạng thái hiện tại, GitHub đã có một checkpoint sạch. Đây là thời điểm tốt để áp dụng topology mới từ công việc kế tiếp thay vì tiếp tục mang debt orchestration của US-109→112 sang các story sau.

Đã xử lý trong 12m 58s

Công cụ được gọi

fication đầy đủ khi route không rõ, đang cân nhắc

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Đã nói chuyện với App

Công cụ được gọi

Đã nói chuyện với App

Công cụ được gọi

Đã nói chuyện với App

Đã hoàn tất nghiên cứu, viết tài liệu chi tiết và **commit trực tiếp lên** `Tools-Codex-web/main`.

Tài liệu mới:

`docs/research/PFM_ORCHESTRATION_RUNTIME_POSTMORTEM_AND_ENFORCED_CONTROL_LOOP_RESEARCH_2026-08-13.md`

Commit:

`a153a8984cc2d2e1bfe0e24131e287f491caee19`

`docs: analyze PFM orchestration runtime failure and control loop`

Commit này chỉ thêm **một file nghiên cứu**, dài **2.657 dòng**, không sửa router/source production. Nội dung file ở commit đã được đọc lại trực tiếp từ GitHub để xác nhận.

Kết luận quan trọng nhất

Sau khi đối chiếu thực trạng bạn mô tả, source PFM, Git history, benchmark trước đây và nghiên cứu bên ngoài, tôi chốt nguyên nhân sâu nhất là:

> **Policy orchestration của chúng ta về cơ bản đúng, nhưng policy vẫn là "guideline"; runtime chưa biến nó thành state machine có enforcement.**

Vì thế khi supervisor Desktop không ready và deep-code lane bị `PROMPT_TOO_LARGE`/deadline, không có hard guard buộc hệ thống phải:

```
EXECUTING
    ↓ failure
REPLAN
    ↓ explicit new route
TRANSFER WRITE OWNERSHIP
```

```
↓
```

Controller vì thế có thể drift dần thành implementation executor qua hàng loạt MCP patch mặc dù topology ban đầu chọn Codex deep-code.

Tài liệu đề xuất thiết kế mới **EOCL — Enforced Orchestration Control Loop**:

```
PREFLIGHT
   ↓
READY
   ↓
ROUTED
   ↓
EXECUTING
   ↓
PROVING
   ↓
ACCEPTING
   ↓
CHECKPOINTED

Exceptional:
BLOCKED
REPLAN
ABORTED
```

Không có state transition hợp lệ thì **không được đổi executor và không được mutation**.

Một số phát hiện đã được GitHub kiểm chứng lại

Có một correction provenance đáng lưu ý. Commit finance thực tế trên `uyenthudao2410-code/HR`, `V_Finance` là:

`89bc4baddaa32614b4689c68e09595acc0f78923`

không phải full SHA `89bc4bad2bd5...` trong phần mô tả ban đầu. Message đúng là `feat(finance): deliver PFM cost control and fast entry`. GitHub hiện xác nhận branch head đó.

GitHub single-commit view cũng cho **43 files, +2.803/-117**, khác con số session report `34 files, +4.262/-883`. Tôi không ép hai con số thành một: report ghi rõ rất có thể chúng đang đo hai phạm vi khác nhau và lấy GitHub single-commit stats làm provenance authoritative.

Phân tích diff cho thấy checkpoint này thực sự trải rộng US-109 → US-112, OpenAPI, backend DTO/service, authorization capability, frontend, generated client, Excel parity và PostgreSQL proof. Vì vậy nhận định “authoritative checkout đã tích lũy quá nhiều integration state trước checkpoint” có bằng chứng Git, không chỉ dựa vào cảm giác vận hành.

Đồng thời, việc bạn **không tự đổi US-112 thành** `implemented` là đúng. Source story giữ các invariant tài chính quan trọng như exact decimal, capability default-off/project-scoped, deterministic identity,

locked-period fail-before-persistence và atomic persistence/outbox/audit. Technical checkpoint và governance completion được tài liệu tách thành hai state độc lập.

Những thay đổi kiến trúc mà report đề xuất

Tài liệu đi tới tám thay đổi ưu tiên:

1. **Execution state machine có enforcement**, thay vì router advisory.
2. **Desktop readiness gate theo mode**: chỉ khi chọn Desktop-coordinated mode mới bắt buộc supervisor/startup/readiness/liveness; không biến Desktop thành global dependency.
3. **Write ownership lease**: mỗi mutable surface chỉ có một owner, gắn với `BASE_HEAD`, worktree fingerprint và allowed paths.
4. Deep-code failure bắt buộc vào `REPLAN`; ChatWeb/MCP không được âm thầm tiếp quản implementation.
5. Packet chuyển từ file/hunk thành **coherent behavior + oracle**. Riêng US-112 được redesign thành ba packet: backend financial boundary, frontend contract boundary, integration acceptance.
6. Concurrent writers phải dùng **isolated Git worktrees**; BE/FE chỉ parallel sau khi financial/public contract đã freeze.
7. Hai wrapper lớn như `verify:pfm-management` và `verify:bff-gate` chuyển thành **proof DAG**, mỗi stage có `PROOF_FINGERPRINT`; một stage đã PASS trên đúng fingerprint không chạy lại chỉ vì stage sau timeout.
8. Thêm **OpenTelemetry + route ledger** để đo time-to-first-dispatch/write/green/accepted, route/model/effort, tool calls, patch failures, proof-cache hits, broad reruns, capacity/deadline, tokens/cache và accepted-result credits.

Hướng này cũng phù hợp với nghiên cứu bên ngoài. Google Research cho thấy trong 180 agent configurations, multi-agent có thể cải thiện mạnh task parallelizable nhưng làm xấu task sequential; centralized coordination cũng kiểm soát error propagation tốt hơn. Nghiên Cứu G… MTRouter ACL 2026 cho thấy routing hiệu quả có xu hướng giảm model switching thay vì đi qua cascade dài. ACL Anthol…

Đặc biệt, **Ledger** rất sát với sự cố PFM này: paper lập luận rằng raw interaction history không đủ để xác định state nào còn hợp lệ trong long-horizon coding; họ đưa execution ledger tách biệt để cung cấp compact current state và ngăn re-execution khi kết quả cũ vẫn valid. Paper báo cáo cả tăng Pass@1 lẫn giảm cost, bao gồm thử nghiệm gắn với Codex. ar… LongHorizon-Harness cũng tách manager/executor/auditor và chỉ cập nhật task state từ environment facts đã được kiểm chứng. ar…

Về multi-writer, nghiên cứu CAID 2026 chỉ ra chính các vấn đề concurrent edits, dependency synchronization và integration của partial work, rồi dùng centralized delegation + isolated workspaces + branch/worktree-based integration. ar… Đây là cơ sở khá mạnh để không tiếp tục dùng một shared dirty checkout cho các Codex writers song song.

Phương án cuối sau incident này

Model topology **không cần thay đổi**:

```
CONTROL                 ChatWeb Sol
DETERMINISTIC           Coding Tools MCP
CHEAP PATCH             Mini low / no-tools
HARDER LEAF             Luna high / no-tools
DEEP CODE               Codex Sol xhigh
CROSS-FILE EXPERIMENT   Terra high
CONSEQUENTIAL           Sol max
PARALLEL EXTREME        Sol ultra
FINAL ACCEPTANCE        ChatWeb Sol + deterministic evidence
```

Thứ phải thay đổi là lớp bên dưới:

từ

```
POLICY-GUIDED ORCHESTRATION
            ↓
STATE-ENFORCED ORCHESTRATION
```

Thứ tự triển khai tôi đề xuất trong report là: **state/write ownership → readiness → typed recovery → proof fingerprint/cache → compact state capsule → OTel/route ledger → behavior packets/checkpoints → rồi mới tiếp tục tune model và parallelism**.

Đây là điểm khác biệt quan trọng: sau incident này, chưa nên "gọi Codex nhiều hơn". Nên làm cho hệ thống **không thể drift khỏi route đã chọn mà không có một explicit, observable, measurable state transition**. Sau khi có telemetry thật, lúc đó mới có thể trả lời định lượng Desktop coordination giúp bao nhiêu %, Sol xhigh giảm rework bao nhiêu, hay Mini/Luna tiết kiệm bao nhiêu accepted-result cost.

NOW, EXECUTOR, BASE, READ, INPUT

Hôm nay 10:48

EXECUTOR_ROUTING_…
Tệp

MODEL_ORCHESTRATI…
Tệp

ORCHESTRATION_MET…
Tệp

PFM_ORCHESTRATION…
Tệp

Phân tích kỹ lại và tìm hướng xử lý tối ưu nhất. Và khi làm phải làm rõ việc triển khai như nào trong các đợt phối hợp phải có log kiểm tra làm tài liệu tối ưu hơn cho sau này

Worked for 8m 8s

Công cụ được gọi

Đã nói chuyện với App

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Tôi đã phân tích lại toàn bộ 4 tài liệu, đối chiếu thêm source hiện tại của `Tools-Codex-web`, và điều chỉnh kiến trúc một bước quan trọng. Kết luận trước vẫn đúng: **không cần đổi topology model**; ChatWeb Sol vẫn là control/final acceptance, MCP là deterministic execution/proof, Mini/Luna cho bounded candidate, Sol xhigh cho deep lane. Phase 2 cũng đã cho thấy topology/packet quality quan trọng hơn việc đơn thuần tăng tier model. MODEL_ORCHESTRATION_RESEARCH_PH...

Điểm tôi thay đổi là **thứ tự triển khai và kiến trúc log**. Postmortem trước đã xác định đúng các lỗi: coordination readiness chưa enforce, deep failure chưa bắt buộc REPLAN, write authority có thể drift, proof chưa resumable và telemetry chưa đủ. PFM_ORCHESTRATION_RUNTIME_POSTM... Nhưng chỉ bổ sung một `route ledger` tổng kết cuối task vẫn chưa đủ. Một route row chứa model, effort, TTA, proof-cache hits... rất hữu ích cho thống kê, nhưng không thể tái dựng chính xác **ai được quyền write lúc nào, route đổi ở state nào, supervisor chết lúc nào, proof nào còn valid, packet nào bị invalid do contract drift**. Các field route ledger trước đây đúng nhưng nên trở thành **materialized view**, không phải source-of-truth. PFM_ORCHESTRATION_RUNTIME_POSTM...

Hướng xử lý tối ưu nhất

Tôi chốt kiến trúc mới thành:

```
EOCL STATE MACHINE
        |
        +----> ORCHESTRATION AUDIT LEDGER   <- canonical truth
        |             |
        |             +--> current TaskState
        |             +--> route ledger
        |             +--> proof ledger/cache
        |             +--> OpenTelemetry
        |             +--> generated Markdown reports
        |
        +----> MCP / Codex execution
```

Canonical source nên là SQLite append-only ledger trong `.coding-tools/orchestration/ledger.sqlite3`. `task-state.json` chỉ còn là current-state projection; `performance.json` chỉ là performance view; OpenTelemetry là live observability; Markdown là tài liệu người đọc; product telemetry vẫn giữ vai trò anonymous analytics. Đây là distinction rất quan trọng vì source hiện tại cho thấy `TaskStateStore` chỉ giữ rolling 100 events, còn `PerformanceTraceStore` cũng giữ tối đa 100 recent calls; product telemetry thì cố ý có bounded queue, có thể disable/drop và không lưu source/tool contents. Vì vậy cả ba đều không thích hợp làm forensic authority.

Một phát hiện từ nghiên cứu mới làm kiến trúc này chặt hơn nữa: **trace ID không đồng nghĩa delegation/authority ID**. Nghiên cứu về delegated execution cho thấy execution traces thông thường

có thể không đủ để xác định ai thực sự được ủy quyền; vì vậy phải persist `delegation_id`, `parent_delegation_id`, `write_lease_id` và authority scope riêng. ar... Nghiên cứu TraceElephant tại ACL 2026 cũng cho thấy full execution traces cải thiện đáng kể khả năng attribution failure so với partial traces. ACL Anthol... Còn Ledger cho long-horizon coding cho thấy explicit execution ledger vừa giúp govern/reuse state vừa giảm unnecessary execution trong evaluation của paper. ar...

Do đó mỗi lần phối hợp từ nay phải có hierarchy:

```
RUN
 ├─ WAVE-001
 │   ├─ DELEGATION-A
 │   │   ├─ PACKET-A1
 │   │   ├─ write lease
 │   │   ├─ model/tool events
 │   │   └─ proof events
 │   └─ DELEGATION-B
 ├─ WAVE-002
 └─ FINAL ACCEPTANCE
```

Một **wave** chỉ hợp lệ khi cùng base/worktree fingerprint, cùng settled invariants, cùng contract fingerprint và cùng execution assumptions. Base hoặc contract đổi, executor failure gây REPLAN, hoặc chuyển sang integration thì wave cũ đóng và wave mới bắt đầu. Điều này giải quyết trực tiếp vấn đề trước đây khi topology âm thầm thay đổi nhưng task vẫn được xem như một execution liên tục.

Thứ tự triển khai tôi đã sửa lại

1. **W0 — Audit substrate + shadow instrumentation.** Thêm `orchestration_ledger.py`, event schema và report generator nhưng chưa đổi routing behavior. Chạy fixture/low-risk tasks để chắc chắn event không mất, restart reconstruct được, secret không leak và Markdown report được generate hoàn toàn từ DB.
2. **W1 — Hard EOCL enforcement.** State transition, readiness gate, write lease, base/worktree guard, typed failure và mandatory REPLAN. Từ đây mới cho phép tiếp tục Desktop-coordinated mode cho finance/auth/privacy high-risk.
3. **W2 — Proof DAG.** Mỗi proof node có stage/version/fingerprint; wrapper timeout chỉ resume node còn thiếu. Broad gate tối đa một lần trên cùng final fingerprint.
4. **W3 — Behavior packet + coordination wave/worktree.** Packet phải là behavior có oracle, không phải file/hunk. Mỗi writer có worktree/lease riêng; wave bắt buộc có manifest và report.
5. **W4 — Adaptive optimization.** Dùng lịch sử các wave accepted tương tự để chọn Mini/Luna/Sol xhigh/Desktop; chưa cần ML router. Đây phù hợp với methodology hiện có là classification + tối đa một evidence-driven fallback. ORCHESTRATION_METHODOLOGY_EXPER...
6. **W5 — Continuous evaluation.** Định kỳ generate report so sánh route/class, median/p90 TTA, credits/accepted result, proof-cache savings, REPLAN reasons, readiness failures, Sol repair turns và single-lane vs parallel performance.

Điểm khác quan trọng so với tài liệu trước là **W0 được đưa lên trước W1**, nhưng chỉ shadow/low-risk. Lý do: nếu implement state machine trước rồi logging sau, chính những đợt thay đổi state machine đầu tiên lại không có baseline tốt để kiểm chứng. Tuy nhiên không được lợi dụng W0 để tiếp tục chạy high-risk Desktop coordination; **high-risk coordinated story phải chờ W1**.

Log bắt buộc cho từng đợt phối hợp

Trước wave phải persist manifest gồm `run_id`, `wave_id`, base/worktree/contract fingerprints, execution mode, route requested/resolved, packets, write owners, proof plan và stop conditions. Trong wave chỉ log các event có ý nghĩa quyết định: state transition, readiness/liveness, delegation, lease acquire/release, model dispatch, candidate/mutation, proof, typed failure, REPLAN và contract invalidation. Sau wave phải auto-generate `wave-NNN.md`; sau story/run auto-generate `run-summary.md`.

Một wave report tốt phải cho phép nhìn ngay:

```
PLANNED ROUTE        vs ACTUAL ROUTE
WRITE AUTHORITY      who / scope / when
STATE TIMELINE       READY -> ROUTED -> EXECUTING -> ...
FAILURE / REPLAN     exact typed cause
PACKET OUTCOME       accepted / blocked / invalidated
PROOF DAG            fingerprint / pass / cache-hit
TIME                 dispatch / first-write / first-green / accepted
COST                 model / tools / failed attempts
DEVIATIONS           planned topology vs actual topology
CHECKPOINT           Git/proof result
```

Từ đó mới tính chính xác được `TIME_TO_FIRST_DISPATCH`, `TIME_TO_FIRST_WRITE`, `TIME_TO_FIRST_GREEN`, `TIME_TO_ACCEPTED`, proof tail, acceptance tail và critical path cho parallel wave. Điều này tốt hơn cách hiện tại dùng gap giữa MCP calls để ước lượng model/network wait.

OpenTelemetry vẫn nên tích hợp để xem live spans, model/tool timings và GenAI attributes, nhưng **không được làm audit authority**, vì OTel tracing hỗ trợ sampling và span có thể không được record/export. OpenTelem… Canonical DB phải lossless cho control transitions; OTel chỉ là projection phục vụ quan sát.

Tài liệu đã được cập nhật lên GitHub

Tôi đã tạo decision record mới:

`docs/research/ORCHESTRATION_WAVE_AUDIT_LEDGER_AND_OPTIMAL_ROLLOUT_PLAN_2026-08-13.md`

Commit:

`ab89263f546f14c43d10f51916841b6a54b4ae35`

`docs: define orchestration wave audit ledger and rollout plan`

GitHub xác nhận commit và file mới. Tôi cũng đọc lại file ở chính commit đó; tài liệu ghi rõ **chưa thay đổi production source**, định nghĩa canonical SQLite audit ledger và rollout `W0 → W5`.

Hiện tại tôi **chưa sửa production router hay skill**. Bước code tiếp theo hợp lý nhất là **W0: Orchestration Audit Ledger + shadow instrumentation**. Sau đó chính việc triển khai W1, W2, W3… sẽ tự để lại wave logs và reports, tức là hệ thống orchestration bắt đầu **tự đo chính quá trình được cải tiến của nó**, thay vì phải đợi incident rồi reconstruct bằng trí nhớ và task history.

Nguồn

Việc sửa lại bạn sẽ thay đổi ở đâu nói rõ chi tiết

Worked for 51s

Sẽ sửa chủ yếu trong `Tools-Codex-web`, cụ thể ở `resources/coding-tools-mcp/coding_tools_mcp/`. Tôi **không định thay model topology và không tăng 8 tool public của MCP**; `server.py` hiện đã có đúng 8 tool smart-facing và route Mini/Luna/Terra/Sol nên phần đó giữ nguyên về contract.

Phương án tôi đề xuất chia thành các thay đổi sau.

1. Thêm lớp log/audit mới — làm trước tiên

Tạo mới:

`resources/coding-tools-mcp/coding_tools_mcp/orchestration_ledger.py`

Đây sẽ là phần quan trọng nhất. Nó quản lý:

```
.coding-tools/orchestration/ledger.sqlite3
```

Các entity chính:

```
RUN
WAVE
DELEGATION
PACKET
STATE_TRANSITION
WRITE_LEASE
EVENT
PROOF
```

Ví dụ một event:

```json
{
  "run_id": "...",
  "wave_id": "...",
  "delegation_id": "...",
  "event_type": "route_selected",
  "state_before": "READY",
  "state_after": "ROUTED",
  "route_requested": "sol_xhigh",
  "route_resolved": "sol_xhigh",
  "base_head": "...",
  "worktree_fp": "...",
  "timestamp": "..."
}
```

Quan trọng: đây là **append-only log**. Không rewrite lịch sử giống `task-state.json`.

`TaskStateStore` hiện chỉ giữ tối đa 100 event rồi cắt tail. Vì vậy nó phù hợp làm current snapshot nhưng không đủ làm audit history.

---

2. Tách EOCL state machine khỏi `server.py`

Tạo mới:

`resources/coding-tools-mcp/coding_tools_mcp/orchestration_control.py`

Không nên nhét toàn bộ state machine thêm vào `server.py`, vì file đó hiện đã chịu trách nhiệm RPC, routing, permissions, candidate execution, bridge integration và rất nhiều logic khác.

Module mới sẽ quản lý state:

```
PREFLIGHT
READY
ROUTED
EXECUTING
PROVING
ACCEPTING
CHECKPOINTED

REPLAN
BLOCKED
ABORTED
```

Và enforce transition:

```
PREFLIGHT -> READY
READY -> ROUTED
ROUTED -> EXECUTING
EXECUTING -> PROVING

EXECUTING --failure--> REPLAN

REPLAN
 -> explicit new route
 -> new ownership lease
 -> EXECUTING
```

Các invariant hard:

```
NO ROUTE CHANGE WITHOUT STATE TRANSITION

NO WRITE OWNER CHANGE
WITHOUT OWNERSHIP TRANSFER

NO MUTATION WITHOUT ACTIVE LEASE

NO PROOF REUSE
WITHOUT FINGERPRINT MATCH

NO PARALLEL WRITER
ON SAME WORKTREE
```

Đây chính là phần còn thiếu mà PFM postmortem đã chỉ ra: route policy đúng nhưng runtime chưa enforce; controller có thể tiếp tục implementation sau deep-lane failure mà không có explicit ownership transfer. PFM_ORCHESTRATION_RUNTIME_POSTM...

---

3. Sửa `server.py` — nhưng chỉ làm integration point

Sửa:

`resources/coding-tools-mcp/coding_tools_mcp/server.py`

Không biến `server.py` thành nơi chứa toàn bộ audit logic.

Tôi chỉ sửa các điểm giao nhau:

```
agent_workflow
    -> create/run/wave context
    -> call orchestration_control

route selection
```

```
    -> record ROUTE_SELECTED

Codex dispatch
    -> record DELEGATION_STARTED

MCP mutation
    -> check active WRITE_LEASE

verification
    -> handoff proof ledger

failure
    -> normalize failure
    -> transition REPLAN

acceptance
    -> finalize wave/run metrics
```

Route table hiện có:

```
mini      GPT-5.4-Mini low
luna      GPT-5.6-Luna high
terra     GPT-5.6-Terra high
sol_xhigh GPT-5.6-Sol xhigh
sol_max   GPT-5.6-Sol max
sol_ultra GPT-5.6-Sol ultra
```

nên **W0/W1 không thay hierarchy này**.

Tôi cũng không expose thêm:

```
orchestration_log
proof_cache
wave_control
...
```

thành tool thứ 9, 10, 11 cho ChatWeb.

Các chức năng đó nằm **nội bộ sau** `agent_workflow`.

---

4. Sửa `task_state.py`

Sửa:

`resources/coding-tools-mcp/coding_tools_mcp/task_state.py`

Hiện tại:

```
task-state.json
task-history.json
events[-100:]
```

sẽ tiếp tục tồn tại để không phá compatibility.

Nhưng nó chuyển thành **projection/current-state cache**, không phải historical authority.

Ví dụ current state sẽ có thêm:

```json
{
  "task_id": "...",
  "run_id": "...",
  "wave_id": "...",
  "orchestration_state": "EXECUTING",
  "execution_mode": "DEEP_SINGLE_LANE",
  "route": "sol_xhigh",
  "write_owner": "codex:lane-1",
  "active_packet": "...",
  "base_head": "...",
  "worktree_fp": "...",
  "latest_proof": "...",
  "failure_class": null
}
```

Nhưng lịch sử đầy đủ được query từ SQLite.

Điều này giữ backward compatibility đồng thời giải quyết giới hạn rolling-event hiện tại.

---

5. Thêm write-lease enforcement

Có thể đặt class trong `orchestration_control.py`, chưa cần tạo module riêng.

Một lease:

```
lease_id
run_id
wave_id
delegation_id

owner:
  MCP
  CODEX:<lane-id>

mode:
  MCP_ONLY
```

```
  BOUNDED_CANDIDATE
  DEEP_SINGLE_LANE
  COORDINATED_MULTI_WORKTREE

base_head
worktree_id
worktree_fp
allowed_paths
acquired_at
revoked_at
```

Mọi mutation phải hỏi:

Python Chạy

```
control.assert_write_allowed(
    owner=...,
    path=...,
    current_base=...,
    current_worktree_fp=...
)
```

Nếu Codex deep lane chết:

```
DEADLINE
     |
     v
revoke lease
     |
     v
REPLAN
```

ChatWeb/MCP **không được tiếp tục sửa feature** trước khi có:

```
explicit reclassification
+
new lease
```

Đây chính là guard quan trọng nhất để ngăn lặp lại PFM authority drift. Postmortem trước đã định nghĩa controller chỉ được feature-mutate trong `MCP_ONLY` hoặc sau explicit ownership transfer. PFM_ORCHESTRATION_RUNTIME_POSTM...

---

6. Sửa `codex_bridge.py`

Sửa:

`resources/coding-tools-mcp/coding_tools_mcp/codex_bridge.py`

không thay bridge thành orchestration owner.

Bridge chỉ phát lifecycle evidence:

```
CODEX_DISPATCH_STARTED
CODEX_THREAD_CREATED
CODEX_MODEL_RESOLVED
CODEX_FIRST_OUTPUT
CODEX_CANDIDATE_READY
CODEX_DEADLINE
CODEX_CAPACITY
CODEX_INTERRUPTED
CODEX_FINISHED
```

Mỗi event phải kèm:

```
run_id
wave_id
delegation_id
packet_id
requested model
resolved model
effort
profile
deadline
token/economic metadata nếu có
```

Đặc biệt cần phân biệt:

```
requested route
!=
resolved route
```

Và:

```
turn interrupted
!=
candidate invalid
```

Phase 1 đã có nhiều case candidate đúng dù turn bị interrupt; đó là lý do deadline phải freeze candidate rồi external-grade thay vì tự động coi là failed.

---

7. Thêm Proof Ledger

Tạo mới:

`resources/coding-tools-mcp/coding_tools_mcp/proof_ledger.py`

Hoặc nếu muốn ít module hơn, phần persistence nằm trong `orchestration_ledger.py`, còn `proof_ledger.py` là service facade.

Mỗi proof:

```
proof_id
stage_id
stage_version

input_fingerprint
base_head
worktree_fp
contract_fp
toolchain_fp

command/check
status
started_at
finished_at
duration
exit_code

cache_hit
upstream_proof_ids
```

Fingerprint đại loại:

```
hash(
    source inputs
  + config
  + toolchain
  + base
  + contract
  + upstream proof
)
```

Khi wrapper timeout:

```
stage A PASS
stage B PASS
stage C TIMEOUT
```

lần tiếp theo:

```
reuse A
reuse B
resume C
```

không phải:

```
rerun A
rerun B
rerun C
```

Đây là đúng phần verification economics đã được postmortem xác định: PASS phải persist theo fingerprint và broad suite chỉ chạy một lần trên final integrated fingerprint.

PFM_ORCHESTRATION_RUNTIME_POSTM...

---

8. Sửa `performance_trace.py`

Sửa:

`resources/coding-tools-mcp/coding_tools_mcp/performance_trace.py`

Hiện nó có:

```
MAX_RECENT = 100
wait_before_ms = gap between MCP calls
```

Cách này vẫn dùng được để hiển thị quick performance info, nhưng không nên dùng để kết luận:

```
model reasoning time
controller wait
Codex execution
parallel critical path
```

vì khoảng trống giữa hai tool call không nói được trong khoảng đó thành phần nào đang làm việc.

Sau sửa:

```
PerformanceTraceStore
    = derived/materialized view
      từ orchestration events
```

Có thể giữ JSON file compatibility, nhưng nguồn metric chuẩn sẽ từ ledger.

---

9. Không biến `telemetry.py` thành audit logger

`resources/coding-tools-mcp/coding_tools_mcp/telemetry.py`

Gần như không sửa ở W0.

Đây là anonymous product telemetry. Source hiện cố ý:

```
không log file paths
không log commands
có thể telemetry=off
DNT disable
CI disable
bounded queue
overflow drop
send errors swallowed
```

Đây là thiết kế đúng cho privacy analytics.

Không nên cố dùng nó làm orchestration audit.

Nếu sau này cần OTel, tôi ưu tiên tạo:

`resources/coding-tools-mcp/coding_tools_mcp/orchestration_otel.py`

thay vì làm `telemetry.py` lẫn hai responsibility.

---

10. Thêm report generator

Tạo mới:

`resources/coding-tools-mcp/coding_tools_mcp/orchestration_report.py`

Nó đọc ledger và auto-generate:

```
.coding-tools/orchestration/reports/
    run-<id>/
        run-summary.md
        wave-001.md
        wave-002.md
        wave-003.md
```

Mỗi `wave-NNN.md` có:

```
# Wave identity
run / wave / story
base / branch / worktree

# Planned execution
mode
route
packets
write ownership
proof plan
```

```
# Actual execution
resolved route
resolved model/effort
state timeline

# Delegations
who received what
authority boundaries

# Mutation
paths
candidate/diff fingerprint

# Failures / replans
failure class
evidence
old route -> new route
why

# Proof
focused
boundary
build/runtime
cache hit/miss

# Economics
dispatch latency
first write
first green
accepted time
model/tool duration
tokens/credits

# Deviations
planned topology vs actual topology

# Outcome
accepted/rejected/blocked
checkpoint
remaining uncertainty
```

Markdown không được hand-maintain.

Nó phải được regenerate từ canonical DB.

---

11. Tests sẽ sửa/thêm ở đâu

Repo hiện chỉ có hai Python test files trong package test directory:

```
resources/coding-tools-mcp/tests/
    test_codex_bridge.py
    test_task_state_and_build.py
```

GitHub hiện xác nhận đúng hai file này.

Tôi sẽ **không nhồi tất cả test mới vào hai file này**.

Nên tạo:

```
resources/coding-tools-mcp/tests/
    test_orchestration_ledger.py
    test_orchestration_control.py
    test_proof_ledger.py
    test_orchestration_report.py
```

và mở rộng:

```
test_task_state_and_build.py
test_codex_bridge.py
```

cho integration compatibility.

Test W0

```
append event survives restart
event ordering stable
wave reconstruction exact
delegation lineage preserved
redaction works
SQLite corruption handled safely
report generated from ledger
TaskState projection matches ledger
```

Test W1

4.

```
write without lease -> rejected

wrong owner -> rejected

wrong worktree fingerprint -> rejected

deep executor failure:
EXECUTING -> REPLAN

controller mutation during REPLAN -> rejected

route change without transition -> rejected
```

```
lease transfer explicit -> accepted
```

Test W2

```
same fingerprint -> proof cache hit

source changes -> cache invalid

config changes -> invalid

toolchain changes -> invalid

contract changes -> invalid

wrapper timeout:
resume missing stage only
```

Test multi-worktree

```
worker A cannot write B paths
worker B cannot write A paths

same worktree concurrent lease rejected

contract change invalidates dependent wave
```

---

12. Các đợt triển khai thực tế

Tôi sẽ **không làm một commit khổng lồ**.

Tách như sau:

```
W0A
SQLite schema + append-only store

W0B
shadow instrumentation
server / task_state / bridge hooks

W0C
wave/run Markdown generator
+ baseline metrics
```

Sau khi W0 xanh:

```
W1A
state transition authority

W1B
write lease + base/worktree guard

W1C
typed recovery + mandatory REPLAN
```

Sau đó:

```
W2A
proof registry

W2B
proof fingerprints/cache

W2C
resume wrapper + broad-once rule
```

Tiếp:

```
W3
behavior packet
Desktop readiness
multi-worktree coordination
```

Rồi mới:

```
W4
route-history optimization
Mini/Luna/Sol class tuning

W5
continuous evaluation
```

---

13. Phần chưa sửa trong những đợt đầu

Tôi sẽ **không** đổi ngay:

```
Mini low
Luna high
Terra high
Sol xhigh
Sol max
Sol ultra
```

Không đổi production threshold chỉ vì postmortem.

Không thêm public MCP tools.

Không bật Terra auto.

Không bật Ultra auto.

Không biến Desktop thành dependency bắt buộc cho mọi task.

Không xóa `task-state.json`.

Không xóa existing product telemetry.

Không thay final acceptance khỏi ChatWeb Sol.

Phase 2 vẫn cho thấy adaptive bounded routing là architecture phù hợp nhất, nhưng chính nó yêu cầu packet builder và telemetry tốt; vì vậy hiện tại nên sửa **execution accountability trước model selection**. MODEL_ORCHESTRATION_RESEARCH_PH...

---

14. Cuối cùng mới cập nhật Skill

Sau khi **runtime W0/W1 đã có test và log thật**, mới cập nhật `sol-lean-codex` để Skill nói đúng cái runtime thực sự enforce:

```
wave lifecycle
write lease
mandatory REPLAN
proof fingerprint
audit requirements
wave report
promotion metrics
```

Không nên sửa Skill trước rồi để nó tiếp tục là guideline chưa có enforcement — đó chính là sai lầm PFM vừa bộc lộ.

Tóm lại, phạm vi source chính sẽ là:

```
NEW
  orchestration_ledger.py
  orchestration_control.py
  proof_ledger.py
  orchestration_report.py
  orchestration_otel.py          # later/optional

MODIFY
  server.py
```

```
  task_state.py
  performance_trace.py
  codex_bridge.py

KEEP SEPARATE
  telemetry.py

NEW TESTS
  test_orchestration_ledger.py
  test_orchestration_control.py
  test_proof_ledger.py
  test_orchestration_report.py

EXTEND TESTS
  test_task_state_and_build.py
  test_codex_bridge.py

LATER
  sol-lean-codex skill
```

Đó là ranh giới thay đổi tôi sẽ dùng khi triển khai.

Nguồn

Triển khai hoàn thiện cho tôi

Worked for 46m 52s

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Commit chính:

`42e6737d8440bdaf9dd13a73a6f7367143fdd12e` — `feat: enforce orchestration audit control loop`.

Các thành phần đã triển khai gồm:

- SQLite append-only audit ledger cho `RUN / WAVE / DELEGATION / EVENT / PROOF / CHECKPOINT`.
- EOCL state machine: `PREFLIGHT → READY → ROUTED → EXECUTING → PROVING → ACCEPTING → CHECKPOINTED`, kèm `REPLAN/BLOCKED/ABORTED`.
- Write lease bắt buộc, kiểm tra owner + base HEAD + worktree fingerprint + allowed paths.
- Deep/model failure không được silent fallback; phải `REPLAN` và có `replan_reason`.
- Proof ledger có fingerprint/cache; proof cũ chỉ reuse khi fingerprint khớp.
- Auto-generated `run-summary.md` và `wave-NNN.md`.
- `task_control audit / accept / reject` ở source mới; `accept` chính là final acceptance của ChatWeb Sol.
- `coordinated_multi_worktree` fail-safe: chưa có supervisor adapter đủ proof thì trả `SUPERVISOR_NOT_READY`, **không tự biến controller thành coder**.
- Public smart MCP surface vẫn giữ **8 tools**, không sinh thêm đống tool model-specific.

Validation trước commit:

```
Python MCP tests:       53/53 PASS
Node stability tests:   5/5 PASS
git diff --cached --check: PASS
```

Tôi cũng đã reload native MCP. Lần reload đầu phát hiện thiếu startup env; tôi sửa lại với:

```
CODING_TOOLS_MCP_TOOL_MODE=smart
CODING_TOOLS_MCP_AUTHORIZED_ROOTS=E:\DEV\Tools-Codex-web
```

Native process mới được quan sát LISTENING port `18765` với PID **28264**. Toàn bộ quá trình, lỗi reload và trạng thái acceptance đã được ghi thành deployment record mới:

`docs/research/EOCL_IMPLEMENTATION_AND_DEPLOYMENT_RECORD_2026-08-13.md`

Commit tài liệu:

`571d30fc7237379fdc124d1b988054813d98bb45` — `docs: record EOCL implementation and deployment evidence`.

Skill cũng đã nâng lên v5

Tôi đã sửa `sol-lean-codex` để hiểu đúng runtime mới, gồm:

```
RUN/WAVE lifecycle
write lease
mandatory REPLAN
proof fingerprint/cache
ACCEPTING -> ChatWeb Sol accept/reject
audit reports
coordination readiness
schema-cache handling
route tuning từ durable logs
không log private chain-of-thought
```

Validation:

```
quick_validate:             PASS
route comparator:           10/10 PASS
package_skill validation:   PASS

Files:       14
Size:        58,701 bytes
SHA-256:     858632ab57fe58c07083f2a1344b8169bad54a28a4947c6a11cf934a4159aa87
```

Download skill v5

Còn đúng một lớp chưa thể hoàn tất trong phiên hiện tại

Sau native reload, **MCP tunnel của chính conversation này giữ session-id cũ và trả** `Session terminated`. Đồng thời wrapper hiện tại vẫn cache schema cũ. Vì vậy tôi không thể trung thực nói rằng đã:

- ghi skill v5 mới vào `C:\Users\Admin\.codex\skills\sol-lean-codex`;
- gọi live `task_control audit/accept/reject`;
- cập nhật custom Skill Library của ChatWeb từ phiên này.

Tôi cố ý **không restart Electron/tunnel** chỉ để ép schema refresh vì việc đó mở rộng blast radius không cần thiết. Deployment record cũng ghi các mục này là `PENDING CLIENT RECONNECT`, không giả thành `DONE`.

Như vậy trạng thái hiện nay là:

```
GitHub source EOCL            DONE
EOCL state enforcement        DONE
Audit ledger                  DONE
Write lease                   DONE
Typed REPLAN                  DONE
Proof fingerprint/cache       DONE
Generated wave reports        DONE
Final acceptance protocol     DONE in server source/tests
53 Python tests               PASS
5 Node stability tests        PASS
```

```
Native MCP corrected restart  OBSERVED
Skill v5 package              VALIDATED
-----------------------------------------------
Fresh ChatWeb schema handshake       PENDING
Windows global skill v5 install      PENDING
ChatWeb Skill Library v5 activation  PENDING
Live post-reload EOCL smoke           PENDING
```

Vì vậy chưa nên giao một finance/auth task lớn trong chính conversation này. Sau khi mở một chat mới/MCP reconnect, việc đầu tiên cần làm là kiểm tra schema mới, chạy một fixture `execute → ACCEPTING → audit → accept → CHECKPOINTED`, rồi cài chính bundle v5 trên máy. Sau đó hệ thống mới có thể coi là **end-to-end production accepted**, không chỉ server-deployed.

sol-lean-codex

Đã chỉnh sửa

Evidence-calibrated Sol orchestration for Codex coding, debugging, review, CI repair, refactoring, migrations, research, and long-running repository work.…

Thử trong đoạn chat

Tiếp tục hoàn thiện cho đạt thì thôi

Tôi tiếp tục theo tiêu chí “đạt end-to-end”, không dừng ở server-deployed. Tôi sẽ khóa đủ 4 điều kiện còn thiếu: **fresh MCP schema**, **live EOCL smoke tới** `CHECKPOINTED`, **global Windows skill v5**, và **ChatWeb skill v5 verification/activation**; bất kỳ bước nào fail sẽ được sửa ngay theo evidence thay vì bỏ qua.

Used sol-lean-codex skill

Tested MCP connector reconnection

MCP tunnel hiện đã đổi từ `Session terminated` sang **404 ở tunnel gateway**, tức không còn là lỗi schema cache mà là đường tunnel của phiên này đã bị thu hồi. Tôi sẽ tiếp tục hoàn thiện các lớp không phụ thuộc tunnel trước: xác minh ChatWeb đã thực sự nạp skill v5, kiểm tra bundle/hash, rồi tìm đường khôi phục kết nối MCP mà không đụng dữ liệu repo.

Used sol-lean-codex skill

ChatWeb hiện đã nạp **skill v5 thật**: resource tree có đúng 14 files và `references/eocl-runtime.md`, nên lớp Skill Library đã đạt. Tunnel thì đã mất hẳn route 404; tôi đang kiểm tra xem repo có **self-hosted Windows GitHub runner** hay deployment workflow hay không. Nếu có, tôi có thể dùng chính runner đó để hoàn tất global skill + restart/tunnel recovery mà không cần MCP hiện tại.

Inspected CI workflows and validated Windows workflow job results

Inspected skill package contents

Fetched main branch commit workflows and tree SHA

Generated verified skill package assets and installation scripts

Retrieved first base64 package chunk

Đã nói chuyện với App

Công cụ được gọi

Công cụ được gọi

Hoàn thiện tiếp

Used sol-lean-codex skill

Tôi bắt đầu bằng acceptance gate hiện tại, không sửa thêm code: kiểm tra MCP đã reconnect chưa, schema EOCL có thực sự xuất hiện chưa, rồi mới chạy smoke. Nếu connector vẫn lỗi, tôi sẽ xử lý connection/runtime trước — không gọi vòng lại logic đã test xanh.

Inspected workspace context and connector status

Kết nối MCP đã hồi phục và hiện workspace trực tiếp là `E:\DEV\Tools-Codex-web`, `tool_mode=smart`, HEAD đúng `571d30fc…`. Nhưng tôi phát hiện **2 file EOCL đang dirty sau commit** (`orchestration_control.py`, `test_orchestration_workflow.py`), nên trước live smoke tôi phải xác định đó là fix hợp lệ hay drift ngoài ý muốn; tôi sẽ không bỏ qua vì như vậy smoke sẽ không còn đại diện cho GitHub `main`.

Inspected uncommitted orchestration changes

Hai thay đổi dirty này là **fix hợp lệ và cần thiết**, không phải drift vô nghĩa: `OrchestrationControl` trước đó không cập nhật in-memory snapshot sau transition/new REPLAN wave, nên `task_control audit/accept` có thể trả wave/state cũ dù DB đúng. Test bổ sung chính xác bắt lỗi này ở chuỗi `REPLAN → wave mới → ACCEPTING → accept → CHECKPOINTED`. Tôi sẽ chạy focused + full suite; nếu xanh, tôi sẽ commit fix riêng trước smoke production.

Ran focused orchestration tests

Ran targeted and complete test suites while polling command session

Committed and pushed Python and EOCL state fixes

Used skill-creator skill

coordinator
Verified and listed the skill archive
Tested access to the global skill directory
Generated Windows installation package from Base64-encoded skill archive
Updated global Lean skill installation
Uploaded skill package to Codex Orchestration
Compared extracted skill files and hashes
Calculated global skill file hashes
Generated SKILL.md Base64 payload
Encoded, compressed, and retrieved skill file data
Installed compressed skill file and verified all 14 files via hashing
Started temporary EOCL smoke server
Inspected server status, MCP initialization, sessions, and endpoint configuration
Requested localhost network permission
Fetched latest main branch workflow runs
Identified production listener process details
Inspected coding tools environment variables
Inspected Electron process supervision
Restarted MCP server, refreshed schemas, and tested detached execution
Constructed PowerShell MCP restart script
Created and launched detached MCP restart helper script
Inspected workspace context and listener restart status
Executed EOCL production live smoke test
Updated checkpoint, inspected tool behavior, and completed smoke test continuation
Inspected current task state
Retrieved workspace context and project overview
Inspected workspace task state and completion status
Investigated task status override after acceptance

Inspected and fixed terminal task state preservation

Preserved terminal task states through commands

Verified listener, updated and launched MCP restart helper, reconnecting workspace

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

Công cụ được gọi

k

Công cụ được gọi

ế

m

Công cụ được gọi

ă

k

Công cụ được gọi

h

Công cụ được gọi

ô

n

Công cụ được gọi

g

Công cụ được gọi

đ

ụ

Công cụ được gọi

n

Công cụ được gọi

g

t

Công cụ được gọi

ớ

i

q

u

a

l

i

t

y

f

l

o

o

r

.

5. **N**
   â
   n
   g
   `c`
   `o`
   `m`
   `p`
   `a`
   `r`
   `e`

ChatGPT có thể mắc lỗi. Dữ liệu không gian làm việc không được dùng để huấn luyện mô hình.

_
r
o
u
t
e
_
p
o
l
i
c
i
e
s
.
p
y
t
ừ
p
o
l
i
c
y
-
a
v
e
r
a
g
e
t
h
à
n
h
p
a

i
r
e
d
-
t
a
s
k
a
n
a
l
y
s
i
s
.
S
c
r
i
p
t
h
i
ệ
n
e
n
f
o
r
c
e
p
a
i
r
e
d

c
o
v
e
r
a
g
e
r
ấ
t
t
ố
t
v
à
d
i
s
q
u
a
l
i
f
y
q
u
a
l
i
t
y
f
a
i
l
u
r
e
/

c
r
i
t
i
c
a
l
d
e
f
e
c
t
,
n
h
ư
n
g
P
a
r
e
t
o
đ
a
n
g
d
ự
a
c
h
ủ
y
ế
u
v
à

o
m
e
a
n
c
o
s
t
+
m
e
a
n
w
a
l
l
t
i
m
e
.
H
a
i
đ
i
ể
m
n
ê
n
s
ử
a
:
h
ỗ
t
r

ợ `r u n _ i d / r e p l i c a t e / s e e d`, vì evaluation yêu cầu

r
e
p
e
a
t
n
o
i
s
y
t
a
s
k
s
t
r
o
n
g
k
h
i
s
c
r
i
p
t
h
i
ệ
n
c
ấ
m
d
u
p
l

i
c
a
t
e
(
t
a
s
k
_
i
d
,
p
o
l
i
c
y
)
;
v
à
b
á
o
p
a
i
r
e
d
d
d
e
l
t
a
t
h
e

o `task_class`, median/tail latency cùng `time_to`

_first_dispatch.evaluation.md đã tự cảnh báo a

v
e
r
a
g
e
c
ó
t
h
ể
c
h
e
r
e
g
r
e
s
s
i
o
n
t
r
ê
n
c
r
i
t
i
c
a
l
/
s
e
q
u

e
n
t
i
a
l
w
o
r
k
,
n
ê
n
t
o
o
l
i
n
g
n
ê
n
e
n
f
o
r
c
e
l
u
ô
n
đ
i
ề
u
đ
ó

thay vì để người đọc tự kiểm tra.

Đánh giá cuối

Nế

u
c
h
ỉ
x
é
t
t
h
i
ế
t
k
ế
p
o
l
i
c
y
,
t
ô
i
x
ế
p
n
ó
k
h
o
ả
n
g
9
/
1
0
.
C

á
c
q
u
y
ế
t
đ
ị
n
h
l
ớ
n
đ
ề
u
đ
ú
n
g
:
T
e
r
r
a
k
h
ô
n
g
c
ò
n
l
à
“
f
a
l

l
b
a
c
k
”
m
à
l
à
w
o
r
k
e
r
c
h
í
n
h
;
L
u
n
a
đ
ư
ợ
c
g
i
ớ
i
h
ạ
n
b
ằ
n
g

o
r
a
c
l
e
;
c
a
s
c
a
d
e
c
ó
e
c
o
n
o
m
i
c
g
a
t
e
;
m
u
l
t
i
-
a
g
e
n
t
c

ó
t
o
p
o
l
o
g
y
g
a
t
e
;
S
o
l
g
i
ű
s
e
m
a
n
t
i
c
o
w
n
e
r
s
h
i
p
;
t
e
l

emetryphânbiệtrequested/resolvedrouting.

N
ế
u
x
é
t
đ
ã
c
h
ứ
n
g
m
i
n
h
t
ố
i
ư
u
t
r
ê
n
t
h
ự
c
t
ế
c
h
ư
a
,
m
ớ
i
k

hoảng **7/10**. Validator, unittests và routing

s
c
e
n
a
r
i
o
s
c
h
ứ
n
g
m
i
n
h
l
o
g
i
c
k
h
ô
n
g
t
ự
m
â
u
t
h
u
ẫ
n
,
n
h

ư
n
g
c
h
ư
a
c
h
ứ
n
g
m
i
n
h
t
i
m
e
-
t
o
-
f
i
r
s
t
-
d
i
s
p
a
t
c
h
,
a
c

c
e
p
t
e
d
-
p
a
t
c
h
l
a
t
e
n
c
y
h
a
y
t
o
t
a
l
e
x
p
e
c
t
e
d
c
o
s
t
t
ő

t
h
ơ
n
b
a
s
e
l
i
n
e
t
r
ê
n
w
o
r
k
l
o
a
d
H
R
/
P
F
M
t
h
ự
c
.
C
h
í
n
h
e

v
a
l
u
a
t
i
o
n
c
o
n
t
r
a
c
t
c
ủ
a
s
k
i
l
l
c
ũ
n
g
đ
ò
i
p
a
i
r
e
d
r
e
p

r
e
s
e
n
t
a
t
i
v
e
r
u
n
s
t
r
ư
ớ
c
k
h
i
p
r
o
m
o
t
e
p
o
l
i
c
y
.

V
ì
v

ậ
y
c
â
u
t
r
ả
l
ờ
i
n
g
ắ
n
n
h
ấ
t
l
à
:
s
k
i
l
l
đ
ã
p
h
â
n
t
í
c
h
v
á
n
đ

è
đ
ú
n
g
v
à
c
ó
c
ơ
c
h
ế
g
i
a
o
v
i
ệ
c
n
h
a
n
h
,
n
h
ư
n
g
v
ấ
n
c
ò
n
m
ộ

t
l
ớ
p
c
o
n
t
r
o
l
-
p
l
a
n
e
o
v
e
r
h
e
a
d
c
ó
t
h
ể
c
ắ
t
.
Ư
u
t
i
ê
n
t

i
ế
p
t
h
e
o
k
h
ô
n
g
n
ê
n
t
h
ê
m
i
n
t
e
l
l
i
g
e
n
c
e
h
a
y
t
h
ê
m
r
u
l

e
;
n
ê
n
t
h
ê
m
f
a
s
t
-
p
a
t
h
r
o
u
t
i
n
g
+
l
e
a
n
p
a
c
k
e
t
+
t
e
l
e

m
e
t
r
y
f
i
r
s
t
-
d
i
s
p
a
t
c
h
,
r
ồ
i
b
e
n
c
h
m
a
r
k
.
S
a
u
b
a
t
h
a

y đổi đó mới có thể kết luận bằng số liệu rằng Sol – Te

r
r
a
–
L
u
n
a
t
h
ự
c
s
ự
n
h
a
n
h
h
ơ
n
c
h
ứ
k
h
ô
n
g
c
h
ỉ
“
đ
ư
ợ
c
t
h
i

ế
t
k
ế
đ
ể
n
h
a
n
h
”
.

P
h
â
n
t
í
c
h
k
ỹ
h
ơ
n
v
à
d
ự
a
t
h
e
o
c
á
c
b

á
o
c
á
o
n
g
h
i
ê
n
c
ứ
u
m
ớ
i
c
u
g
n
f
t
h
ả
o
l
u
ậ
n
t
r
ê
n
k
h
ô
n
g
g
i
a
n
m

ạ
n
g
đ
ể
r
a
s
k
i
l
l
t
ố
i
ư
u
h
ơ
n
n
ữ
a

T
ô
i
đ
ã

đ
i
s
â
u
t
h
ê
m
t
h
e
o
c
ả
p
a
p
e
r
m
ớ
i
,
b
e
n
c
h
m
a
r
k
2
0
2
6
,
t
à
i

l
i
ệ
u
O
p
e
n
A
l
h
i
ệ
n
h
à
n
h
v
à
c
á
c
b
á
o
c
á
o
/
t
h
ả
o
l
u
ậ
n
r
u
n

t
i
m
e
t
h
ự
c
t
ế
, r
ồ
i
t
á
i
c
ấ
u
t
r
ú
c
`s`
`o`
`l`
`-`
`l`
`e`
`a`
`n`
`-`
`c`
`o`
`d`
`e`
`x`
t
h
à

nhmộtbảnleanhơn.Kếtluậnquantrọngnhấtlà:**bả**

n
t
ố
i
ư
u
h
ơ
n
k
h
ô
n
g
p
h
ả
i
l
à
r
o
u
t
e
r
p
h
ứ
c
t
ạ
p
h
ơ
n
h
a
y
n
h

i
è
u
a
g
e
n
t
h
ơ
n
;
n
ó
p
h
ả
i
r
a
q
u
y
ế
t
đ
ị
n
h
s
ớ
m
h
ơ
n
,
k
i
ể
m
t

r
a
r
u
n
t
i
m
e
/
o
r
a
c
l
e
t
r
ư
ớ
c
,
g
i
ả
m
c
h
i
p
h
í
c
o
o
r
d
i
n
a
t

i
o
n
v
à
đ
o
đ
ư
ợ
c
c
h
í
n
h
c
h
i
p
h
í
đ
i
ề
u
p
h
ố
i
c
ủ
a
n
ó
.

V
ề
n
g

h
i
ê
n
c
ứ
u
,
c
ó
v
à
i
t
í
n
h
i
ệ
u
m
ớ
i
l
à
m
t
h
a
y
đ
ổ
i
t
h
i
ế
t
k
ế
.

Nghiên cứu về scaling agents systems cho thấy multi

i-agentchỉđangtiềnkhicấutrúccôngviệcthựcs

ự
s
o
n
g
s
o
n
g
;
v
ớ
i
t
á
c
v
ụ
t
u
ầ
n
t
ự
/
t
o
o
l
-
h
e
a
v
y
,
c
o
o
r
d

i
n
a
t
i
o
n
o
v
e
r
h
e
a
d
c
ó
t
h
ể
l
à
m
k
ế
t
q
u
ả
g
i
ả
m
m
ạ
n
h
,
t
r
o

n
g
k
h
i
c
e
n
t
r
a
l
i
z
e
d
c
o
o
r
d
i
n
a
t
i
o
n
k
i
ể
m
s
o
á
t
e
r
r
o
r

propagation tốt hơn. Điều này giúp `single-lane`

`b`
`y`
`d`
`e`
`f`
`a`
`u`
`l`
`t`
, nhưng mạnh hơn bản cũ ở chỗ tôi thêm **har**

d
v
e
t
o
t
r
ư
ớ
c
k
h
i
d
ự
n
g
D
A
G
, t h a y v ì t í n h E V s a u k h i đ ã

bắt đầu cer emony điều phối.

Ở phía routing, LLM

RouterBench2026chothấyrouterphứctạpkhôngm

ặ
c
n
h
i
ê
n
t
h
ắ
n
g
b
a
s
e
l
i
n
e
đ
ơ
n
g
i
ả
n
,
c
ò
n
C
o
n
t
e
x
t
u
a
l

Router chọt thấy nearest-neighbor trên lịch sử ta

s
k
t
ư
ơ
n
g
t
ự
c
ó
t
h
ể
r
ấ
t
c
ạ
n
h
t
r
a
n
h
n
g
a
y
c
ả
k
h
i
l
ị
c
h
s
ử

c
h
ư
a
l
ớ
n
.
M
ặ
t
k
h
á
c
,
m
ộ
t
p
r
e
p
r
i
n
t
r
ấ
t
m
ớ
i
v
ề
x
R
o
u
t
e

Benchlại báo cáo learn edrouting có thể vượt fixe

d
b
a
s
e
l
i
n
e
k
h
i
đ
ư
ợ
c
h
u
ấ
n
l
u
y
ệ
n
/
e
v
a
l
u
a
t
e
đ
ú
n
g
c
á
c

h
.
V
ì
h
a
i
t
í
n
h
i
ệ
u
n
à
y
k
h
ô
n
g
m
â
u
t
h
u
ẫ
n
n
ế
u
n
h
ì
n
t
h
e
o

m
a
t
u
r
i
t
y
c
u
r
v
e
,
b
ả
n
m
ớ
i
d
ù
n
g
s
i
m
p
l
e
f
a
s
t
-
p
a
t
h
+
r

o
u
t
e
l
e
d
g
e
r
t
r
ư
ớ
c
,
v
à
c
h
ỉ
c
h
o
p
h
é
p
l
e
a
r
n
e
d
/
c
a
l
i
b

r
a
t
e
d
r
o
u
t
e
r
x
u
ấ
t
h
i
ệ
n
k
h
i
p
a
i
r
e
d
e
v
i
d
e
n
c
e
c
h
ứ
n
g

m
i
n
h
n
ó
t
h
ắ
n
g
b
a
s
e
l
i
n
e
.

M
ộ
t
t
h
a
y
đ
ổ
i
q
u
a
n
t
r
ọ
n
g

khác đến từ các bench mark repository-level. Rep

o
R
e
a
s
o
n
c
h
ỉ
r
a
b
o
t
t
l
e
n
e
c
k
đ
á
n
g
c
h
ú
ý
n
ằ
m
ở
k
h
ả
n
ă
n
g

t
ổ
n
g
h
ợ
p
n
h
i
ề
u
d
e
p
e
n
d
e
n
c
y
/
s
t
a
t
e
c
ù
n
g
l
ú
c
,
c
h
ứ
k
h

ô
n
g
đ
ơ
n
g
i
ả
n
ở
s
ố
f
i
l
e
;
c
á
c
n
g
h
i
ê
n
c
ứ
u
r
e
p
o
-
r
e
a
s
o
n

i
n
g
k
h
á
c
c
ũ
n
g
c
h
o
t
h
ấ
y
n
ạ
p
n
h
i
ề
u
c
o
n
t
e
x
t
h
ơ
n
k
h
ô
n
g

n
h
ấ
t
t
h
i
ế
t
g
i
ú
p
v
ì
c
o
n
t
e
x
t
n
h
i
ễ
u
v
à
c
a
l
l
-
c
h
a
i
n
k
h

ó
t
r
u
y
h
ồ
i
.
V
ì
v
ậ
y
t
ô
i
đ
ư
a
i
n
t
e
g
r
a
t
i
o
n
w
i
d
t
h
t
h
à
n
h

b
i
ế
n
r
o
u
t
i
n
g
h
ạ
n
g
n
h
ấ
t
:
m
ộ
t
t
h
a
y
đ
ổ
i
3
f
i
l
e
n
h
ư
n
g
l

i
ê
n
q
u
a
n
a
u
t
h
→
p
e
r
s
i
s
t
e
n
c
e
→
b
i
l
l
i
n
g
c
ó
t
h
ể
p
h
ả
i
ở

Sol, trongkhimigration50filevớitransformxá

c
đ
ị
n
h
v
à
c
o
m
p
i
l
e
r
o
r
a
c
l
e
v
ẫ
n
c
ó
t
h
ể
đ
i
t
o
o
l
s
/
L
u
n
a

.



E
n
v
i
r
o
n
m
e
n
t
v
à
o
r
a
c
l
e
c
ũ
n
g
đ
ư
ợ
c
n
â
n
g
l
ê
n
t
r
ư
ớ

cimplementtationon.GitTaskBenchchothấymộtphần

n
l
ớ
n
f
a
i
l
u
r
e
c
ủ
a
c
o
d
i
n
g
a
g
e
n
t
x
u
ấ
t
p
h
á
t
t
ừ
s
e
t
u
p
/

dependency/environment check không phải không ảnh hưởng

v
i
ế
t
c
o
d
e
;
đ
ồ
n
g
t
h
ờ
i
n
g
h
i
ê
n
c
ứ
u
m
ớ
i
v
ề
a
g
e
n
t
-
g
e
n
e

r
a
t
e
d
t
e
s
t
s
c
h
o
t
h
ấ
y
“
v
i
ế
t
n
h
i
ề
u
t
e
s
t
h
ơ
n
”
k
h
ô
n
g
t

ự
đ
ộ
n
g
c
h
u
y
ể
n
t
h
à
n
h
t
ỷ
l
ệ
g
i
ả
i
t
a
s
k
c
a
o
h
ơ
n
.
D
o
đ
ó
f
l

o
w
m
ớ
i
b
ắ
t
đ
ầ
u
b
ằ
n
g
e
n
v
i
r
o
n
m
e
n
t
/
o
r
a
c
l
e
p
r
e
f
l
i
g
h

t
:
x
á
c
n
h
ậ
n
r
e
p
r
o
d
u
c
t
i
o
n
/
c
o
m
p
i
l
e
r
/
t
e
s
t
/
s
c
h
e
m

a
/
r
u
n
t
i
m
e
c
h
e
c
k
t
h
ự
c
s
ự
c
h
ạ
y
đ
ư
ợ
c
r
ồ
i
m
ớ
i
t
r
ả
t
i
ề
n

c
h
o
w
o
r
k
e
r
c
o
d
i
n
g
.
T
e
s
t
t
r
ở
t
h
à
n
h
o
r
a
c
l
e
c
ó
m
ụ
c
đ
í

c
h
,
k
h
ô
n
g
c
ò
n
l
à
r
i
t
u
a
l
b
ắ
t
m
ọ
i
w
o
r
k
e
r
p
h
ả
i
t
ạ
o
t
h
ê

m
.

Tôi cũng thêm một lớp **extension-quality gat**

e
c
h
o
t
a
s
k
d
à
i
.
C
á
c
b
e
n
c
h
m
a
r
k
l
o
n
g
-
h
o
r
i
z
o
n
2
0
2
6
c

h
o
t
h
ấ
y
a
g
e
n
t
c
ó
t
h
ể
đ
ạ
t
c
h
e
c
k
p
o
i
n
t
c
h
ứ
c
n
ă
n
g
t
r
o
n

g
k
h
i
c
o
d
e
b
a
s
e
d
ầ
n
p
h
ì
n
h
,
t
r
ù
n
g
l
ặ
p
v
à
k
h
ó
m
ở
r
ộ
n
g
;

đ
â
y
l
à
l
o
ạ
i
s
u
y
g
i
ả
m
m
à
u
n
i
t
t
e
s
t
x
a
n
h
k
h
ô
n
g
b
ắ
t
đ
ư
ợ

c
.
V
ì
v
ậ
y
S
o
l
k
h
ô
n
g
đ
ư
ợ
c
n
g
h
i
ệ
m
t
h
u
t
a
s
k
d
à
i
c
h
ỉ
b
ằ
n

g
b
e
h
a
v
i
o
r
g
a
t
e
:
v
ớ
i
r
e
p
e
a
t
e
d
/
c
r
o
s
s
-
c
u
t
t
i
n
g
w
o

r
k
,
n
ó
p
h
ả
i
k
i
ể
m
t
r
a
t
h
ê
m
d
u
p
l
i
c
a
t
i
o
n
,
c
o
m
p
l
e
x
i
t

y
c
o
n
c
e
n
t
r
a
t
i
o
n
,
i
n
t
e
r
f
a
c
e
a
c
c
r
e
t
i
o
n
v
à
a
r
c
h
i
t

e
c
t
u
r
a
l
d
r
i
f
t
—
n
h
ư
n
g
c
h
ỉ
ở
s
u
r
f
a
c
e
đ
ã
c
h
ạ
m
,
k
h
ô
n
g

biến thành clean up vô hạn.

OpenAI hiện cũng khu

y
ế
n
n
g
h
ị
l
e
a
n
p
r
o
m
p
t
,
P
r
o
g
r
a
m
m
a
t
i
c
T
o
o
l
C
a
l
l
i
n
g

c
h
o
c
h
u
ỗ
i
t
o
o
l
d
e
t
e
r
m
i
n
i
s
t
i
c
v
à
m
u
l
t
i
-
a
g
e
n
t
c
h
o

w
o
r
k
s
t
r
e
a
m
p
h
â
n
r
ã
s
ạ
c
h
;
h
ư
ớ
n
g
d
ẫ
n
m
o
d
e
l
h
i
ệ
n
t
ạ
i

n
h
ấ
n
m
ạ
n
h
p
h
ả
i
b
e
n
c
h
m
a
r
k
e
f
f
o
r
t
t
r
ê
n
w
o
r
k
l
o
a
d
t
h

ậ
t
t
h
a
y
v
ì
g
i
ả
đ
ị
n
h
m
ộ
t
e
f
f
o
r
t
c
ố
đ
ị
n
h
t
ố
i
ư
u
.
B
ả
n
m
ớ

i
v
ì
t
h
ế
ư
u
t
i
ê
n
p
r
o
g
r
a
m
m
a
t
i
c
/
m
e
c
h
a
n
i
c
a
l
e
x
e
c
u
t

i
o
n
t
r
ư
ớ
c
w
o
r
k
e
r
,
g
i
ữ
p
r
o
m
p
t
/
p
a
c
k
e
t
n
g
ắ
n
v
à
k
h
ô
n

g
d
ù
n
g
e
f
f
o
r
t
c
a
o
h
ơ
n
đ
ể
b
ù
c
h
o
r
o
u
t
e
s
a
i
t
ầ
n
g
.

M
ộ

t
t
í
n
h
i
ệ
u
r
ấ
t
t
h
ự
c
t
ế
t
ừ
c
ộ
n
g
đ
ồ
n
g
/
r
u
n
t
i
m
e
n
g
à
y
1
2

/
8
/
2
0
2
6
l
à
r
o
u
t
e
l
o
g
i
c
k
h
ô
n
g
đ
ủ
:
r
o
u
t
e
c
ò
n
p
h
ả
i
t
h

ự
c
t
h
i
đ
ư
ợ
c
t
r
ê
n
đ
ú
n
g
b
i
n
a
r
y
đ
a
n
g
c
h
ạ
y
.
l
s
s
u
e
h
i
ệ
n

tại của `openai/codex` bị ostandalone `0.147.0` cót

hể spawn Luna trong một cấu hình mảng binary 0.147.0

-alpha.6.5 bundled với VSCode lại chỉ expose Sol

/
T
e
r
r
a
.
C
á
c
b
á
o
c
á
o
t
r
ư
ớ
c
đ
ó
c
ũ
n
g
c
h
o
t
h
ấ
y
m
o
d
e
l
c
a

t
a
l
o
g
c
ủ
a
m
a
i
n
s
e
s
s
i
o
n
v
à
a
l
l
o
w
l
i
s
t
c
ủ
a
s
u
b
a
g
e
n
t

c
ó
t
h
ể
l
ệ
c
h
n
h
a
u
.
V
ì
v
ậ
y
t
ô
i
c
h
u
y
ể
n
t
ừ
“
m
o
d
e
l
-
a
w
a
r

e
r
o
u
t
e
r
”
s
a
n
g
c
a
p
a
b
i
l
i
t
y
-
g
a
t
e
d
r
o
u
t
e
r
:
r
e
q
u
e
s

t
e
d
m
o
d
e
l
c
h
ỉ
l
à
ý
đ
ị
n
h
;
r
e
s
o
l
v
e
d
m
o
d
e
l
/
r
u
n
t
i
m
e
m

e
t
a
d
a
t
a
m
ớ
i
l
à
s
ự
t
h
ậ
t
.
N
ế
u
k
h
ô
n
g
q
u
a
n
s
á
t
đ
ư
ợ
c
c
h
i

l
d
r
o
u
t
e
,
s
k
i
l
l
k
h
ô
n
g
đ
ư
ợ
c
t
u
y
ê
n
b
ố
đ
ã
c
h
ạ
y
L
u
n
a
/
T

e
r
r
a
,
v
à
k
h
i
d
ự
t
o
á
n
c
h
i
p
h
í
p
h
ả
i
d
ù
n
g
t
i
e
r
k
h
ả
d
ĩ
c
a

o
h
ơ
n
t
h
a
y
v
ì
g
i
ả
đ
ị
n
h
o
p
t
i
m
i
s
t
i
c
.

B
ả
n
s
k
i
l
l
m
ớ
i
đ

ãđượcthayđổithеocácđiểmchínhsauu:

- **Fast-pat**

h
r
o
u
t
e
r
t
r
ư
ớ
c
f
u
l
l
r
o
u
t
e
r
:
`t`
`o`
`o`
`l`
`→`
`S`
`o`
`l`
`c`
`r`
`i`
`t`
`i`
`c`
`a`
`l`
`/`
`s`

h
a
r
e
d
→
L
u
n
a
s
t
r
o
n
g
-
o
r
a
c
l
e
a
f
→
T
e
r
r
a
o
t
h
e
r
w
i
s

e

.

C

h

ỉ

m

ở

c

l

a

s

s

i

f

i

c

a

t

i

o

n

đ

ầ

y

đ

ủ

k

h

i

r

o

u

t

e

t

h

ậ

t

s

ự

m

ơ
h
ồ
,
đ
a
n
g
c
â
n
e
f
f
o
r
t
/
c
a
s
c
a
d
e
/
b
u
d
g
e
t
h
o
ặ
c
t
e
l
e
m

etrycóthểđổiquyếtđịnh.

- **Integrationwidth+se**

r
i
a
l
i
t
y
+
t
o
o
l
d
e
n
s
i
t
y
+
e
n
v
i
r
o
n
m
e
n
t
r
e
a
d
i
n
e
s
s
đ

ư
ợ
c
b
ổ
s
u
n
g
v
à
o
f
u
l
l
c
l
a
s
s
i
f
i
e
r
.
F
i
l
e
c
o
u
n
t
/
r
e
p
o

s i z e k h ô n g c ò n l à p r o x y c h í n h .

- **H a r d m u l t i - a g e n t v**

e
t
o
c
h
ạ
y
t
r
ư
ớ
c
t
o
p
o
l
o
g
y
p
l
a
n
n
i
n
g
.
S
e
q
u
e
n
t
i
a
l
r
e

asoning, tight tool/model loop, shared mutable

h
o
t
s
p
o
t
,
w
e
a
k
o
r
a
c
l
e
,
h
i
g
h
i
n
t
e
g
r
a
t
i
o
n
w
i
d
t
h
h
o

ặ
c
r
e
v
i
e
w
b
a
c
k
l
o
g
đ
ề
u
g
i
ữ
s
i
n
g
l
e
-
l
a
n
e
.

- **A**
d
a
p
t
i
v
e

p
a
c
k
e
t
c
o
n
t
r
a
c
t
:
l
e
a
f
t
h
ư
ờ
n
g
c
h
ỉ
c
ò
n
`O`
`B`
`J`
`E`
`C`
`T`
`I`
`V`
`E`
`/`

SCOPE/CONTEXT/PROOF/STOP.

Full packet chỉ dùn

g
c
h
o
c
o
n
c
u
r
r
e
n
c
y
,
c
a
s
c
a
d
e
,
c
r
i
t
i
c
a
l
w
o
r
k
,
u
n
s
t

a b l e i n t e r f a c e h o ặ c n h i ề u w o r k t r e e e .

- **C a p a b i l i t y**

m
a
p
t
h
e
o
e
x
a
c
t
r
u
n
t
i
m
e
:
l
ư
u
r
e
q
u
e
s
t
e
d
/
r
e
s
o
l
v
e
d

childroute,effort,direct/named/separatese

s
s
i
o
n
s
u
p
p
o
r
t
,
i
s
o
l
a
t
i
o
n
v
à
l
i
v
e
n
e
s
s
;
k
h
ô
n
g
p
r
o

b
e
l
ạ
i
m
ỗ
i
t
a
s
k
.

- **C**
a
s
c
a
d
e
t
r
ở
n
ê
n
k
h
ó
k
í
c
h
h
o
ạ
t
h
ơ
n
:

n
g
o
à
i
m
o
d
e
l
c
o
s
t
c
ò
n
t
í
n
h
r
e
p
e
a
t
e
d
c
o
n
t
e
x
t
,
v
a
l
i

d
a
t
i
o
n
,
h
a
n
d
o
f
f
,
r
e
w
o
r
k
v
à
l
a
t
e
n
c
y
p
e
n
a
l
t
y
.
F
a
i

l
u
r
e
l
à
m
l
ộ
s
e
m
a
n
t
i
c
a
m
b
i
g
u
i
t
y
l
à
t
í
n
h
i
ệ
u
r
o
u
t
e
t

r
ự
c
t
i
ế
p
l
ê
n
t
i
e
r
t
r
ê
n
,
k
h
ô
n
g
r
e
t
r
y
c
a
s
c
a
d
e
.

- **L**
o
n
g

-horizonqualitygate: nghiệm thu thêm khả năng m

ở
r
ộ
n
g
c
ủ
a
p
h
ầ
n
c
o
d
e
v
ừ
a
c
h
ạ
m
,
k
h
ô
n
g
c
h
ỉ
c
o
r
r
e
c
t
n
e

s
s
h
i
ệ
n
t
ạ
i
.

- **E**
v
a
l
u
a
t
i
o
n
m
ớ
i
đ
o
o
v
e
r
h
e
a
d
c
ủ
a
c
h
í
n
h

orchestrator: thêm `TIME_TO_FIRST_DISPATCH`, S

o
l
t
o
k
e
n
s
/
c
o
o
r
d
i
n
a
t
i
o
n
t
u
r
n
s
t
r
ư
ớ
c
d
i
s
p
a
t
c
h
,
p

a
c
k
e
t
-
b
u
i
l
d
c
o
s
t
,
i
n
t
e
g
r
a
t
i
o
n
w
i
d
t
h
,
o
r
a
c
l
e
s
t

r
e
n
g
t
h
,
e
n
v
i
r
o
n
m
e
n
t
r
e
a
d
i
n
e
s
s
v
à
m
a
i
n
t
a
i
n
a
b
i
l

i
t
y
.

- **C**
o
m
p
a
r
a
t
o
r
đ
ư
ợ
c
n
â
n
g
c
á
p
:
h
ỗ
t
r
ợ
`r`
`u`
`n`
`_`
`i`
`d`
/
r
e
p

l
i
c
a
t
e
,
m
a
i
n
t
a
i
n
a
b
i
l
i
t
y
g
a
t
e
,
p
9
0
w
a
l
l
t
i
m
e
,
f
i

r
s
t
-
d
i
s
p
a
t
c
h
l
a
t
e
n
c
y
,
p
a
i
r
e
d
d
e
l
t
a
v
ớ
i
`
-
-
b
a
s
e
l

`i`
`n`
`e`
v
à
b
r
e
a
k
d
o
w
n
`_`
`-`
`b`
`y`
`-`
`t`
`a`
`s`
`k`
`-`
`c`
`l`
`a`
`s`
`s`
.
N
h
ờ
đ
ó
p
o
l
i
c
y

k
h
ô
n
g
t
h
ể
t
h
ắ
n
g
c
h
ỉ
n
h
ờ
m
e
a
n
c
o
s
t
đ
ẹ
p
t
r
o
n
g
k
h
i
t
h
u

a
m
ộ
t
c
l
a
s
s
q
u
a
n
t
r
ọ
n
g
.

- **L**
e
a
r
n
e
d
r
o
u
t
i
n
g
b
ị
t
r
ì
h
o
ã

n
c
ó
c
h
ủ
ý
:
r
o
u
t
e
l
e
d
g
e
r
+
n
e
a
r
e
s
t
-
s
i
m
i
l
a
r
-
t
a
s
k
t

r
ư
ớ
c
;
c
h
ỉ
n
â
n
g
s
a
n
g
r
o
u
t
e
r
p
h
ứ
c
t
ạ
p
k
h
i
p
a
i
r
e
d
t
e
l

emetrychothấyParetoimprovementthựcsự.

Cóm

ộ
t
đ
i
ể
m
đ
á
n
g
c
h
ú
ý
v
ề
`A`
`G`
`E`
`N`
`T`
`S`
`.`
`m`
`d`
/
r
e
p
o
s
i
t
o
r
y
i
n
s
t
r

u
c
t
i
o
n
s
:
b
ằ
n
g
c
h
ứ
n
g
2
0
2
6
h
i
ệ
n
c
h
ư
a
đ
ồ
n
g
n
h
ấ
t
.
M
ộ
t

nghiên cứu ghi nhận instruction file sẽ tăng

exploration/inference cost mà không cải thiện

u
c
c
e
s
s
,
t
r
o
n
g
k
h
i
m
ộ
t
n
g
h
i
ê
n
c
ứ
u
t
h
ự
c
n
g
h
i
ệ
m
k
h
á
c

t
r
ê
n
P
R
w
o
r
k
f
l
o
w
l
ạ
i
g
h
i
n
h
ậ
n
g
i
ả
m
r
u
n
t
i
m
e
/
o
u
t
p
u

t
t
o
k
e
n
s
.
T
ô
i
k
h
ô
n
g
h
a
r
d
-
c
o
d
e
k
ế
t
l
u
ậ
n
“
A
G
E
N
T
S
.
m

d
t
ố
t
”
h
a
y
“
x
ấ
u
”
;
s
k
i
l
l
m
ớ
i
c
o
i
r
e
p
o
s
i
t
o
r
y
g
u
i
d
a
n

c
e
l
à
i
n
p
u
t
c
ầ
n
t
u
â
n
t
h
ủ
n
h
ư
n
g
y
ê
u
c
ầ
u
i
n
s
t
r
u
c
t
i
o
n

s
p
h
ả
i
n
g
ắ
n
,
d
e
c
i
s
i
o
n
-
b
e
a
r
i
n
g
v
à
đ
ư
ợ
c
k
i
ể
m
c
h
ứ
n
g

c
ụ
c
b
ộ
.
Đ
â
y
l
à
c
á
c
h
a
n
t
o
à
n
h
ơ
n
k
h
i
l
i
t
e
r
a
t
u
r
e
c
h
ư
a

hộitụ.

Về câu hỏi bàn đầu của bạn — **đã phân tích và**

à
g
i
a
o
v
i
ệ
c
n
h
a
n
h
c
h
ư
a
?
”
—
b
ả
n
m
ớ
i
t
ố
t
h
ơ
n
r
õ
r
ệ
t
ở
đ
ú

n
g
đ
i
ể
m
n
à
y
.
B
ả
n
t
r
ư
ớ
c
c
ó
t
h
ể
đ
i
q
u
a
c
l
a
s
s
i
f
i
c
a
t
i
o

n
7
c
h
i
ề
u
r
ồ
i
d
ự
n
g
p
a
c
k
e
t
t
ư
ơ
n
g
đ
ố
i
đ
ầ
y
đ
ủ
t
r
ư
ớ
c
k
h
i

g
i
a
o
m
ộ
t
l
e
a
f
đ
ơ
n
g
i
ả
n
.
B
ả
n
m
ớ
i
c
ó
t
h
ể
r
o
u
t
e
m
ộ
t
l
e
a

f
r
õ
r
à
n
g
s
a
u
f
i
n
i
s
h
-
l
i
n
e
+
o
r
a
c
l
e
/
e
n
v
i
r
o
n
m
e
n
t
c

h
e
c
k
m
à
k
h
ô
n
g
m
ở
t
o
à
n
b
ộ
r
o
u
t
i
n
g
p
o
l
i
c
y
.
Q
u
a
n
t
r
ọ
n

g
h
ơ
n
,
n
ó
g
i
ờ
đ
o
t
r
ự
c
t
i
ế
p
`T`
`I`
`M`
`E`
`_`
`T`
`O`
`_`
`F`
`I`
`R`
`S`
`T`
`_`
`D`
`I`
`S`
`P`
`A`
`T`
`C`

H
v
à
c
o
o
r
d
i
n
a
t
i
o
n
c
o
s
t
t
r
ư
ớ
c
d
i
s
p
a
t
c
h
,
n
ê
n
s
a
u
m
ộ

t
t
ậ
p
t
a
s
k
t
h
ự
c
t
é
t
a
c
ó
t
h
ể
c
h
ứ
n
g
m
i
n
h
S
o
l
đ
ã
n
h
a
n
h
h

ơ
n
b
a
o
n
h
i
ê
u
t
h
a
y
v
ì
đ
á
n
h
g
i
á
c
ả
m
t
í
n
h
.

T
ô
i
v
ẫ
n
k
h
ô

n
g
g
ắ
n
n
h
ã
n
“
g
l
o
b
a
l
o
p
t
i
m
u
m
”
.
K
h
ô
n
g
p
a
p
e
r
h
a
y
b
e
n

c
h
m
a
r
k
c
ô
n
g
k
h
a
i
n
à
o
c
ó
t
h
ể
c
h
ứ
n
g
m
i
n
h
p
o
l
i
c
y
t
ố
i
ư

u
c
h
o
d
i
s
t
r
i
b
u
t
i
o
n
H
R
/
P
F
M
h
o
ặ
c
r
e
p
o
c
ụ
t
h
ể
c
ủ
a
b
ạ
n

.
T
r
ạ
n
g
t
h
á
i
t
ố
t
h
ơ
n
c
ủ
a
s
k
i
l
l
n
à
y
l
à
c
a
l
i
b
r
a
t
i
o
n
-

r
e
a
d
y
:
c
o
l
d
-
s
t
a
r
t
p
o
l
i
c
y
đ
ã
m
ạ
n
h
,
c
o
n
t
r
o
l
-
p
l
a
n

e
n
h
ệ
h
ơ
n
v
à
h
ệ
t
h
ố
n
g
đ
o
đ
ủ
đ
ể
t
ự
l
o
ạ
i
r
o
u
t
e
k
h
ô
n
g
h
i
ệ

u
q
u
ả
s
a
u
d
ữ
l
i
ệ
u
t
h
ự
c
.

K
ế
t
q
u
ả
k
i
ể
m
t
r
a
c
u
ố
i
:
s
k
i
l

l
v
a
l
i
d
a
t
o
r
đ
ạ
t
,
9
/
9
u
n
i
t
t
e
s
t
s
đ
ạ
t
,
P
y
t
h
o
n
c
o
m
p
i

l
e
đ
ạ
t
,
k
h
ô
n
g
c
ò
n
c
á
c
n
h
ã
n
c
ũ
`S`
`o`
`l`
`-`
`L`
`u`
`n`
`a`
,
`L`
`u`
`n`
`a`
`m`
`a`
`x`
h
a

y Terra, package giữ đúng root solana-codex/ và cập nhật skill cho máy Cluôn chotôi

c
ó
đ
ầ
y
đ
ủ
1
2
f
i
l
e
.

Tải bản
`skill.zip`
đã tối ưu

Lưu thay đổi

Thử trong đoạn chat

Nguồn